
Machine Learning

A Beginner's Guide

Treats every formula gently.
Assumes only basic linear algebra and elementary calculus.
Each symbol is unpacked the first time it appears.

Topics covered	38
Style	Verbose, conversational, beginner-first
Math prerequisites	Comfortable with high-school algebra
What you don't need	Calculus or matrix algebra in advance
What we will skip	Derivations and proofs

If a formula scares you, stop and read the box right after it.
Every formula in this guide is followed by a sentence-by-sentence unpacking.

How to Read This Guide

This document is organised in 8 parts and 38 topics. Each topic follows the same recipe so you always know what to expect:

1. **Why we care** – a tiny story telling you what real-world problem this idea solves.
2. **The big idea** – the concept in plain English, before any math.
3. **The formula** – written out and then *decoded symbol by symbol*. Don't skip the decoder; it is the whole point.
4. **Worked example** – an actual numerical calculation you can follow on paper.
5. **Exam focus** – what is most likely to come up on a test.
6. **Common confusions** – mistakes that students reliably make, called out so you don't make them.

A note on math notation. Mathematicians love single-letter symbols because they keep formulas short. Sadly that makes them look like alphabet soup to a beginner. We promise: every symbol used in this guide will be explained the first time it appears, and the explanations will be repeated when the symbol re-appears in a different context.

What we will skip. Almost all derivations and proofs. This guide is about *understanding what the formulas do*, not about deriving them. When a formula simply pops out of nowhere, that's deliberate – you can take a derivation course later.

Contents

How to Read This Guide	1
I Foundations of Machine Learning	10
Part I – Foundations	10
1 Introduction to AI and Machine Learning	10
1.1 The big idea	10
1.2 Three flavours of machine learning	10
1.3 The standard ML workflow	11
1.4 The vocabulary you will see in every chapter	11
2 Supervised vs. Unsupervised Learning	13
2.1 Supervised learning – in more detail	13
2.2 Unsupervised learning – in more detail	13
2.3 Other paradigms (brief mention)	14
3 Reinforcement Learning	15
3.1 The pieces of an RL problem	15
3.2 The interaction loop	15
3.3 The total reward we want to maximise	16
3.4 Value functions	16
3.5 Exploration versus exploitation	16
4 Background: Linear Algebra and Calculus You Need	18
4.1 Vectors – a list of numbers	18
4.2 Adding and scaling vectors	18
4.3 Dot product – a single number measuring similarity	19
4.4 Length of a vector (norm)	20
4.5 Matrices – a grid of numbers	20
4.6 Transpose – flipping rows and columns	20
4.7 Matrix multiplication	21
4.8 Inverse of a matrix	21
4.9 Eigenvalues and eigenvectors – in plain English	22
4.10 Derivatives – in plain English	23

4.11	Convex vs. non-convex functions	24
5	Linear Regression	26
5.1	Simple linear regression – one feature	26
5.2	The slope and intercept formulas	26
5.3	Multiple linear regression – many features	27
5.4	The loss function: Mean Squared Error	27
5.5	The standard assumptions	28
6	Ordinary Least Squares (OLS)	30
6.1	Setting up the data in matrix form	30
6.2	The MSE loss in matrix form	30
6.3	The OLS formula	31
6.4	Geometric picture – the projection interpretation	31
6.5	When the formula breaks	32
7	Probability – The Bare Minimum	34
7.1	Random variables	34
7.2	Expected value – the average	34
7.3	Variance – the spread	35
7.4	Covariance and correlation	35
7.5	Joint, marginal, and conditional probability	35
7.6	Bayes’ rule – inversion of probabilities	36
7.7	The Gaussian (normal) distribution	36
7.8	Common distributions you’ll meet	37
7.9	Two laws that make statistics work	37
8	Statistical Framework for Linear Regression	38
8.1	The probabilistic model behind linear regression	38
8.2	Maximum Likelihood Estimation – “find the parameters that make the data most plausible”	38
8.3	Maximum a Posteriori (MAP) estimation	39
8.4	Estimator quality – bias, variance, MSE	39
9	Hypothesis Testing	41
9.1	Null and alternative hypotheses	41
9.2	Two kinds of mistake	41
9.3	t -test for a regression coefficient	41

9.4	p -value – the verdict	42
9.5	Confidence interval	42
9.6	F -test – testing several coefficients at once	42
10	Goodness of Fit	44
10.1	Three sums of squares	44
10.2	R^2 – the headline metric	44
10.3	The big trap: R^2 never decreases	45
10.4	Adjusted R^2	45
10.5	Residual standard error (RSE)	45
10.6	Diagnostic plots to inspect	45
II	Model Assessment, Selection and Resampling	47
Part II	– Model Selection & Resampling	47
11	Model Assessment and the Bias–Variance Tradeoff	47
11.1	Why training error is misleading	47
11.2	Bias and variance, defined intuitively	48
11.3	The bias–variance decomposition	48
11.4	The trade-off in a table	49
12	Subset Selection – Choosing Which Features to Use	51
12.1	The combinatorial nightmare	51
12.2	Best subset selection – the gold standard (when feasible)	51
12.3	Forward stepwise selection – start small, grow	51
12.4	Backward stepwise selection – start big, shrink	52
12.5	Choosing among models of different sizes	52
13	Cross-Validation	53
13.1	The basic idea: k -fold cross-validation	53
13.2	Visualising 5-fold CV	54
13.3	Leave-One-Out CV (LOOCV)	54
13.4	Choosing k	54
13.5	What CV is used for	54

14 The Bootstrap	56
14.1 The bootstrap procedure	56
14.2 Out-of-bag (OOB) samples	57
15 In-Sample Selection: AIC, BIC, C_p	58
15.1 The pattern: training RSS + complexity penalty	58
15.2 The four criteria	58
15.3 When to use which	59
III Optimization for Machine Learning	60
Part III – Optimization	60
16 Gradient Descent	60
16.1 The update rule	60
16.2 Picture	61
16.3 Three flavours: batch, stochastic, mini-batch	61
16.4 Choosing the learning rate η	61
17 Convex Optimization	63
17.1 What does “convex” mean?	63
17.2 Tests for convexity	63
17.3 Why convexity is the dream	63
17.4 Convex losses in ML	64
18 Primal Problems	65
18.1 Standard form of a constrained problem	65
18.2 The hard-margin SVM primal	65
18.3 The soft-margin SVM primal	66
19 Dual Problems	67
19.1 The Lagrangian – folding constraints into the objective	67
19.2 The dual function	67
19.3 Weak and strong duality	67
19.4 The SVM dual – the punchline	68
20 KKT Conditions	69
20.1 The four KKT conditions	69

20.2 What KKT tells us about SVM support vectors	69
IV Classification, Perceptron and Support Vector Machines	71
Part IV – Classification & SVM	71
21 Classification – The Big Picture	71
21.1 The setup	71
21.2 The 0-1 loss and accuracy	71
21.3 The confusion matrix	72
21.4 The ROC curve and AUC	72
22 The Perceptron Algorithm	74
22.1 The model and the learning rule	74
22.2 Convergence – the Novikoff theorem	74
22.3 The big limitation: XOR	75
23 Hard-Margin Classification	76
23.1 What is the margin?	76
23.2 The hard-margin SVM optimisation	76
23.3 The catch	77
24 Soft-Margin SVM	78
24.1 The optimisation problem	78
24.2 Equivalent “hinge loss” formulation	78
24.3 Identifying support vectors after training	79
25 Kernel Functions	80
25.1 The idea	80
25.2 Mercer’s theorem – when is k a valid kernel?	80
25.3 Common kernels	80
25.4 Picture	81
26 Kernel SVM	83
26.1 The kernelised dual	83
26.2 The decision function	83
26.3 Why kernel SVM works so well	84

V	Regularized Regression	85
Part V	– Regularized Regression	85
27	Ridge Regression	85
27.1	The Ridge objective	85
27.2	The closed-form Ridge solution	85
27.3	Properties of Ridge	86
27.4	Geometric picture	86
27.5	Connection to MAP estimation	86
28	LASSO	88
28.1	The LASSO objective	88
28.2	Why LASSO produces zeros – the geometric reason	88
28.3	Soft thresholding (when X is orthonormal)	89
28.4	Ridge vs. LASSO – when to use which	89
28.5	Elastic Net – the best of both	89
29	Kernelized Ridge Regression	91
29.1	The representer theorem	91
29.2	The closed-form solution	91
29.3	Putting it all together	91
VI	Probabilistic Classification	93
Part VI	– Probabilistic Classification	93
30	Logistic Regression	93
30.1	The sigmoid function	93
30.2	The model	94
30.3	The cross-entropy loss	94
30.4	Logistic regression as a discriminative model	95
31	Gaussian Discriminant Analysis (GDA / LDA)	96
31.1	The generative model	96
31.2	Estimating parameters by MLE	96
31.3	Predicting with Bayes' rule	97

31.4 LDA vs. logistic regression	97
32 Naive Bayes	99
32.1 The naive assumption	99
32.2 Classification rule	99
32.3 Variants	100
32.4 Laplace smoothing – the zero-frequency rescue	100
VII Ensemble Methods	102
Part VII – Ensemble Methods	102
33 Bagging (Bootstrap Aggregating)	102
33.1 The bagging algorithm	102
33.2 Why does bagging reduce variance?	103
33.3 Out-of-bag (OOB) error – a free validation set	103
33.4 Bias of bagging	103
34 Random Forests	104
34.1 The random forest algorithm	104
34.2 Default values for m	104
34.3 Why feature subsampling helps	104
34.4 Hyperparameters and defaults	105
34.5 Why random forests are popular	105
35 Boosting (AdaBoost)	106
35.1 The AdaBoost algorithm	106
35.2 Bagging vs. boosting at a glance	107
35.3 The math made concrete	107
35.4 When boosting overfits	107
VIII Unsupervised Learning	109
Part VIII – Unsupervised Learning	109
36 Unsupervised Learning – Big Picture	109
36.1 Three core tasks	109

36.2 The big difficulty: evaluation without labels	109
37 K-Means Clustering	111
37.1 The objective: total within-cluster squared distance	111
37.2 Lloyd's algorithm	111
37.3 A worked example in 1D	112
37.4 How do you pick K ?	112
37.5 Limitations of k -means	112
38 Principal Component Analysis (PCA)	114
38.1 The big idea	114
38.2 The procedure	114
38.3 Variance explained – how to choose k	115
38.4 Eigenvalues of a 2×2 matrix (so you can do it on paper)	115
38.5 Properties to remember	116
Appendix: Reference Material	118
Appendix – Reference	118
A. Glossary	118
B. Formula Cheat Sheet	125
C. Common Confusions Recap	126
D. Pre-Exam Checklist	126

Part I

Foundations of Machine Learning

1 Introduction to AI and Machine Learning

Difficulty: Easy. Mostly vocabulary. • **How to read:** Read once. Memorise the four-part workflow.

Why We Care

You want a computer program that recognises whether an email is spam. You *could* sit down and try to write down every rule: “if the email contains the word LOTTERY, mark it as spam”; “if the sender’s address has more than three numbers, suspicious”; and so on. But spam keeps changing. Tomorrow’s spam will use words you haven’t thought of. Hand-coded rules break down fast.

What if, instead, you showed the program a few thousand examples of spam and not-spam, and let it figure out the rules itself? That, in one sentence, is the idea behind machine learning.

1.1 The big idea

Artificial Intelligence (AI) is a huge field about building machines that do things humans find “intelligent”: understand language, recognise faces, plan a route, play chess.

Machine Learning (ML) is a part of AI in which the machine *learns from examples* instead of being told the rules. Show the machine many examples of houses and their prices, and it will figure out how to predict the price of a new house. Show it many photos labelled as “cat” or “dog”, and it will figure out what makes a cat look like a cat.

Deep learning is a part of machine learning that uses very large, multi-layer networks of simple computing units (“neurons”) to learn from massive amounts of data. We won’t cover deep learning specifically in this guide, but everything you learn here is the foundation it stands on.

1.2 Three flavours of machine learning

There are three main *paradigms* – three different ways an ML system can be set up depending on what kind of feedback it gets while learning.

- **Supervised learning.** You give the system input/output pairs and ask it to learn the mapping. “Here are 1000 emails, each labelled spam or not-spam – now classify a new email.” The “supervision” is the labels.

- **Unsupervised learning.** You give only inputs, no labels, and ask the system to find structure on its own. “Here are 1000 customer profiles – group similar customers together.” Nobody tells the system what counts as a group; it has to figure that out.
- **Reinforcement learning.** An “agent” interacts with an environment, takes actions, gets rewards or punishments, and learns a strategy that maximises long-run reward. Think of training a dog with treats, or a robot learning to walk.

We will spend most of this guide on supervised and unsupervised learning, with one chapter on reinforcement learning.

1.3 The standard ML workflow

Almost every supervised-learning project follows the same five steps:

1. **Collect data** – a bunch of (input, output) examples.
2. **Pick a hypothesis class** – the family of models you’ll consider (e.g. “all straight lines”, “all decision trees of depth at most 5”).
3. **Define a loss function** – a number that says how badly the model is currently doing. Smaller = better.
4. **Optimise** – find the model in your chosen family that has the smallest loss on your data. This is where calculus / gradient descent / closed-form formulas come in.
5. **Evaluate** – test the trained model on *new* data it hasn’t seen, to estimate how well it will do in the real world.

1.4 The vocabulary you will see in every chapter

Memorise these terms now – they are used everywhere:

- **Feature** (or **input**, or **predictor**) – something you observe about an example. For a house: square footage, age, number of bedrooms. We usually write a single feature as x . When there are several features we write $x_1, x_2, \dots, x_p$ (where p is the total number of features).
- **Label** (or **target**, or **response**) – the thing you are trying to predict. House price, email spam-ness, etc. We usually write the label as y .
- **Training set** – the examples your model learns from. Often written $\{(x_i, y_i)\}$ for $i = 1, 2, \dots, n$, where n is the number of training examples.
- **Test set** – examples held back from training, used only to evaluate the trained model.
- **Hypothesis class** – the set of all possible models you are choosing from. Sometimes written $\mathcal{H}$.
- **Loss function** – a number measuring how wrong the current model is. Often written L or ℓ .
- **Generalisation** – how well the model performs on new, unseen data. This is what we actually care about; performance on the training set is misleading.

Intuition

The single most important idea in ML. We do *not* care about how well a model fits the training data. We care about how well it predicts *new* data. A model that perfectly

memorises the training set but does poorly on new data is useless. Most of the techniques in this guide – regularisation, cross-validation, ensemble methods – exist to manage this distinction.

Practice Problem

Classify each scenario as supervised learning, unsupervised learning, or reinforcement learning.

1. Predicting tomorrow's stock price from the last 30 days of prices.
2. Grouping news articles into "topics" nobody has labelled.
3. A self-driving car learning to merge onto a highway through trial and error in simulation.
4. A bank predicting whether a loan applicant will default, given past records of (applicant features, did they default).
5. Compressing a 1,000-pixel image into a 50-dimensional code that captures the essence.

Solution

1. Supervised (regression). Inputs: 30 days of prices. Output: tomorrow's price (a number).
2. Unsupervised (clustering). No labels were given.
3. Reinforcement. Trial and error with rewards.
4. Supervised (binary classification). Output: defaulted, yes/no.
5. Unsupervised (dimensionality reduction). No target variable.

2 Supervised vs. Unsupervised Learning

Difficulty: Easy. Vocabulary and categories. • **How to read:** Read once. The classification table is the take-away.

Why We Care

Two situations:

Situation A. A bank has 10 years of loan records. For each loan they know who the customer was, their income, their credit score, and whether they ended up paying the loan back. They want to use this data to predict whether a *new* loan applicant will default.

Situation B. A streaming service has 50 million users and the list of shows each one watched. They have no labels at all – just viewing histories. They want to discover “types” of viewers (binge watchers, sports fans, anime obsessives) for marketing purposes.

A is supervised: we have inputs (income, credit score) AND outputs (paid back or defaulted). B is unsupervised: we have only inputs (viewing histories) and no pre-defined “types” to predict.

2.1 Supervised learning – in more detail

In a supervised problem you have n training examples, each consisting of an *input* and an *output*:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

Reading this notation. x_i is the input for example i – it might be a single number, or it might be a list of numbers (square footage, age, bedrooms, ...). y_i is the output for the same example. The subscript i is just an index into the data set.

The goal is to find a function $\hat{f}$ (read “ f -hat”) that takes a new input x and produces a good guess for y :

$$\hat{y} = \hat{f}(x).$$

The hat means “estimate” or “prediction”. $\hat{y}$ is the model’s guess; y would be the (unknown) true value.

Two flavours of supervised learning depending on what kind of output y is:

- **Regression:** y is a number. House price, exam score, tomorrow’s temperature. We measure errors with something like $(y - \hat{y})^2$.
- **Classification:** y is a label from a small set. Spam vs. not-spam (2 classes), or digit 0–9 (10 classes), or image-tag (millions of classes). We measure errors with the fraction of wrong predictions, or with cross-entropy.

2.2 Unsupervised learning – in more detail

In an unsupervised problem you have only inputs:

$$x_1, x_2, \dots, x_n.$$

No labels. Your job is to discover something interesting about these inputs. Three common tasks:

- **Clustering** – group similar inputs together. “Find the five types of viewers in our streaming data.” Output: a cluster ID for each input.
- **Dimensionality reduction** – represent each input with fewer numbers while keeping its essence. An image might be 1,000,000 pixels; we might compress it to 50 numbers that still capture most of the visible information. Output: a smaller-dimensional version of each input.
- **Density estimation** – learn the underlying probability distribution of the inputs. Output: a function that tells you, for any new x , how likely it is.

2.3 Other paradigms (brief mention)

- **Semi-supervised learning** – a small amount of labelled data and a large amount of unlabelled data. Common in practice because labelling is expensive.
- **Self-supervised learning** – the system invents its own labels from the data. Modern language models work this way: they learn by predicting the next word from previous words.
- **Active learning** – the system asks a human to label only the most informative examples, saving labelling effort.

Common Confusion

“**Clustering is just classification without labels.**” No. Classification predicts a label from a fixed, pre-defined set (“spam”, “not-spam”). Clustering invents groups whose meaning isn’t pre-defined – “cluster 3” is just an integer until you go look at what’s in it.

Practice Problem

For each task, say (a) supervised vs unsupervised, and (b) if supervised, regression vs classification.

1. Predict a customer’s lifetime value in dollars from their first three months of activity.
2. Group photographs into “probably the same person” clusters.
3. Decide if a tumour is benign or malignant given a biopsy report.
4. Rank search results in order of relevance.
5. Estimate house price from 20 features.

Solution

1. Supervised, regression (output is a number).
2. Unsupervised (no labels, just photos).
3. Supervised, binary classification.
4. Supervised, ranking (a special form of classification with ordering).
5. Supervised, regression.

3 Reinforcement Learning

Difficulty: Moderate. RL has its own vocabulary. • **How to read:** Read carefully. The MDP tuple and the discounted return are the take-aways.

Why We Care

Imagine teaching a robot to walk. You can't easily provide labelled training examples ("in this exact body position, the right action is to move joint 7 by 0.4 radians"). What you *can* do is let the robot try things, fall over a lot, and reward it whenever it stays upright for longer. Over many trials it figures out a walking strategy.

That setup – agent acts, environment responds, agent gets a scalar reward – is reinforcement learning.

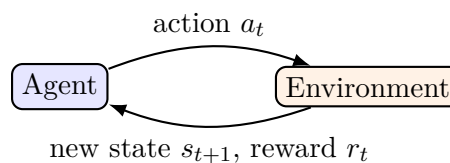
3.1 The pieces of an RL problem

An RL problem has the following ingredients:

- **State** s – the current situation the agent is in. For chess, the state is the configuration of pieces on the board.
- **Action** a – a choice the agent makes. For chess, picking a move.
- **Reward** r – a number the environment hands back after the action. Could be +1 for winning, -1 for losing, 0 otherwise.
- **Policy** π – the agent's strategy: a rule that says, for each state s , which action a to take. Written $\pi(a | s)$ when the choice is random, or $\pi(s) = a$ when deterministic.
- **Discount factor** γ – a number between 0 and 1 that decides how much future rewards matter compared to immediate ones. $\gamma = 0$: only the next reward counts. γ close to 1: the agent is patient and looks far ahead.

3.2 The interaction loop

At each time step t :



1. Agent observes the state s_t .
2. Agent picks an action a_t using its policy π .
3. Environment computes the next state s_{t+1} and a reward r_t .
4. Repeat from step 1.

3.3 The total reward we want to maximise

The agent's goal is to pick actions that maximise the **discounted return**:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

Reading This Formula

- G_t – the total “score” starting from time t . G stands for “gain”.
- r_t – the reward you get right now.
- γr_{t+1} – the next reward, scaled down by γ because it happens later.
- $\gamma^2 r_{t+2}$ – the reward two steps from now, scaled even smaller because it's even further away.

The dots “...” mean we keep going forever. Because $\gamma < 1$, the terms shrink fast and the sum is a finite number.

Why discount? Two reasons. First, it captures the intuition that immediate rewards are usually preferable to far-future ones. Second, it guarantees the infinite sum is well-defined; without discounting it could blow up to infinity.

3.4 Value functions

The **state-value function** $V^\pi(s)$ tells you how good state s is, assuming you follow policy π from then on:

$$V^\pi(s) = \mathbb{E}[G_t \mid s_t = s].$$

The **action-value function** $Q^\pi(s, a)$ tells you how good it is to take action a in state s and follow π afterwards:

$$Q^\pi(s, a) = \mathbb{E}[G_t \mid s_t = s, a_t = a].$$

Reading This Formula

- $\mathbb{E}[\cdot]$ means “expected value of” – the long-run average if you played out the situation many times (because rewards and transitions can be random).
- $\mid$ means “given” or “conditional on”. So $\mathbb{E}[G_t \mid s_t = s]$ is the expected total reward, *given* that we are starting in state s .

Difference between V and Q . V asks “how good is this state?” (averaged over the actions π would pick). Q asks “how good is taking this specific action in this state?”. Q is more fine-grained, which is why most modern RL algorithms work with Q .

3.5 Exploration versus exploitation

Suppose you know that action a_1 usually gives reward +5 and action a_2 has only been tried once and gave +3. Should you keep picking a_1 (*exploit* what you know) or try a_2 a few more times to make sure it isn't actually better (*explore*)?

This trade-off is fundamental to RL. A classic compromise is the ϵ -**greedy** policy:

- With probability $1 - \epsilon$, pick the best action you know (exploit).

- With probability ε , pick a random action (explore).

A typical ε is 0.1, meaning the agent explores about 10% of the time.

Intuition

RL differs from supervised learning in three big ways:

1. Feedback is a single number (reward), not a correct label.
2. Data points are sequential and correlated, not independent.
3. Your actions *determine* which data you see next – there is no fixed training set.

These differences make RL much harder than supervised learning, but also applicable to many problems supervised learning cannot touch.

Practice Problem

An agent receives rewards $r_0 = 1$, $r_1 = 2$, $r_2 = 4$, $r_3 = 0$, and then $r_t = 0$ for all subsequent t . Compute the discounted return G_0 for two values of the discount factor:

1. $\gamma = 0$
2. $\gamma = 0.5$

Solution

$$G_0 = r_0 + \gamma r_1 + \gamma^2 r_2 + \gamma^3 r_3 + \dots$$

(1) $\gamma = 0$: $G_0 = 1 + 0 + 0 + 0 = 1$. The agent only values the immediate reward.

(2) $\gamma = 0.5$: $G_0 = 1 + (0.5)(2) + (0.5)^2(4) + (0.5)^3(0) = 1 + 1 + 1 + 0 = 3$.

4 Background: Linear Algebra and Calculus You Need

Difficulty: Hard for an absolute beginner. The single most important chapter to internalise.

- **How to read:** Read all of it. Do every practice problem.

This is the math chapter. We promised the prerequisite was minimal – so here is everything you actually need, explained from scratch. If a piece of math feels foreign elsewhere in the guide, come back here.

The chapter is structured as:

1. Vectors and what we do with them.
2. Matrices and matrix operations.
3. Calculus you'll need (derivatives, gradients, convexity).

4.1 Vectors – a list of numbers

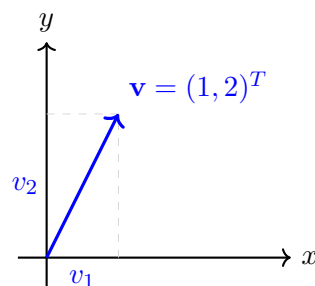
A **vector** is just a list of numbers, written as a column:

$$\mathbf{v} = \begin{pmatrix} 3 \\ 1 \\ 4 \end{pmatrix}.$$

We write the vector in bold ($\mathbf{v}$) and its entries as v_1, v_2, v_3 . This particular vector has 3 entries; we say it “lives in $\mathbb{R}^3$ ” (read: “R-three”), the set of all 3-dimensional real vectors.

Why vectors matter in ML. One example with several features *is* a vector. A house with 3 features (area, age, bedrooms) might be the vector $(2000, 15, 3)^T$. The superscript T stands for “transpose” (more on that in a moment); writing it lets us put the vector inline as a row instead of a column to save paper.

Geometric picture. A 2-vector is an arrow in the plane from the origin to the point (v_1, v_2) .



4.2 Adding and scaling vectors

You add two vectors of the same length entry-by-entry:

$$\begin{pmatrix} 3 \\ 1 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 5 \\ 0 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \\ 4 \end{pmatrix}.$$

You multiply a vector by a number (a “scalar”) by multiplying every entry:

$$2 \cdot \begin{pmatrix} 3 \\ 1 \\ 4 \end{pmatrix} = \begin{pmatrix} 6 \\ 2 \\ 8 \end{pmatrix}.$$

That’s it.

Practice Problem

Given $\mathbf{u} = (1, 4)^T$ and $\mathbf{v} = (-2, 3)^T$, compute:

1. $\mathbf{u} + \mathbf{v}$
2. $3\mathbf{u}$
3. $2\mathbf{u} - \mathbf{v}$

Solution

1. $\mathbf{u} + \mathbf{v} = (1 + (-2), 4 + 3)^T = (-1, 7)^T$.
2. $3\mathbf{u} = (3, 12)^T$.
3. $2\mathbf{u} = (2, 8)^T$, so $2\mathbf{u} - \mathbf{v} = (2 - (-2), 8 - 3)^T = (4, 5)^T$.

4.3 Dot product – a single number measuring similarity

Given two vectors of the same length, their **dot product** is what you get when you multiply matching entries and add them up:

$$\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3.$$

Example:

$$\begin{pmatrix} 3 \\ 1 \\ 4 \end{pmatrix} \cdot \begin{pmatrix} 2 \\ 5 \\ 0 \end{pmatrix} = 3 \cdot 2 + 1 \cdot 5 + 4 \cdot 0 = 11.$$

We will also see this written as $\mathbf{u}^T \mathbf{v}$, which means *exactly* the same thing.

What does it mean intuitively? The dot product measures how “aligned” two vectors are. When they point in the same direction, the dot product is big and positive. When they point in opposite directions, it’s big and negative. When they’re at right angles, it’s zero.

Practice Problem

Compute the dot products:

1. $(1, 2, 3)^T \cdot (4, 0, -1)^T$
2. $(1, 0, 0)^T \cdot (0, 1, 0)^T$. What does the answer say geometrically?
3. $(2, 3)^T \cdot (2, 3)^T$. What well-known quantity does this equal?

Solution

1. $(1)(4) + (2)(0) + (3)(-1) = 4 + 0 - 3 = 1$.
2. $(1)(0) + (0)(1) + (0)(0) = 0$. These two vectors are perpendicular – which makes sense; they point along different axes.

3. $(2)(2) + (3)(3) = 4 + 9 = 13$. This equals $\|\mathbf{v}\|_2^2$, the squared length. In general $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|_2^2$.

4.4 Length of a vector (norm)

The **length** (or **norm**) of a vector $\mathbf{v}$ is its distance from the origin:

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}.$$

The double bars mean “length”. This is just the Pythagorean theorem extended to more dimensions.

There are other “norms” too:

- $\|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n|$ – sum of absolute values. Sometimes called the “Manhattan distance” because it’s the distance you’d walk on a city grid.
- $\|\mathbf{v}\|_2 = \sqrt{v_1^2 + \cdots + v_n^2}$ – ordinary Euclidean length. When we write $\|\mathbf{v}\|$ without a subscript, we mean this.
- $\|\mathbf{v}\|_\infty = \max_i |v_i|$ – the largest absolute entry.

You’ll see all three when we get to regularisation (Topics 27–28).

Practice Problem

For $\mathbf{v} = (3, -4, 12)^T$, compute $\|\mathbf{v}\|_1$, $\|\mathbf{v}\|_2$, and $\|\mathbf{v}\|_\infty$.

Solution

- $\|\mathbf{v}\|_1 = |3| + |-4| + |12| = 3 + 4 + 12 = 19$.
- $\|\mathbf{v}\|_2 = \sqrt{9 + 16 + 144} = \sqrt{169} = 13$.
- $\|\mathbf{v}\|_\infty = \max(3, 4, 12) = 12$.

4.5 Matrices – a grid of numbers

A **matrix** is a rectangular grid of numbers. An $m \times n$ matrix has m rows and n columns:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \quad (\text{this is } 2 \times 3).$$

Why matrices matter in ML. Your entire training data set fits naturally into a matrix. If you have n examples each with p features, your data is an $n \times p$ matrix X . The rows are examples; the columns are features.

4.6 Transpose – flipping rows and columns

The transpose of a matrix swaps rows and columns. If A is 2×3 , then A^T is 3×2 :

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}, \quad A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

For a vector, transpose flips between column form and row form.

Why we need it. Matrix multiplication has shape rules (next section); transpose lets us reshape things so the rules are satisfied.

4.7 Matrix multiplication

You can multiply an $m \times n$ matrix by an $n \times k$ matrix to get an $m \times k$ matrix. The middle numbers (both n) must match!

The entry in row i , column j of the result is the dot product of row i of the first matrix with column j of the second:

Tiny example.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 \\ 6 \end{pmatrix} = \begin{pmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{pmatrix} = \begin{pmatrix} 17 \\ 39 \end{pmatrix}.$$

Important warning. $AB \neq BA$ in general. Matrix multiplication is not commutative.

In ML. Predicting on a single example $\mathbf{x}$ with weight vector $\boldsymbol{\theta}$ is exactly the dot product $\boldsymbol{\theta}^T \mathbf{x}$. Predicting on the entire training set at once is the matrix-vector product $X\boldsymbol{\theta}$, where each row of X is one example.

Practice Problem

Let $A = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}$, $C = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$.

1. Compute $A\mathbf{b}$.
2. Compute A^T .
3. Compute AC .

Solution

1. $A\mathbf{b} = \begin{pmatrix} (1)(4) + (2)(5) \\ (3)(4) + (0)(5) \end{pmatrix} = \begin{pmatrix} 14 \\ 12 \end{pmatrix}.$
2. $A^T = \begin{pmatrix} 1 & 3 \\ 2 & 0 \end{pmatrix}$ (rows of A become columns).
3. $AC = \begin{pmatrix} (1)(1) + (2)(2) & (1)(0) + (2)(1) \\ (3)(1) + (0)(2) & (3)(0) + (0)(1) \end{pmatrix} = \begin{pmatrix} 5 & 2 \\ 3 & 0 \end{pmatrix}.$

4.8 Inverse of a matrix

For some square matrices A , there is another square matrix called the **inverse** A^{-1} with the property

$$A^{-1}A = AA^{-1} = I,$$

where I is the **identity matrix** – 1's on the diagonal, 0's elsewhere – which acts like the number 1 for matrices.

For matrices, “inverse” plays the role that “divide by” plays for numbers.

Important. Not every matrix has an inverse. A matrix without an inverse is called **singular**. Roughly: a matrix is singular if two of its rows (or columns) “carry the same information” – one is a multiple of another, or one is a sum of the others. This will become a real issue when we discuss multicollinearity in linear regression.

Inverse of a 2×2 matrix. For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$,

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix},$$

provided $ad - bc \neq 0$. The number $ad - bc$ is the **determinant**. When the determinant is zero, the matrix is singular and the inverse doesn't exist.

Practice Problem

Compute the inverse of $A = \begin{pmatrix} 4 & 2 \\ 1 & 3 \end{pmatrix}$, then verify $A^{-1}A = I$.

Solution

$$\det(A) = (4)(3) - (2)(1) = 10.$$

$$A^{-1} = \frac{1}{10} \begin{pmatrix} 3 & -2 \\ -1 & 4 \end{pmatrix} = \begin{pmatrix} 0.3 & -0.2 \\ -0.1 & 0.4 \end{pmatrix}.$$

$$\text{Verify: } A^{-1}A = \frac{1}{10} \begin{pmatrix} (3)(4) + (-2)(1) & (3)(2) + (-2)(3) \\ (-1)(4) + (4)(1) & (-1)(2) + (4)(3) \end{pmatrix} = \frac{1}{10} \begin{pmatrix} 10 & 0 \\ 0 & 10 \end{pmatrix} = I. \checkmark$$

4.9 Eigenvalues and eigenvectors – in plain English

For a square matrix A , an **eigenvector** $\mathbf{v}$ is a special vector: when A acts on it, the result is just $\mathbf{v}$ stretched (or shrunk) by some scalar λ :

$$A\mathbf{v} = \lambda\mathbf{v}.$$

The number λ is the **eigenvalue** – the stretch factor. $\mathbf{v}$'s direction doesn't change; only its length does.

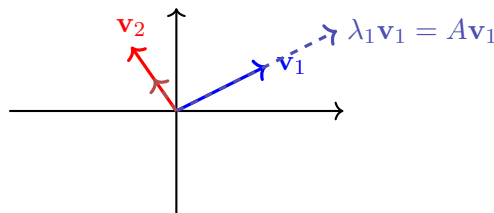
Why we care. Many ML methods (PCA, the kernel trick, spectral clustering) work by finding the eigenvectors of an important matrix. We'll meet this in detail in Topic 38 (PCA).

How to find eigenvalues for a 2×2 matrix. For $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, solve the **characteristic polynomial**:

$$\lambda^2 - (a + d)\lambda + (ad - bc) = 0.$$

This is a quadratic in λ ; use the quadratic formula or factor.

Geometric picture. A deforms the plane. Most vectors get rotated by the deformation. But there are special directions whose vectors only get stretched, never rotated – those are the eigendirections, and the stretch factor is the eigenvalue.



Practice Problem

Find the eigenvalues of $A = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$.

Solution

Characteristic polynomial: $\lambda^2 - (3 + 2)\lambda + (3 \cdot 2 - 1 \cdot 0) = \lambda^2 - 5\lambda + 6 = 0$.

Factor: $(\lambda - 2)(\lambda - 3) = 0$.

Eigenvalues: $\lambda_1 = 3$, $\lambda_2 = 2$.

Shortcut for triangular matrices: eigenvalues are simply the diagonal entries.

Positive (semi-)definite matrices. A symmetric matrix is **positive semi-definite** (PSD), written $A \succeq 0$, when all its eigenvalues are ≥ 0 . **Positive definite** ($A \succ 0$) when all eigenvalues are > 0 . Two matrices we'll meet that are always PSD: $X^T X$ (used in OLS) and covariance matrices (used in PCA).

4.10 Derivatives – in plain English

For a function $f(x)$ of one variable, the **derivative** $f'(x)$ at a point x tells you how fast f is changing there.

- $f'(x) > 0$: f is increasing as x grows.
- $f'(x) < 0$: f is decreasing as x grows.
- $f'(x) = 0$: f is locally flat – this is where minima and maxima happen.

Derivatives you should know.

$f(x)$	$f'(x)$
c (constant)	0
x^n	$n x^{n-1}$
e^x	e^x
$\ln x$	$1/x$
$f(x) + g(x)$	$f'(x) + g'(x)$
$c \cdot f(x)$	$c \cdot f'(x)$
$f(g(x))$	$f'(g(x)) \cdot g'(x)$ (chain rule)

Practice Problem

Compute the derivatives.

1. $f(x) = 3x^2 + 5x - 7$
2. $f(x) = (2x + 1)^3$ (chain rule)
3. $f(x) = \ln(x^2 + 1)$ (chain rule)

Solution

1. $f'(x) = 6x + 5$.
2. Outer is u^3 , inner is $u = 2x + 1$. $f'(x) = 3u^2 \cdot 2 = 6(2x + 1)^2$.
3. Outer is $\ln u$, inner is $u = x^2 + 1$. $f'(x) = (1/u) \cdot 2x = \frac{2x}{x^2 + 1}$.

For a function $f(x_1, x_2, \dots, x_n)$ of *several* variables, the analogue of the derivative is a vector of partial derivatives, called the **gradient**, written ∇f (the upside-down triangle is called “nabla”):

$$\nabla f = \begin{pmatrix} \partial f / \partial x_1 \\ \partial f / \partial x_2 \\ \vdots \\ \partial f / \partial x_n \end{pmatrix}.$$

$\partial f / \partial x_i$ (“partial f over partial x_i ”) is the derivative of f with respect to one variable while pretending all the others are constants.

Geometric meaning. The gradient ∇f at a point $\mathbf{x}$ is the direction in which f *increases fastest*. Its length is how fast it’s increasing. The opposite direction $(-\nabla f)$ is the fastest *decrease* – which is what gradient descent will exploit.

Practice Problem

Compute the gradient of $f(x, y) = x^2 + 3xy + y^2$ at the point $(x, y) = (1, 2)$.

Solution

$$\partial f / \partial x = 2x + 3y. \quad \partial f / \partial y = 3x + 2y.$$

$$\text{At } (1, 2): \quad \nabla f = (2 + 6, 3 + 4)^T = (8, 7)^T.$$

This vector points in the direction of fastest *increase* of f . To minimise, move opposite to the gradient: $-\nabla f = (-8, -7)^T$.

4.11 Convex vs. non-convex functions

A function f is **convex** if its graph looks like a bowl: any line segment between two points on the graph lies on or above the graph. Examples: $f(x) = x^2$ is convex; $f(x) = -\ln x$ (for $x > 0$) is convex.

A non-convex function has hills and valleys; it can have many local minima. Examples: $f(x) = x^4 - 3x^2 + 1$ has two valleys.

Intuition

Why convexity is the dream. If your loss function is convex and you find a point where the gradient is zero, you have found the *global* minimum – there are no other valleys. Most of the classical ML losses (linear regression, logistic regression, SVMs) are convex by design, which is a big reason they're popular. Neural network losses are NOT convex; that's why training a deep model is trickier and depends on smart optimisation tricks.

Practice Problem

Determine whether each function is convex on its natural domain.

1. $f(x) = e^x$
2. $f(x) = \cos x$
3. $f(x) = x^2 - 4x + 7$

Solution

1. $f''(x) = e^x > 0$ everywhere. **Convex.**
2. $f''(x) = -\cos x$, which is negative on $(-\pi/2, \pi/2)$ and positive on $(\pi/2, 3\pi/2)$. **Not convex.**
3. $f''(x) = 2 \geq 0$. **Convex.** (It's a parabola opening upward.)

5 Linear Regression

Difficulty: Foundational. Easy idea, fundamental machinery. • **How to read:** Read carefully. Linear regression is the launching pad for the next 6 topics.

(Reminder: Vectors and dot product was introduced in Topic 4.)

Why We Care

You want to predict the price of a house based on its size. You have data on 100 past sales: for each house you know the area in square feet (x) and the price in dollars (y). When you plot the data, the points roughly form a line: bigger houses tend to cost more. Linear regression draws “the best line” through this cloud of points and then uses it to predict prices for new houses.

5.1 Simple linear regression – one feature

When there’s only one feature x , linear regression assumes the relationship between x and y is a straight line:

$$\hat{y} = \beta_0 + \beta_1 x.$$

Reading This Formula

- $\hat{y}$ – the model’s predicted value of y for input x . The hat reminds you it’s an estimate, not the truth.
- x – your one feature (e.g. square footage).
- β_0 – the **intercept**. This is the predicted y when $x = 0$. (β is the Greek letter “beta”.)
- β_1 – the **slope**. How much $\hat{y}$ goes up when you increase x by 1.

A line in 2D has exactly two parameters: where it crosses the y -axis, and how steep it is. Linear regression’s job is to learn β_0 and β_1 from the data.

5.2 The slope and intercept formulas

For one feature, there are simple closed-form formulas for the best β_0 and β_1 :

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

Reading This Formula

- $\bar{x}$ – the average of all the x_i ’s. $\bar{y}$ – average of the y_i ’s. (x -bar and y -bar.)
- $\sum_{i=1}^n$ – the sum from $i = 1$ to $i = n$. We add up whatever follows. “ $\sum$ ” is the capital Greek letter sigma.
- $(x_i - \bar{x})(y_i - \bar{y})$ – when an x above average pairs with a y above average, this is positive (they move together). When one is above and the other below, it’s negative. Summing these up captures whether x and y tend to move in the same direction.

- $(x_i - \bar{x})^2$ – the spread of the x -values around their mean. We divide by it so the slope is correctly scaled.

The formula for $\hat{\beta}_0$ is just enforcing that the fitted line passes through the centre of mass $(\bar{x}, \bar{y})$ of the data.

Worked Example

$x = (1, 2, 3, 4, 5), \quad y = (3, 4, 2, 5, 6).$

$\bar{x} = 3, \bar{y} = 4.$

Numerator: $(1-3)(3-4) + (2-3)(4-4) + (3-3)(2-4) + (4-3)(5-4) + (5-3)(6-4)$
 $= 2 + 0 + 0 + 1 + 4 = 7.$

Denominator: $(1-3)^2 + (2-3)^2 + (3-3)^2 + (4-3)^2 + (5-3)^2 = 4 + 1 + 0 + 1 + 4 = 10.$

$\hat{\beta}_1 = 7/10 = 0.7, \quad \hat{\beta}_0 = 4 - 0.7 \cdot 3 = 1.9.$

Fitted line: $\hat{y} = 1.9 + 0.7x.$

For a new house with $x = 6$, our prediction is $\hat{y} = 1.9 + 0.7 \cdot 6 = 6.1.$

5.3 Multiple linear regression – many features

Real problems have many features. House price depends on size, age, bedrooms, location, Multiple linear regression assumes

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_d x_d.$$

Reading This Formula

- $x_1, x_2, \dots, x_d$ – the d features (size, age, ...).
- θ_0 – intercept (we now use θ instead of β just because it's a different chapter; same idea).
- $\theta_1, \dots, \theta_d$ – one weight per feature. θ_j tells you how $\hat{y}$ changes when you increase feature j by 1 unit, holding the others fixed.

Compact form: we can write the same equation as $\hat{y} = \theta^T x$, where θ and x are vectors. x has a leading 1 prepended (so the dot product reproduces θ_0):

$$x = (1, x_1, x_2, \dots, x_d)^T, \quad \theta = (\theta_0, \theta_1, \dots, \theta_d)^T.$$

5.4 The loss function: Mean Squared Error

How do we judge whether a particular θ is good? We compute the prediction $\hat{y}_i = \theta^T x_i$ for every training example, and look at how far off it is from the truth y_i . The standard measure is **Mean Squared Error**:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \theta^T x_i)^2.$$

Reading This Formula

- $L(\theta)$ – “the loss as a function of θ ”. Different weights produce different predictions which produce different losses.
- $\frac{1}{n}$ – divide by n so the loss is an *average* error per example, not a sum.
- $(y_i - \theta^T x_i)^2$ – the error on example i , squared. We square because (a) we want errors below and above to count equally; (b) squaring makes the math nicer.

The whole formula reads: “the average squared error across the training set”.

Our goal is to find the θ that makes $L(\theta)$ as small as possible. In the next topic we’ll see exactly how.

5.5 The standard assumptions

When we use linear regression for inference (computing p -values, confidence intervals), we assume the data was actually generated like this:

$$y_i = \theta^T x_i + \varepsilon_i,$$

where ε_i is random noise that satisfies:

- the relationship is truly linear (no missing curvature);
- the noise has mean 0 (no systematic bias);
- the noise has the same variance for every example (called **homoscedasticity**);
- the noise is uncorrelated across examples;
- the columns of X are linearly independent (no two features carry identical information).

If all five hold, OLS (next topic) is the “Best Linear Unbiased Estimator” (BLUE) – meaning it has the smallest possible variance among all linear unbiased methods.

Practice Problem

You collect 5 data points:

$$x = (2, 4, 6, 8, 10), \quad y = (3, 7, 8, 12, 15).$$

Fit a simple linear regression line $\hat{y} = \beta_0 + \beta_1 x$.

1. Compute $\bar{x}$ and $\bar{y}$.
2. Compute $\hat{\beta}_1$ using the slope formula.
3. Compute $\hat{\beta}_0$.
4. Predict $\hat{y}$ at $x = 7$.

Solution

$$(1) \bar{x} = (2 + 4 + 6 + 8 + 10)/5 = 6. \quad \bar{y} = (3 + 7 + 8 + 12 + 15)/5 = 9.$$

(2) Compute deviations:

i	$x_i - \bar{x}$	$y_i - \bar{y}$	product	$(x_i - \bar{x})^2$
1	-4	-6	24	16
2	-2	-2	4	4
3	0	-1	0	0
4	2	3	6	4
5	4	6	24	16

Sums: $\sum(x_i - \bar{x})(y_i - \bar{y}) = 58$, $\sum(x_i - \bar{x})^2 = 40$.

$\hat{\beta}_1 = 58/40 = 1.45$.

(3) $\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 9 - 1.45 \cdot 6 = 9 - 8.7 = 0.3$.

Fitted line: $\hat{y} = 0.3 + 1.45x$.

(4) At $x = 7$: $\hat{y} = 0.3 + 1.45 \cdot 7 = 10.45$.

6 Ordinary Least Squares (OLS)

Difficulty: Foundational. Most famous formula in stats. • **How to read:** Memorise the formula and the shape rules.

(Reminder: Matrix multiplication and inverse was introduced in Topic 4.)

Why We Care

Topic 5 ended with: “find the θ that minimises the squared error.” This topic is the answer. Astonishingly, in linear regression we can write down the optimal θ in a single closed-form formula – no iteration, no gradient descent, just one matrix calculation. This formula is called the **normal equations**, and the resulting estimator is called **Ordinary Least Squares (OLS)**.

6.1 Setting up the data in matrix form

Take your n training examples and stack them as rows of a big matrix:

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1d} \\ 1 & x_{21} & x_{22} & \cdots & x_{2d} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{nd} \end{pmatrix}.$$

Reading This Formula

- X is the **design matrix**. It's $n \times (d + 1)$ – n rows (one per example) and $d + 1$ columns (d features plus the leading 1 column for the intercept).
- Row i is example i 's feature vector.
- Column j is the values of feature j across all examples.

The leading column of 1's is a trick. It lets the intercept θ_0 behave like a weight on a constant feature, so we can write $X\theta$ instead of $\theta_0 + X\theta_{\text{rest}}$.

We also stack all the labels into a column vector

$$y = (y_1, y_2, \dots, y_n)^T.$$

And the parameters into another column vector

$$\theta = (\theta_0, \theta_1, \dots, \theta_d)^T.$$

6.2 The MSE loss in matrix form

When we compute $X\theta$ we get a column of all the predictions: $X\theta = (\theta^T x_1, \theta^T x_2, \dots, \theta^T x_n)^T$.

The vector of errors is then $X\theta - y$. Squaring and summing the errors is the same as computing the squared length of that vector:

$$L(\theta) = \|X\theta - y\|_2^2 = \sum_{i=1}^n (y_i - \theta^T x_i)^2.$$

(We dropped the $1/n$ factor; minimising L or L/n gives the same θ .)

6.3 The OLS formula

Setting the gradient of L with respect to θ equal to zero gives the **normal equations**:

$$X^T X \theta = X^T y.$$

If $X^T X$ is invertible, we can solve for θ directly:

Key Formula

$$\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y$$

Reading This Formula

This is the most famous formula in statistics. Let me decode every piece.

- $\hat{\theta}_{\text{OLS}}$ – our estimated parameter vector. “OLS” = “Ordinary Least Squares”.
- X is your $n \times (d+1)$ data matrix. y is your column of n labels.
- X^T – X transposed. Rows become columns. Shape changes from $n \times (d+1)$ to $(d+1) \times n$.
- $X^T X$ – a matrix multiplication. Shape: $(d+1) \times (d+1)$. This is a small square matrix that summarises how each feature relates to every other feature – specifically, its (i, j) entry is the dot product of column i and column j of X .
- $(X^T X)^{-1}$ – the inverse of that small square matrix. For matrices, the inverse plays the role of “divide by”. This inverse exists only when no two columns of X are linearly dependent.
- $X^T y$ – a multiplication producing a $(d+1) \times 1$ column. Its entries are the dot products of each column of X with y .
- Multiply: $(X^T X)^{-1} (X^T y)$ – this is a $(d+1) \times (d+1)$ matrix times a $(d+1) \times 1$ vector, giving a $(d+1) \times 1$ vector. Those $d+1$ numbers are your weights $\hat{\theta}_0, \hat{\theta}_1, \dots, \hat{\theta}_d$.

What to remember: OLS is just one matrix inverse and two matrix multiplications. Software does this for you. You should recognise the formula on sight.

6.4 Geometric picture – the projection interpretation

There’s a beautiful geometric picture of what OLS does. Imagine your labels y as a single point in n -dimensional space. The set of *all possible* prediction vectors $X\theta$ (as θ varies) is a flat “plane” in this space called the **column space** of X .

OLS finds the point on this plane that is *closest* to y in the squared-distance sense. Geometrically, this is the perpendicular projection of y onto the plane. The formula $\hat{y} = X\hat{\theta}_{\text{OLS}} = Hy$, where $H = X(X^T X)^{-1} X^T$, is exactly the projection. H is called the **hat matrix** because it puts the hat on y ($\hat{y} = Hy$).

Intuition

The picture: y is somewhere out in space. The model can only produce predictions on a flat surface (the column space). OLS picks the point on the surface closest to y – the projection. The leftover “perpendicular” bit (the residuals $y - \hat{y}$) cannot be explained by linear combinations of the features; it’s the part the model is forced to ignore.

6.5 When the formula breaks

Two situations make $X^T X$ non-invertible:

- **Multicollinearity:** two features are linear functions of each other (e.g. height in cm and height in inches both included). One column carries no new information.
- $p > n$: more features than examples. In genomics this happens all the time.

When this happens, OLS doesn't work. Solutions:

- Drop redundant features.
- Use **regularisation** – specifically Ridge regression (Topic 27), which makes the inversion problem go away.
- Use the **pseudo-inverse**, a generalisation of the inverse that always exists.

Exam Focus

You should be able to:

- Recall $\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y$ from memory.
- State the condition for the inverse to exist: X has full column rank, i.e. no perfect collinearity.
- Compute it by hand for tiny problems where X is 2×2 or 3×2 .

Practice Problem

Compute the OLS estimator for the following tiny problem.

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}.$$

(The first column of 1's is for the intercept.)

Solution

Step 1. Compute $X^T X$.

$$X^T = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix}.$$

$$X^T X = \begin{pmatrix} 1+1+1 & 1+2+3 \\ 1+2+3 & 1+4+9 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}.$$

Step 2. Compute $(X^T X)^{-1}$.

$$\det = (3)(14) - (6)(6) = 42 - 36 = 6.$$

$$(X^T X)^{-1} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix}.$$

Step 3. Compute $X^T y$.

$$X^T y = \begin{pmatrix} 2+3+5 \\ 2+6+15 \end{pmatrix} = \begin{pmatrix} 10 \\ 23 \end{pmatrix}.$$

Step 4. Multiply.

$$\hat{\theta} = \frac{1}{6} \begin{pmatrix} 14 & -6 \\ -6 & 3 \end{pmatrix} \begin{pmatrix} 10 \\ 23 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 140 - 138 \\ -60 + 69 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 2 \\ 9 \end{pmatrix} = \begin{pmatrix} 1/3 \\ 3/2 \end{pmatrix}.$$

Fitted model: $\hat{y} = 1/3 + (3/2)x$.

Sanity check at $x = 2$: $\hat{y} = 1/3 + 3 = 3.33$, very close to the actual $y_2 = 3$. ✓

7 Probability – The Bare Minimum

Difficulty: Moderate. Lots of vocabulary. • **How to read:** Skim if you’ve seen probability before; otherwise read carefully.

Why We Care

Almost all of statistical machine learning is built on probability theory. You don’t need to be a probability expert – this guide will teach you the few facts that get used over and over.

7.1 Random variables

A **random variable** is a quantity whose value depends on the outcome of some random process. Roll a die: the result is a random variable taking values $1, 2, \dots, 6$. Measure tomorrow’s temperature: that’s also a random variable, but its values are continuous.

We split random variables into two flavours:

- **Discrete:** possible values are countable (die rolls, number of customer arrivals, spam-or-not). Described by a **probability mass function** (PMF): $\mathbb{P}(X = x)$ – the probability that X takes the value x .
- **Continuous:** possible values fill an interval (heights, temperatures, response times). Described by a **probability density function** (PDF) $f(x)$, where the probability of X falling in some range is the area under f over that range:

$$\mathbb{P}(a \leq X \leq b) = \int_a^b f(x) dx.$$

7.2 Expected value – the average

The **expected value** of a random variable X is its long-run average value.

For discrete X :

$$\mathbb{E}[X] = \sum_x x \mathbb{P}(X = x).$$

For continuous X :

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x f(x) dx.$$

Reading This Formula

- $\mathbb{E}[X]$ – “expected value of X ”. Same idea as “mean” or “average”.
- For discrete: weight each possible value by its probability and add up.
- For continuous: same idea, but “weighted sum” becomes “weighted integral”. The integral $\int f(x) dx$ is just calculus’s way of summing up infinitely many tiny pieces.

A useful property: **linearity**. $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$, no matter how X and Y are related.

7.3 Variance – the spread

The **variance** of X is the average squared distance from the mean:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Reading This Formula

Variance measures spread. A random variable with low variance is tightly clustered around its mean; one with high variance is all over the place. The two formulas above are equivalent (the second is often easier to compute).

The **standard deviation** is just $\sqrt{\text{Var}(X)}$, often written σ . It has the same units as X and is easier to interpret (“the average distance from the mean”).

7.4 Covariance and correlation

For two random variables X and Y :

$$\text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X] \mathbb{E}[Y].$$

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X) \text{Var}(Y)}}.$$

Reading This Formula

- Covariance measures whether X and Y tend to move *together* (positive cov.) or in opposite directions (negative cov.). Its units depend on the units of X and Y , which is awkward.
- Correlation rescales covariance to $[-1, +1]$.
 - $+1$: perfect linear positive relationship.
 - -1 : perfect linear negative.
 - 0 : no linear relationship (but a non-linear one is possible!).

“Correlation does not imply causation” is a famous warning that comes from the fact that two variables can have non-zero correlation without either causing the other (a third variable might cause both).

7.5 Joint, marginal, and conditional probability

Two random variables together have a **joint** distribution $p(x, y)$. From it we get:

- **Marginal** of X : how often X takes value x , ignoring Y :

$$p(x) = \sum_y p(x, y) \quad (\text{or integrate, if continuous}).$$

- **Conditional** of Y given $X = x$: how Y behaves once we know $X = x$:

$$p(y | x) = \frac{p(x, y)}{p(x)}.$$

Independence: X and Y are independent (written $X \perp Y$) if knowing one tells you nothing about the other:

$$p(x, y) = p(x)p(y).$$

7.6 Bayes’ rule – inversion of probabilities

Key Formula

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A) \mathbb{P}(A)}{\mathbb{P}(B)}.$$

Reading This Formula

- $\mathbb{P}(A | B)$ – “probability of A given B ”. How likely is A , once we know B has happened?
- $\mathbb{P}(B | A)$ – the reversed conditional. How likely is B , given A ?
- $\mathbb{P}(A)$ – the unconditional or **prior** probability of A (before knowing B).
- $\mathbb{P}(B)$ – a normalising factor that ensures everything sums to 1.

Bayes’ rule **flips a conditional probability around**. We often know $\mathbb{P}(B | A)$ but want $\mathbb{P}(A | B)$ – diagnostic problems are the canonical example: we know $\mathbb{P}(\text{positive test} | \text{disease})$ from the lab, but the patient asks $\mathbb{P}(\text{disease} | \text{positive test})$.

7.7 The Gaussian (normal) distribution

The most important continuous distribution. Its PDF is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Reading This Formula

- μ (“mu”) – the mean. Where the bell-curve is centred.
- σ^2 (“sigma-squared”) – the variance. Controls width. Bigger σ = wider bell.
- $\sqrt{2\pi\sigma^2}$ – a normalising constant ensuring the total area under the curve is 1.
- $\exp(\cdot)$ – exponential function $e^{(\cdot)}$.
- $-(x - \mu)^2$ – this part says “the further x is from the mean, the smaller the probability density”. Squaring symmetrises it: above and below the mean are equally likely.

The bell shape arises because the density is highest at the mean and falls off rapidly as you move away.

A short-hand: “ $X \sim \mathcal{N}(\mu, \sigma^2)$ ” means “ X is normally distributed with mean μ and variance σ^2 ”.

7.8 Common distributions you'll meet

Distribution	Models	Mean	Variance
Bernoulli(p)	one yes/no event	p	$p(1-p)$
Binomial(n, p)	sum of n Bernoullis	np	$np(1-p)$
Poisson(λ)	count of rare events / time	λ	λ
Uniform(a, b)	every value in $[a, b]$ equally likely	$(a+b)/2$	$(b-a)^2/12$
Normal(μ, σ^2)	continuous bell-curve	μ	σ^2
Exponential(λ)	time between Poisson events	$1/\lambda$	$1/\lambda^2$

7.9 Two laws that make statistics work

Law of Large Numbers (LLN). As you collect more samples, the sample average $\bar{X}_n = (X_1 + \dots + X_n)/n$ converges to the true mean $\mathbb{E}[X]$. This is why averaging works.

Central Limit Theorem (CLT). Take n independent samples from *any* distribution with finite variance, and average them. The distribution of that average, suitably re-scaled, looks more and more like a Gaussian as n grows. This is why Gaussians appear everywhere.

Practice Problem

A discrete random variable X takes values $\{1, 2, 3\}$ with probabilities $\{0.2, 0.5, 0.3\}$.

1. Compute $\mathbb{E}[X]$.
2. Compute $\mathbb{E}[X^2]$.
3. Compute $\text{Var}(X)$.

Solution

- (1) $\mathbb{E}[X] = (1)(0.2) + (2)(0.5) + (3)(0.3) = 0.2 + 1.0 + 0.9 = 2.1$.
 (2) $\mathbb{E}[X^2] = (1)(0.2) + (4)(0.5) + (9)(0.3) = 0.2 + 2.0 + 2.7 = 4.9$.
 (3) $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = 4.9 - 2.1^2 = 4.9 - 4.41 = 0.49$.
Standard deviation: $\sigma = \sqrt{0.49} = 0.7$.

Practice Problem

A test for a disease is 99% accurate (both sensitivity and specificity). 1% of the population actually has the disease. You test positive. What is the probability you have the disease?

Solution

Let D = “has disease”, P = “positive test”.
 We know: $\mathbb{P}(D) = 0.01$, $\mathbb{P}(P | D) = 0.99$, $\mathbb{P}(P | \bar{D}) = 0.01$.
 By Bayes:

$$\mathbb{P}(D | P) = \frac{\mathbb{P}(P | D) \mathbb{P}(D)}{\mathbb{P}(P | D) \mathbb{P}(D) + \mathbb{P}(P | \bar{D}) \mathbb{P}(\bar{D})} = \frac{(0.99)(0.01)}{(0.99)(0.01) + (0.01)(0.99)} = \frac{0.0099}{0.0198} = 0.5.$$

Surprising answer: 50%. Even with a 99% accurate test, because the disease is rare, half of all positive results are false positives. This is a famous statistical paradox.

8 Statistical Framework for Linear Regression

Difficulty: Moderate. The bridge from probability to ML. • **How to read:** Read once, focus on the punchline “Gaussian noise $\Rightarrow$ MLE = OLS”.

(Reminder: Bayes’ rule and Gaussian distribution was introduced in Topic 7.)

Why We Care

In Topic 5 we wrote down linear regression and its loss function. In Topic 6 we minimised the loss to get the OLS estimator. But where did the squared-error loss come from? Why not absolute-value error, or fourth-power error? This topic explains *why squared error is natural*: it falls out of an assumption that the noise is Gaussian.

8.1 The probabilistic model behind linear regression

We assume the data was generated like this:

$$y_i = \theta^T x_i + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Reading This Formula

- y_i – the observed label for example i .
- $\theta^T x_i$ – the “true” linear part: a weighted sum of the features.
- ε_i – random noise drawn from a Gaussian with mean 0 and some unknown variance σ^2 . The squiggly $\sim$ is read as “is distributed as”.

Equivalent way of saying the same thing: given x_i , the label y_i is itself Gaussian:

$$y_i | x_i \sim \mathcal{N}(\theta^T x_i, \sigma^2).$$

The mean of this Gaussian is what the linear model predicts; the variance σ^2 measures how noisy the world is.

8.2 Maximum Likelihood Estimation – “find the parameters that make the data most plausible”

If the data really was generated this way, then any candidate θ assigns a **likelihood** – a probability density – to the observed data. Maximum Likelihood Estimation (MLE) picks the θ that maximises this likelihood.

For our Gaussian-noise model, the log-likelihood works out to:

$$\log \mathcal{L}(\theta) = \text{const} - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \theta^T x_i)^2.$$

The constant doesn’t depend on θ . The second term *does* depend on θ : it’s the negative of the squared error sum, scaled by $1/(2\sigma^2)$.

Punchline:

$$\arg \max_{\theta} \log \mathcal{L}(\theta) = \arg \min_{\theta} \sum_{i=1}^n (y_i - \theta^T x_i)^2 = \hat{\theta}_{\text{OLS}}.$$

If the noise is Gaussian, MLE and OLS give exactly the same answer. Squared error wasn't an arbitrary choice; it was the choice that's optimal under Gaussian noise.

Reading This Formula

- $\arg \max_{\theta} f(\theta)$ – “the value of θ that maximises f ”.
- $\arg \min_{\theta} g(\theta)$ – “the value of θ that minimises g ”.
- Maximising the log-likelihood is the same as minimising the negative log-likelihood, which (because the constant doesn't matter) is the same as minimising the squared-error sum.

8.3 Maximum a Posteriori (MAP) estimation

MLE assumes nothing about θ in advance. But often we have prior beliefs – for instance, “the weights are probably small” or “most weights are probably zero”. We can encode these beliefs as a **prior distribution** $p(\theta)$ and then use Bayes' rule to combine the prior with the data:

$$p(\theta \mid \mathcal{D}) \propto p(\mathcal{D} \mid \theta) p(\theta).$$

The **MAP estimator** is the θ that maximises this posterior:

$$\hat{\theta}_{\text{MAP}} = \arg \max_{\theta} [\log p(\mathcal{D} \mid \theta) + \log p(\theta)].$$

The remarkable observation is that different priors give different familiar regularisers:

Prior on θ	Equivalent loss	Method
Uniform (improper)	$-\log \mathcal{L}$	MLE / OLS
$\theta \sim \mathcal{N}(0, \tau^2 I)$	$-\log \mathcal{L} + \frac{\lambda}{2} \ \theta\ _2^2$	Ridge
$\theta \sim \text{Laplace}(0, b)$	$-\log \mathcal{L} + \lambda \ \theta\ _1$	LASSO

We'll meet Ridge and LASSO properly in Topics 27–28.

8.4 Estimator quality – bias, variance, MSE

When we treat the data as random and our estimator $\hat{\theta}$ as a function of that random data, $\hat{\theta}$ is itself random. Three properties matter:

- **Bias:** $\mathbb{E}[\hat{\theta}] - \theta^*$. “On average, do we hit the true value?” An unbiased estimator has bias 0.
- **Variance:** $\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$. “How spread out are our estimates around their average?”
- **Mean Squared Error:** $\text{MSE} = \text{Bias}^2 + \text{Variance}$. This is the actual quantity we want small.

A famous trade-off: increasing model complexity reduces bias but raises variance. We'll see this everywhere – it's the topic of Topic 11.

Practice Problem

You're told the following about an estimator $\hat{\theta}$:

$$\mathbb{E}[\hat{\theta}] = 2.0, \quad \theta^* = 2.5, \quad \text{Var}(\hat{\theta}) = 0.25.$$

1. Compute the bias.
2. Compute the MSE.
3. Is the estimator unbiased?

Solution

- (1) Bias = $\mathbb{E}[\hat{\theta}] - \theta^* = 2.0 - 2.5 = -0.5$.
- (2) MSE = Bias² + Var = $(-0.5)^2 + 0.25 = 0.25 + 0.25 = 0.5$.
- (3) No, the bias is non-zero. An unbiased estimator would have $\mathbb{E}[\hat{\theta}] = \theta^*$.

9 Hypothesis Testing

Difficulty: Moderate. Mostly procedural. • **How to read:** Read carefully. The Type I/II table and the t -statistic formula are the take-aways.

Why We Care

You fit a regression model with 5 features. One coefficient comes out $\hat{\theta}_3 = 0.02$. Does this mean feature 3 actually matters, or is the 0.02 just noise from random sampling? Hypothesis testing answers this question with statistical rigour.

9.1 Null and alternative hypotheses

You always set up two hypotheses:

- H_0 – the **null hypothesis**. The default, “boring” assumption (e.g. “feature 3 has zero true effect”, $\theta_3 = 0$).
- H_1 – the **alternative hypothesis**. The interesting claim (e.g. “feature 3 does matter”, $\theta_3 \neq 0$).

You collect data and compute a **test statistic** that summarises how unlikely your observation is, assuming H_0 is true. If the observation looks very unlikely under H_0 , you reject H_0 in favour of H_1 .

9.2 Two kinds of mistake

TruthDecision	Reject H_0	Don't reject
H_0 true	Type I error (prob. α)	Correct
H_0 false	Correct (power = $1 - \beta$)	Type II error (prob. β)

Reading This Formula

- **Type I error:** rejecting H_0 when it was actually true. This is a “false alarm”. Statisticians control this by setting a **significance level** α – usually 0.05 or 0.01. An α of 0.05 means: if H_0 is really true, we'll mistakenly reject it 5% of the time.
- **Type II error:** failing to reject H_0 when it was actually false. This is “missing a real effect”. The probability of making this mistake is β .
- **Power:** $1 - \beta$, the probability of correctly rejecting a false H_0 . More data raises power.

9.3 t -test for a regression coefficient

We want to test $H_0 : \theta_j = 0$ vs. $H_1 : \theta_j \neq 0$. The test statistic is

$$t = \frac{\hat{\theta}_j}{\widehat{\text{SE}}(\hat{\theta}_j)}.$$

Reading This Formula

- $\hat{\theta}_j$ – our estimated coefficient.
- $\widehat{\text{SE}}(\hat{\theta}_j)$ – the estimated **standard error**. This is essentially the standard deviation of $\hat{\theta}_j$ – how much we expect it to vary if we re-collected the data. Software computes this from the data.
- t – the ratio. A large $|t|$ means the estimate is many standard errors away from zero, so it would be very surprising under H_0 .

Under H_0 , this t follows a known distribution called the Student t -distribution with $n - d - 1$ degrees of freedom. We use that distribution to compute the p -value.

9.4 p -value – the verdict

$$p\text{-value} = \mathbb{P}(|T| \geq |t_{\text{obs}}| \mid H_0).$$

Reading This Formula

- t_{obs} – the actual t -value we computed from our data.
- “ $|T| \geq |t_{\text{obs}}|$ ” – how unusual is a t -value at least this far from zero?
- “ $|H_0$ ” – assuming the null hypothesis is true.

The decision rule:

- If $p\text{-value} < \alpha$ (e.g. < 0.05): reject H_0 . The effect is “statistically significant”.
- Otherwise: don’t reject. There is not enough evidence to claim a non-zero effect.

9.5 Confidence interval

A 95% **confidence interval** for θ_j is

$$\hat{\theta}_j \pm t^* \cdot \widehat{\text{SE}}(\hat{\theta}_j),$$

where t^* is the appropriate t -table critical value (about 1.96 for large samples).

What it means: if you re-collected data and re-fitted many times, about 95% of the constructed intervals would contain the true θ_j .

Connection to testing: a 95% CI excludes 0 *exactly when* $H_0 : \theta_j = 0$ is rejected at the 5% level.

9.6 F -test – testing several coefficients at once

If you want to test that *several* coefficients are jointly zero (e.g. “do these 3 features collectively help?”), you use the F -test. Compare a small (reduced) model with RSS_0 against the full model with RSS_1 :

$$F = \frac{(\text{RSS}_0 - \text{RSS}_1)/q}{\text{RSS}_1/(n - d - 1)}.$$

Reading This Formula

- q – the number of coefficients you’re testing simultaneously.
- $(\text{RSS}_0 - \text{RSS}_1)$ – how much extra error you get by throwing those q features out. Big = features matter.
- $\text{RSS}_1/(n - d - 1)$ – the noise level in the full model.
- F is the ratio of “signal added by the features” to “ambient noise”, adjusted for degrees of freedom. Large F = reject H_0 .

Common Confusion

“All my t -tests came out non-significant, so my model is useless.” Wrong! When predictors are correlated, each individual t -test can be non-significant while the joint F -test is highly significant. Always look at both.

Practice Problem

You fit a regression with $n = 100$ observations and $d = 4$ predictors. For predictor X_3 you obtain $\hat{\theta}_3 = 0.8$ and $\widehat{\text{SE}}(\hat{\theta}_3) = 0.25$.

1. Compute the t -statistic.
2. Use the rule of thumb $|t| > 2 \Rightarrow$ significant at the 5% level. Is X_3 significant?
3. Construct an approximate 95% confidence interval for θ_3 (use $t^* \approx 1.98$).

Solution

- (1) $t = 0.8/0.25 = 3.2$.
- (2) $|3.2| > 2$, so yes – X_3 is significant at the 5% level. We reject $H_0 : \theta_3 = 0$.
- (3) CI: $0.8 \pm 1.98 \cdot 0.25 = 0.8 \pm 0.495$, giving $[0.305, 1.295]$. This interval excludes 0, consistent with rejecting the null.

10 Goodness of Fit

Difficulty: Easy. Mostly arithmetic on RSS and TSS. • **How to read:** Read once. The R^2 formula is the headline.

Why We Care

You fit a regression. How well does it actually fit the data? We need a number that says “the model explains X% of the variation in y ”. That number is R^2 .

10.1 Three sums of squares

Define:

- **Total Sum of Squares (TSS):** $TSS = \sum_{i=1}^n (y_i - \bar{y})^2$ – the total variation in y around its mean. This is what we’d “explain” if we predicted $\bar{y}$ for every example (the worst possible linear model).
- **Residual Sum of Squares (RSS):** $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ – the leftover variation after the model’s predictions.
- **Explained Sum of Squares (ESS):** $ESS = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$ – the variation captured by the model’s predictions.

When the model has an intercept, these add up neatly:

$$TSS = ESS + RSS.$$

10.2 R^2 – the headline metric

Key Formula

$$R^2 = 1 - \frac{RSS}{TSS} = \frac{ESS}{TSS}.$$

Reading This Formula

- RSS/TSS – the fraction of variation *not* explained.
- $1 - RSS/TSS$ – the fraction *explained*.
- $R^2 = 0$: model is no better than predicting the mean.
- $R^2 = 1$: model fits the data perfectly ($RSS = 0$).
- $R^2 = 0.85$: “the model explains 85% of the variation in y ”.

Useful identities:

- $R^2 = \text{Corr}(y, \hat{y})^2$ – it’s the squared correlation between observed and predicted.
- For simple linear regression with one feature: $R^2 = \text{Corr}(x, y)^2$.

10.3 The big trap: R^2 never decreases

Add any feature to the model – even a totally useless one – and R^2 will go up (or at worst stay the same). This is because the optimiser can always set the new coefficient to 0 to recover the old fit, but it will usually find a slightly better fit.

Consequence: you cannot use R^2 to compare models with different numbers of features.

10.4 Adjusted R^2

Adjusted R^2 penalises adding features that don't help:

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - d - 1}.$$

Reading This Formula

- n – number of training examples.
- d – number of features (not counting the intercept).
- As d grows, the fraction $(n - 1)/(n - d - 1)$ grows, inflating the $(1 - R^2)$ term and pulling $\bar{R}^2$ down.

$\bar{R}^2$ can decrease when a useless feature is added. That's why it's preferred for model comparison.

10.5 Residual standard error (RSE)

$$\text{RSE} = \sqrt{\frac{\text{RSS}}{n - d - 1}} = \hat{\sigma}.$$

This estimates the typical size of a prediction error, in the same units as y .

Worked Example

TSS = 300, RSS = 120, $n = 20$, $d = 3$.

$R^2 = 1 - 120/300 = 0.6$. The model explains 60% of variation.

$\bar{R}^2 = 1 - (1 - 0.6) \cdot \frac{19}{16} = 1 - 0.475 = 0.525$.

$\text{RSE} = \sqrt{120/16} = \sqrt{7.5} \approx 2.74$ – typical prediction error is about 2.74 in y -units.

10.6 Diagnostic plots to inspect

A high R^2 doesn't guarantee a good model. Always look at:

- **Residuals vs. fitted values.** Should look like random scatter. A pattern (curve, funnel) suggests a problem.
- **Q-Q plot of residuals.** If errors are Gaussian, the points should lie on a straight line.
- **Leverage h_{ii}** – how much an individual point pulls the fit. Big h_{ii} means an influential point.

- **Cook's distance** – combines residual size and leverage to flag genuinely problematic points.
- **Variance Inflation Factor (VIF)** – $VIF_j = 1/(1 - R_j^2)$ where R_j^2 regresses feature j on the others. $VIF_j > 5$ or 10 flags multicollinearity.

Exam Focus

You will be asked to:

- Compute R^2 from RSS and TSS.
- Compute $\bar{R}^2$ from R^2 , n , and d .
- Explain why we use $\bar{R}^2$ instead of R^2 for model comparison.
- State that high R^2 does NOT imply good prediction on new data (that's what cross-validation is for, Topic 13).

Practice Problem

A regression with $n = 50$ and $d = 4$ predictors has TSS = 200 and RSS = 50.

1. Compute R^2 .
2. Compute $\bar{R}^2$.
3. Compute the residual standard error (RSE).
4. If you add one more useless predictor and RSS stays at 50, what happens to R^2 ? What about $\bar{R}^2$?

Solution

- (1) $R^2 = 1 - 50/200 = 1 - 0.25 = 0.75$. The model explains 75% of the variance.
 (2) $\bar{R}^2 = 1 - (1 - 0.75) \cdot \frac{49}{45} = 1 - 0.25 \cdot 1.0889 = 1 - 0.2722 = 0.7278$.
 (3) $RSE = \sqrt{50/(50 - 4 - 1)} = \sqrt{50/45} \approx 1.054$.
 (4) R^2 stays at 0.75 (RSS unchanged, TSS unchanged). But $\bar{R}^2 = 1 - 0.25 \cdot \frac{49}{44} = 1 - 0.2784 = 0.7216$ **decreases**. This is exactly why we use $\bar{R}^2$ for comparing models with different numbers of predictors.

Part II

Model Assessment, Selection and Resampling

11 Model Assessment and the Bias–Variance Tradeoff

Difficulty: Conceptual but vital. The deepest idea in ML. • **How to read:** Read carefully. Make sure you can diagnose bias vs. variance from a train/test gap.

Why We Care

You train two models on the same data:

Model A – a simple straight line. It fits the training data poorly: errors are big. It predicts the test data with errors that are about equally big.

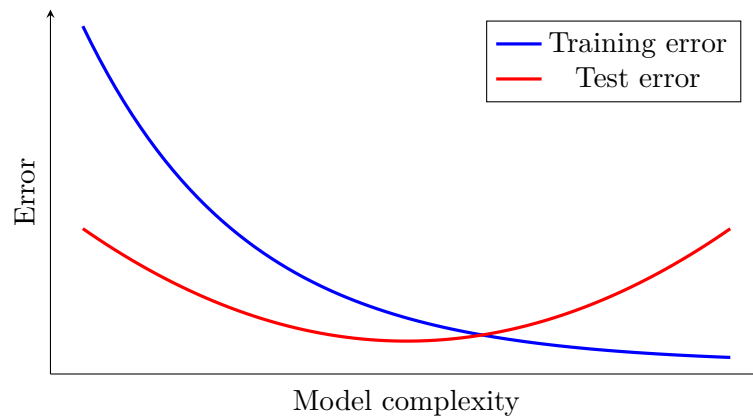
Model B – a wiggly polynomial of degree 20. It passes through nearly every training point, so its training error is almost zero. But on test data its errors are huge.

Which model is “better”? Neither is great. *A* is too rigid; *B* has memorised the training noise. This trade-off between rigidity and flexibility is the deepest idea in machine learning, and it has a mathematical name: the **bias–variance tradeoff**.

11.1 Why training error is misleading

When you make a model more complex (more parameters, deeper trees, higher polynomial degree), its training error *always* decreases. Eventually it can fit the training data perfectly – training error 0.

But the model has now memorised the random noise in the training data. On new data that noise will be different, so the model will fail. Test error first decreases (as the model captures real patterns) but then *increases* (as the model starts memorising noise). The classic “U-curve”:



The bottom of the test-error curve is the “sweet spot”. Finding it is what model selection is all about.

11.2 Bias and variance, defined intuitively

Imagine doing this thought experiment: re-collect a fresh training set from scratch many times. Each time, train the same model. You end up with many trained models – all from the same procedure but different data.

- **Bias** – how far the *average* of all those trained models is from the truth. Big bias means the model class is inherently too rigid to capture the true function.
- **Variance** – how much the trained models differ from each other. Big variance means the model is too sensitive to the random fluctuations in the training set.

A simple model (e.g. a straight line) has high bias (it can’t curve to fit a wiggly truth) but low variance (different training sets all give similar lines).

A complex model (e.g. a 20th-degree polynomial) has low bias (it can fit almost anything) but high variance (different training sets give wildly different polynomials).

11.3 The bias–variance decomposition

For a single test point x_0 , the expected squared error of the model’s prediction can be split into three parts:

Key Formula

$$\mathbb{E}[(Y - \hat{f}(x_0))^2] = \underbrace{(f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\hat{f}(x_0) - \mathbb{E}[\hat{f}(x_0)])^2]}_{\text{Variance}} + \underbrace{\sigma_\varepsilon^2}_{\text{Irreducible}}.$$

Reading This Formula

- Y – the true label at x_0 (random because of noise).
- $f(x_0)$ – the true underlying function value.
- $\hat{f}(x_0)$ – our model’s prediction. This is random because it depends on which training set we drew.

- $\mathbb{E}[\hat{f}(x_0)]$ – the average of our predictions over all possible training sets.
- **Bias²**: $(f(x_0) - \mathbb{E}[\hat{f}(x_0)])^2$. How far off our average prediction is from the truth, squared.
- **Variance**: how much our prediction wiggles around its own mean.
- **Irreducible error** σ_ε^2 : the inherent randomness in Y that no model can eliminate (the world is noisy).

Important takeaway: the test error has three components. Two of them depend on the model (bias and variance); one is fundamental (irreducible). We can shift bias and variance against each other by adjusting model complexity.

11.4 The trade-off in a table

Complexity	Bias	Variance	Train err.	Symptom
Too simple	High	Low	High	Underfitting – both errors big
Just right	Low	Low	Med	Best generalisation
Too complex	Low	High	Low	Overfitting – low train, high test

Worked Example

You train three models on the same housing dataset:

Model A (linear, 2 features)	Train MSE 45,000	Test MSE 46,000
Model B (poly degree 8)	Train MSE 1,200	Test MSE 98,000
Model C (poly degree 3)	Train MSE 8,000	Test MSE 9,500

Diagnosis:

- A: high train AND test error → *high bias*, underfitting.
- B: tiny train error, huge test error → *high variance*, overfitting.
- C: moderate train and test error, both close together → best balance.

A clear gap between training and test error is the classic signature of overfitting (high variance).

Intuition

The bias–variance idea explains why we need techniques like cross-validation, regularisation, ensembles, and bagging. Each one manages the tradeoff differently:

- Regularisation: *slightly* increase bias to dramatically cut variance.
- Bagging: average many high-variance models to cut variance.
- Boosting: combine many high-bias models to cut bias.

Practice Problem

For each scenario, decide if the problem is high bias, high variance, or balanced.

1. Linear model on data with a clear quadratic shape. Train MSE = 80, Test MSE = 85.

2. Random forest with 1000 trees on a small dataset of 50 examples. Train MSE = 2, Test MSE = 200.
3. Polynomial regression of degree 5 on 200 examples. Train MSE = 30, Test MSE = 35.

Solution

- (1) High bias (underfitting). Both errors are similar AND both high $\Rightarrow$ the model is too simple to capture the truth.
- (2) High variance (overfitting). Tiny train error and huge test error $\Rightarrow$ memorising the training data.
- (3) Balanced. Train and test errors are close together AND both moderate. The model has the right complexity.

12 Subset Selection – Choosing Which Features to Use

Difficulty: Easy. Mostly procedural. • **How to read:** Read once. Memorise the costs (2^p vs $1 + p(p+1)/2$).

Why We Care

You have 20 candidate features for predicting house price. Should you include all 20? Some of them are probably noise. Including too many adds variance; excluding important ones adds bias. Subset selection is the disciplined approach to picking which features to keep.

12.1 The combinatorial nightmare

If you have p features, the number of possible subsets is 2^p (each feature is either in or out). For $p = 10$, that's 1,024 subsets. For $p = 20$, over a million. For $p = 40$, over a trillion. Pure exhaustive search becomes infeasible quickly.

12.2 Best subset selection – the gold standard (when feasible)

1. For each $k = 0, 1, 2, \dots, p$:
 - Fit all $\binom{p}{k}$ models that use exactly k features.
 - Pick the one with the smallest RSS – call it $\mathcal{M}_k$.
2. Among $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$, pick a single best model using cross-validation, AIC, BIC, or adjusted R^2 .

Important: step 2 cannot use training RSS – because RSS *always* decreases as you add features, this would always pick the full model. We need a complexity-aware criterion.

12.3 Forward stepwise selection – start small, grow

1. Start with $\mathcal{M}_0$ – intercept only.
2. For $k = 0, 1, \dots, p-1$:
 - Of the $p - k$ features not yet in the model, find the one whose addition gives the biggest drop in RSS.
 - Add it. Call the new model $\mathcal{M}_{k+1}$.
3. Pick the final model among $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$ using a complexity-aware criterion.

Number of models fitted: $1 + p + (p-1) + \dots + 1 = 1 + p(p+1)/2$.

For $p = 10$: 56 models versus 1,024 for best subset. Much cheaper.

Drawback: forward stepwise is *greedy*. Once a feature is in, it stays. It might miss the truly best subset (for example, a pair of features that are useful only when together).

12.4 Backward stepwise selection – start big, shrink

The reverse: start with all p features, repeatedly remove the feature whose removal least increases RSS, until you're down to the intercept.

Drawback: cannot start when $p > n$ (the full model is unidentifiable).

12.5 Choosing among models of different sizes

Whichever search strategy you use, at the end you have a sequence $\mathcal{M}_0, \mathcal{M}_1, \dots$ – one model per size. To pick the best size, use:

- Cross-validation (Topic 13)
- AIC or BIC (Topic 15)
- Adjusted R^2

Never the training RSS alone – that's the cardinal mistake.

Worked Example

$p = 3$ features: X_1 (sq. ft.), X_2 (bedrooms), X_3 (age). Forward stepwise:

Step 1. Fit each one-feature model.

$\{X_1\}$: RSS = 500. $\{X_2\}$: 800. $\{X_3\}$: 1200.

Best: $\mathcal{M}_1 = \{X_1\}$.

Step 2. Add one feature to $\{X_1\}$.

$\{X_1, X_2\}$: 320. $\{X_1, X_3\}$: 480.

Best: $\mathcal{M}_2 = \{X_1, X_2\}$.

Step 3. Add one feature to $\{X_1, X_2\}$.

$\{X_1, X_2, X_3\}$: 300 = $\mathcal{M}_3$.

Now use cross-validation (or AIC/BIC) to choose between $\mathcal{M}_0, \mathcal{M}_1, \mathcal{M}_2, \mathcal{M}_3$.

Practice Problem

1. How many models would best subset selection fit for $p = 6$ features?
2. How many for forward stepwise selection with the same $p = 6$?
3. Which method is greedy? What does “greedy” mean here?

Solution

(1) $2^p = 2^6 = 64$ models.

(2) $1 + p(p+1)/2 = 1 + 6 \cdot 7/2 = 1 + 21 = 22$ models. Roughly $3\times$ fewer.

(3) Forward stepwise is greedy. “Greedy” here means it makes the locally best choice at each step (add the feature that most reduces RSS right now), without considering whether that choice could prevent finding the globally best subset later. Best subset selection is exhaustive: it considers every possible subset, so it always finds the global optimum (at huge computational cost).

13 Cross-Validation

Difficulty: Easy. The most useful technique in the whole guide. • **How to read:** Read once. Memorise the k -fold formula.

Why We Care

We've said many times: "don't use training error to evaluate models." But what if you don't have enough data to set aside a separate test set? Cross-validation is the clever workaround: use the data multiple times, each time holding out a different piece for evaluation.

13.1 The basic idea: k -fold cross-validation

Split the training data into k equal-sized chunks called **folds** (common choices: $k = 5$ or $k = 10$). Then:

1. Set aside fold 1. Train on folds 2 to k . Evaluate on fold 1. Record the error: call it MSE_1 .
2. Set aside fold 2. Train on folds 1, 3, 4, ..., k . Evaluate on fold 2. Record MSE_2 .
3. Continue until every fold has had its turn as the held-out set.

The cross-validation estimate of test error is the average:

Key Formula

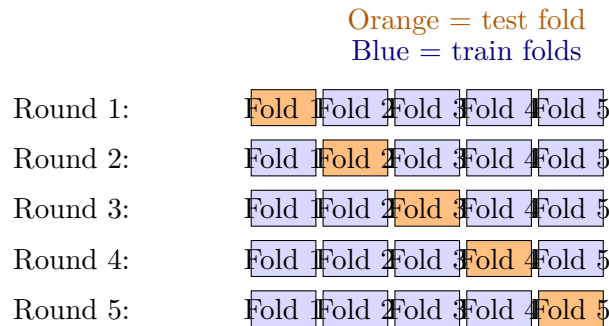
$$\text{CV}_{(k)} = \frac{1}{k} \sum_{j=1}^k \text{MSE}_j.$$

Reading This Formula

- MSE_j – the test error you got the time fold j was the held-out set.
- $\frac{1}{k} \sum_{j=1}^k$ – average over the k folds.
- $\text{CV}_{(k)}$ – the cross-validation estimate. Roughly, “the typical test error you'd see on data the model wasn't trained on”.

Why this works: every example is used for evaluation exactly once and for training exactly $k - 1$ times. We extract the maximum amount of information from a fixed dataset.

13.2 Visualising 5-fold CV



13.3 Leave-One-Out CV (LOOCV)

A special case where $k = n$. Each fold is a single example. You fit n models, each trained on $n - 1$ examples and evaluated on the one left out.

Pros: unbiased estimate. Uses (nearly) all the data for training each time.

Cons: expensive (you fit the model n times) and high variance across folds (because each test set is just 1 example).

For linear regression there's a magical shortcut that avoids re-fitting:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - h_{ii}} \right)^2,$$

where h_{ii} is the i th diagonal of the hat matrix H from Topic 6. One model fit; CV estimate done.

13.4 Choosing k

- $k = 5$ or $k = 10$: a good default. Reasonable bias, low variance, cheap.
- $k = n$ (LOOCV): nearly unbiased, but high variance and expensive.
- $k = 2$ (split-half): cheap but pessimistically biased – each model only sees half the data.

Worked Example

You run 5-fold CV. The five fold MSEs are $(MSE_1, \dots, MSE_5) = (11.2, 9.8, 12.4, 10.6, 11.0)$.

$CV_{(5)} = (11.2 + 9.8 + 12.4 + 10.6 + 11.0)/5 = 11.0$.

This 11.0 is your estimate of the test MSE. Compare it across candidate models; pick the one with the smallest.

13.5 What CV is used for

- Choosing the model size in subset selection.
- Choosing regularisation strength λ in Ridge / LASSO.
- Choosing K in k -NN.
- Choosing the depth of a decision tree.

- In short: choosing *any* hyperparameter.

Practice Problem

You run 10-fold cross-validation to compare two models. The fold MSEs are:

- Model A: (8, 12, 9, 11, 10, 9, 13, 8, 11, 9)
- Model B: (20, 5, 18, 6, 22, 4, 19, 5, 21, 4)

1. Compute $CV_{(10)}$ for each model.
2. Which model would you pick based on CV alone?
3. What additional information might you also want before deciding?

Solution

- (1) Model A: $\text{sum} = 100$, $\text{mean} = 10.0$. Model B: $\text{sum} = 124$, $\text{mean} = 12.4$.
(2) Model A has the lower CV error, so pick A.
(3) Look at the *spread* of fold-level errors. Model A's fold MSEs are tightly clustered around 10. Model B's swing wildly from 4 to 22. Model B is unstable – its performance depends heavily on which examples land in the test fold. Even if it had a slightly lower average, A would be the safer bet.

14 The Bootstrap

Difficulty: Easy. One trick to remember: sampling with replacement. • **How to read:** Read once. Memorise the OOB fraction ≈ 0.368 .

Why We Care

You have one dataset of 100 customers, and you've estimated the average spending per customer to be \$73. But how confident are you in that \$73? If you collected another 100 customers tomorrow, you'd get a slightly different number. How spread out are those numbers? This is the **standard error** of your estimate.

In an ideal world, you'd just collect more datasets and see directly how \$73 varies. But you only have one. The bootstrap is a clever trick: pretend your dataset *is* the population, and "re-sample" from it many times to simulate having multiple datasets.

14.1 The bootstrap procedure

Given your dataset $\mathcal{Z} = \{z_1, z_2, \dots, z_n\}$:

1. Draw a **bootstrap sample** $\mathcal{Z}^{*1}$ of size n from $\mathcal{Z}$, sampling **with replacement**. This means you might pick the same example twice and never pick another.
2. Compute your statistic of interest on $\mathcal{Z}^{*1}$. Call the result $\hat{\alpha}^{*1}$.
3. Repeat steps 1–2 B times (typical B : 100 to 1000), getting $\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$.
4. Estimate the standard error of $\hat{\alpha}$ by the standard deviation of the bootstrap estimates:

Key Formula

$$\widehat{\text{SE}}_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (\hat{\alpha}^{*b} - \bar{\hat{\alpha}}^*)^2}, \quad \bar{\hat{\alpha}}^* = \frac{1}{B} \sum_{b=1}^B \hat{\alpha}^{*b}.$$

Reading This Formula

- $\hat{\alpha}^{*b}$ – the statistic computed on the b th bootstrap sample. The asterisk reminds us it's a bootstrap version.
- $\bar{\hat{\alpha}}^*$ – the average of all B bootstrap estimates.
- The big square root computes the sample standard deviation of the bootstrap estimates – exactly how spread out they are.
- $1/(B-1)$ instead of $1/B$ – a small bias correction; for big B they're nearly the same.

The interpretation: this standard deviation *estimates* the standard error of $\hat{\alpha}$. We never actually re-collected data, but bootstrap mimics what would happen if we did.

Worked Example

$B = 4$ bootstrap estimates: $\hat{\alpha}^* = (2, 4, 4, 6)$.

Mean: $\bar{\hat{\alpha}}^* = (2 + 4 + 4 + 6)/4 = 4$.

Squared deviations: $(2 - 4)^2 + (4 - 4)^2 + (4 - 4)^2 + (6 - 4)^2 = 4 + 0 + 0 + 4 = 8$.

Variance: $8/(B - 1) = 8/3 \approx 2.67$.

Standard error: $\sqrt{2.67} \approx 1.63$.

14.2 Out-of-bag (OOB) samples

When you draw a bootstrap sample with replacement, some original examples will be missing from it. How many?

Reading This Formula

The probability that a particular example is *not* chosen on a single draw is $1 - 1/n$. The probability of not being chosen in any of the n draws is $(1 - 1/n)^n$. As n grows large this approaches $1/e \approx 0.368$.

So roughly 37% of the original examples don't appear in any given bootstrap sample. These are called the **out-of-bag (OOB)** examples.

OOB examples are useful: they form a free validation set. When we get to Random Forests (Topic 34) we'll use the OOB examples to estimate test error without doing CV.

Practice Problem

You ran $B = 5$ bootstrap iterations and computed an estimate $\hat{\alpha}^{*b}$ each time:

$$\hat{\alpha}^* = (10, 12, 11, 13, 14).$$

1. Compute the bootstrap mean $\bar{\hat{\alpha}}^*$.
2. Compute the bootstrap standard error.

Solution

(1) $\bar{\hat{\alpha}}^* = (10 + 12 + 11 + 13 + 14)/5 = 60/5 = 12$.

(2) Squared deviations from the mean:

$$(10 - 12)^2 + (12 - 12)^2 + (11 - 12)^2 + (13 - 12)^2 + (14 - 12)^2 = 4 + 0 + 1 + 1 + 4 = 10.$$

Variance: $10/(B - 1) = 10/4 = 2.5$.

Standard error: $\sqrt{2.5} \approx 1.58$.

15 In-Sample Selection: AIC, BIC, C_p

Difficulty: Easy. Just plug and compare. • **How to read:** Skim the formulas. Pay attention to AIC penalty 2 vs. BIC penalty $\ln n$.

Why We Care

We've said: don't use training RSS to compare models with different numbers of parameters. Cross-validation works but is computationally expensive (you fit the model k times). AIC and BIC are clever *adjustments* to training RSS that approximate test error *from the training data alone* – one fit, no CV needed.

15.1 The pattern: training RSS + complexity penalty

Each criterion has the same shape:

$$\text{score} = (\text{training fit}) + (\text{penalty for using more parameters})$$

The penalty pushes back against RSS's tendency to favour bigger models. Different criteria use different penalties.

15.2 The four criteria

For a least-squares fit with d parameters and noise estimate $\hat{\sigma}^2$:

$$\begin{aligned} C_p &= \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2) \\ \text{AIC} &= \frac{1}{n\hat{\sigma}^2}(\text{RSS} + 2d\hat{\sigma}^2) \\ \text{BIC} &= \frac{1}{n}(\text{RSS} + (\ln n) d\hat{\sigma}^2) \\ \bar{R}^2 &= 1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)} \end{aligned}$$

Reading This Formula

- d – number of parameters in the candidate model (size of the feature subset).
- $\hat{\sigma}^2$ – estimate of the noise variance, usually computed from the full model: $\hat{\sigma}^2 = \text{RSS}_{\text{full}}/(n-p-1)$.
- Each criterion adds something proportional to d to RSS. Bigger $d \Rightarrow$ bigger penalty.
- For **AIC**, the penalty is $2d\hat{\sigma}^2$ – linear in d , multiplier 2.
- For **BIC**, the penalty is $(\ln n)d\hat{\sigma}^2$ – linear in d , multiplier $\ln n$. When $n > 7$ we have $\ln n > 2$, so BIC penalises complexity more aggressively.
- C_p is essentially the same as AIC for OLS.

Decision rule: pick the model with the smallest C_p , AIC, or BIC. Or the largest $\bar{R}^2$.

15.3 When to use which

Criterion	Behaviour	Use when
AIC	Picks larger models	You expect lots of small effects.
BIC	Picks smaller models	You believe in a sparse true model and have lots of data.
$\bar{R}^2$	Convenient, less rigorous	Quick comparison; not for serious model selection.
CV	General-purpose, reliable	Whenever computation allows.

Intuition

Why is BIC “consistent” but AIC isn’t? Consistency means: as the data grows ($n \rightarrow \infty$), the criterion picks the true model with probability 1. BIC’s penalty grows with n (because of $\ln n$), so it eventually dominates random chance and picks the true sparse model. AIC’s penalty stays at 2, so it can be fooled by lucky noise even with infinite data. In practice, AIC tends to pick a model that’s slightly too large.

Practice Problem

With $n = 100$ training examples and noise estimate $\hat{\sigma}^2 = 4$, compare two candidate models:

- Model A: $d = 3$ predictors, $\text{RSS} = 380$.
- Model B: $d = 7$ predictors, $\text{RSS} = 340$.

Compute AIC and BIC for each. Which model does each criterion prefer?

Solution

Recall: $\text{AIC} = (\text{RSS} + 2d\hat{\sigma}^2)/(n\hat{\sigma}^2)$, $\text{BIC} = (\text{RSS} + (\ln n)d\hat{\sigma}^2)/n$.

For $n = 100$: $\ln 100 \approx 4.605$.

Model A. $\text{AIC} = (380 + 2 \cdot 3 \cdot 4)/(100 \cdot 4) = 404/400 = 1.010$.

$\text{BIC} = (380 + 4.605 \cdot 3 \cdot 4)/100 = (380 + 55.26)/100 = 4.353$.

Model B. $\text{AIC} = (340 + 2 \cdot 7 \cdot 4)/400 = (340 + 56)/400 = 0.990$.

$\text{BIC} = (340 + 4.605 \cdot 7 \cdot 4)/100 = (340 + 128.94)/100 = 4.689$.

Decisions. AIC: B wins ($0.990 < 1.010$).

BIC: A wins ($4.353 < 4.689$).

This is the classic AIC-vs-BIC pattern – AIC favours the larger model, BIC the smaller.

Part III

Optimization for Machine Learning

16 Gradient Descent

Difficulty: Foundational. Powers every neural network. • **How to read:** Read carefully. Memorise the update rule.

(Reminder: Gradients was introduced in Topic 4.)

Why We Care

You're standing on a hillside in fog. You can't see the bottom of the valley, but you can feel which direction is downhill where you are now. What's the fastest strategy to reach the valley floor? Take a step in the steepest downhill direction, then look around again, then take another step, and so on.

That intuition *is* gradient descent. It's how almost every machine learning model with no closed-form solution gets trained – including every neural network.

16.1 The update rule

Key Formula

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla L(\theta^{(t)}).$$

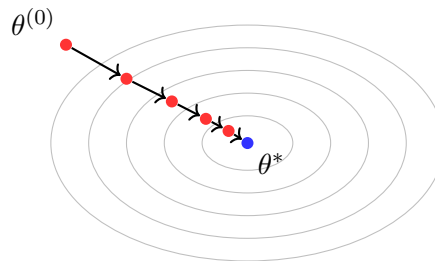
Reading This Formula

- $\theta^{(t)}$ – the current value of the parameters. The superscript (t) counts iterations: $\theta^{(0)}$ is the starting point, $\theta^{(1)}$ after one step, etc.
- $\theta^{(t+1)}$ – the next iterate.
- $L(\theta)$ – the loss function we are minimising.
- $\nabla L(\theta^{(t)})$ – the *gradient* of L at the current point. A vector pointing in the direction of *steepest increase*.
- $-\nabla L$ – the opposite of the gradient. Direction of steepest *decrease*. This is where we want to go!
- η – the **learning rate**. A small positive number (often 0.01 to 0.1) controlling how big a step we take.

The whole rule reads: “move in the direction of steepest descent, with step size η .” Repeat

until the gradient is tiny (close to zero) – which means we’ve found a flat spot, hopefully a minimum.

16.2 Picture



The grey ovals are level curves (constant loss). Each step moves perpendicular to the level curve, towards lower loss.

16.3 Three flavours: batch, stochastic, mini-batch

For ML losses that are sums over training examples, the gradient is a sum of per-example gradients:

$$\nabla L(\theta) = \sum_{i=1}^n \nabla \ell(\theta; x_i, y_i).$$

- **Batch gradient descent.** Use all n training examples to compute each gradient. Stable but expensive when n is large.
- **Stochastic gradient descent (SGD).** Use ONE training example per step:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla \ell(\theta^{(t)}; x_i, y_i).$$

Each step is noisy (because one example is a poor estimate of the full gradient) but cheap. Many noisy steps tend to land in roughly the same place as fewer expensive batch steps.

- **Mini-batch SGD.** The compromise: use a small batch of b examples (often $b = 32$ or 128 or 256). Standard for neural networks.

16.4 Choosing the learning rate η

- Too small $\Rightarrow$ painfully slow convergence.
- Too large $\Rightarrow$ steps overshoot, oscillate, or diverge.
- Modern optimisers (Adam, RMSProp) adapt η per parameter automatically. We won’t go into them here, but most deep-learning code uses Adam by default.

Worked Example

One step of gradient descent on linear regression.

$L(\theta) = \frac{1}{2n} \|X\theta - y\|^2$. Gradient: $\nabla L = \frac{1}{n} X^T (X\theta - y)$.

Take $X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}$, $y = \begin{pmatrix} 3 \\ 5 \end{pmatrix}$, $\theta^{(0)} = (0, 0)^T$, $\eta = 0.1$.

Predictions: $X\theta^{(0)} = (0, 0)^T$.

Residuals: $r = X\theta^{(0)} - y = (-3, -5)^T$.

Gradient: $\frac{1}{2}X^T r = \frac{1}{2} \begin{pmatrix} -3 & -5 \\ -3 & -10 \end{pmatrix} = \begin{pmatrix} -4 \\ -6.5 \end{pmatrix}$.

Update: $\theta^{(1)} = (0, 0)^T - 0.1 \cdot (-4, -6.5)^T = (0.4, 0.65)^T$.

Practice Problem

Minimise $f(x) = (x - 3)^2$ using gradient descent.

1. Compute $f'(x)$.
2. Starting at $x^{(0)} = 0$ with $\eta = 0.5$, perform two gradient descent steps and report $x^{(2)}$.
3. What is the true minimum?

Solution

(1) $f'(x) = 2(x - 3)$.

(2) At $x^{(0)} = 0$: $f'(0) = -6$. $x^{(1)} = 0 - 0.5 \cdot (-6) = 3$.

At $x^{(1)} = 3$: $f'(3) = 0$. $x^{(2)} = 3 - 0.5 \cdot 0 = 3$.

(3) The minimum is at $x = 3$. GD reached it in one step because $\eta = 0.5$ was perfectly tuned for this quadratic.

17 Convex Optimization

Difficulty: Conceptual. Tests are mostly second-derivative checks. • **How to read:** Read once. The convexity tests in Topic 4 are what you'll use.

Why We Care

Why does gradient descent reliably find the right answer for some loss functions but not others? The key property is **convexity**. Convex losses have a single global minimum; gradient descent always finds it. Non-convex losses have multiple local minima; gradient descent can get stuck in a bad one.

17.1 What does “convex” mean?

A function f is **convex** if its graph looks like a bowl: any straight line connecting two points on the graph lies on or above the graph.

A **convex set** is one where any line segment between two points in the set stays in the set. Examples: a circle, a square, all of $\mathbb{R}^n$. Non-examples: a crescent moon, a donut.

17.2 Tests for convexity

Second-derivative test (one variable): f is convex if $f''(x) \geq 0$ for all x .

Examples:

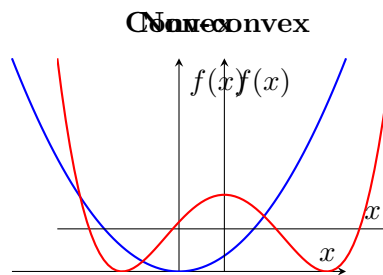
- $f(x) = x^2$: $f''(x) = 2 \geq 0$. Convex everywhere.
- $f(x) = -\ln x$ (for $x > 0$): $f''(x) = 1/x^2 > 0$. Convex.
- $f(x) = x^3$: $f''(x) = 6x$ – changes sign at $x = 0$. NOT convex.

Hessian test (multiple variables): f is convex if its **Hessian** $\nabla^2 f$ is positive semi-definite at every point. “Positive semi-definite” is the matrix version of “ ≥ 0 ” – roughly, the matrix has no negative eigenvalues.

17.3 Why convexity is the dream

For a convex f , three magical things happen:

1. Every local minimum is a global minimum – there's only one valley.
2. Any point with $\nabla f = 0$ is a global minimum.
3. Gradient descent (with appropriate η) provably converges to the global minimum from any starting point.



The convex function on the left has a single bowl. The non-convex function on the right has two valleys; gradient descent might find either depending on where it starts.

17.4 Convex losses in ML

Many classical ML methods have convex loss functions *by design*:

- Linear regression with MSE: convex.
- Logistic regression with cross-entropy: convex.
- Support Vector Machines with hinge loss: convex.
- Ridge and LASSO: convex.

Non-convex examples:

- Neural network training: highly non-convex.
- k -means clustering: non-convex (in centroid positions).
- Decision tree fitting: non-convex (combinatorial).

For non-convex problems we accept that we might find a local minimum rather than the global one. Multiple random restarts and careful initialisation help.

Practice Problem

Determine whether each function is convex on its domain.

1. $f(x) = 2x^2 + 3x + 1$
2. $f(x) = 1/x$ on $x > 0$
3. $f(x) = -x^2$
4. $f(x) = e^{-x}$

Solution

1. $f''(x) = 4 \geq 0$. **Convex.**
2. $f'(x) = -1/x^2$, $f''(x) = 2/x^3 > 0$ on $x > 0$. **Convex.**
3. $f''(x) = -2 < 0$. **Not convex** (in fact, concave – an upside-down bowl).
4. $f''(x) = e^{-x} > 0$. **Convex.**

18 Primal Problems

Difficulty: Moderate. Notation-heavy. • **How to read:** Read carefully. The SVM primal is what to memorise.

Why We Care

Many ML problems involve *constrained* optimisation: find the best parameters subject to some restrictions. “Find the largest margin hyperplane such that every training point is correctly classified.” The original constrained form is called the *primal*. In the next topic we’ll see how every constrained problem has a partner problem called the *dual*, which is sometimes easier to solve.

18.1 Standard form of a constrained problem

$$\min_x f_0(x) \quad \text{subject to} \quad \begin{cases} f_i(x) \leq 0, & i = 1, \dots, m, \\ h_j(x) = 0, & j = 1, \dots, p. \end{cases}$$

Reading This Formula

- $f_0(x)$ – the **objective function**, the thing we’re minimising.
- $f_i(x) \leq 0$ – **inequality constraints**. m of them.
- $h_j(x) = 0$ – **equality constraints**. p of them.
- x – the variables we are choosing.

A point x that satisfies all constraints is called **feasible**. We’re searching only among feasible points.

18.2 The hard-margin SVM primal

The first place we’ll see this in action is Support Vector Machines. For binary classification with $y_i \in \{-1, +1\}$ and linearly separable data, the SVM primal asks:

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1, \quad \text{for all } i = 1, \dots, n.$$

Reading This Formula

- w – the weight vector defining the hyperplane $w^T x + b = 0$.
- b – the intercept (also called bias).
- $\frac{1}{2} \|w\|_2^2$ – minimising this is equivalent to *maximising* the geometric margin. More on this in Topic 23.
- $y_i(w^T x_i + b) \geq 1$ – says every training point lies on or beyond a “correct side” margin.
- “ ≤ 0 ” form: rewrite as $1 - y_i(w^T x_i + b) \leq 0$.

In words: “find the hyperplane with the smallest weight norm that separates the two

classes by at least margin 1.”

18.3 The soft-margin SVM primal

When the data isn't perfectly separable, we add slack variables ξ_i :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad \begin{cases} y_i(w^T x_i + b) \geq 1 - \xi_i, & \text{for all } i, \\ \xi_i \geq 0, & \text{for all } i. \end{cases}$$

Reading This Formula

- ξ_i – a non-negative number measuring how much example i violates the margin. $\xi_i = 0$ means it's safely on the correct side; $\xi_i > 0$ means it has crept into the margin or across the boundary.
- $C \sum \xi_i$ – a penalty proportional to the total violation.
- C – a user-chosen positive number balancing margin width against violations.
 - Big C – penalty is severe; we tolerate fewer violations, tighter fit (lower bias, higher variance).
 - Small C – penalty is gentle; we accept more violations, wider margin (higher bias, lower variance).

Practice Problem

Write down the SVM **soft-margin** primal optimisation problem. Identify which terms are minimised and which are constraints. What role does C play in the bias–variance tradeoff?

Solution

Primal:

$$\min_{w,b,\xi} \frac{1}{2} \|w\|_2^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Minimised: $\frac{1}{2} \|w\|^2$ (small w = wide margin) plus the slack penalty $C \sum \xi_i$.

Constraints: the margin condition (with slack), and $\xi_i \geq 0$.

Role of C :

- Large $C \Rightarrow$ violations costly $\Rightarrow$ tight fit, low bias, high variance.
- Small $C \Rightarrow$ violations cheap $\Rightarrow$ wide margin, high bias, low variance.

C is a hyperparameter chosen by cross-validation.

19 Dual Problems

Difficulty: Hard. The most abstract topic so far. • **How to read:** Read carefully but focus on the SVM dual – that’s what you’ll be tested on.

Why We Care

Every constrained optimisation problem has a partner problem called the *dual*. Sometimes the dual is easier to solve than the original (“primal”). More importantly, for SVMs, the dual is what makes the **kernel trick** possible – a beautiful idea we’ll meet in Topic 25.

19.1 The Lagrangian – folding constraints into the objective

We bake the constraints into the objective with multipliers:

$$\mathcal{L}(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{j=1}^p \nu_j h_j(x).$$

Reading This Formula

- $\mathcal{L}$ – the **Lagrangian**, a new objective that combines the original objective with the constraints.
- $\lambda_i \geq 0$ – **Lagrange multipliers** for the inequality constraints. One per inequality.
- ν_j (no sign restriction) – multipliers for the equality constraints.

Why this works: suppose x violates the inequality $f_i(x) \leq 0$, i.e. $f_i(x) > 0$. Then $\lambda_i f_i(x) > 0$ for any $\lambda_i \geq 0$, which inflates $\mathcal{L}$ – so this is bad if we’re minimising. The multipliers automatically penalise violations.

19.2 The dual function

For each fixed (λ, ν) , minimise the Lagrangian over x :

$$g(\lambda, \nu) = \min_x \mathcal{L}(x, \lambda, \nu).$$

This g is called the **dual function**. Then maximise g over the multipliers:

$$\max_{\lambda \geq 0, \nu} g(\lambda, \nu).$$

That’s the **dual problem**.

19.3 Weak and strong duality

Weak duality (always holds): $\max g \leq \min f_0$.

The dual gives a lower bound on the primal.

Strong duality (holds for nice convex problems with constraint qualifications):

$$\max g = \min f_0.$$

The dual is exactly equal to the primal.

For SVMs, strong duality holds, so we can solve either problem.

19.4 The SVM dual – the punchline

After applying these tools to the soft-margin SVM, the dual problem turns out to be:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle \quad \text{subject to} \quad 0 \leq \alpha_i \leq C, \sum_{i=1}^n \alpha_i y_i = 0.$$

Reading This Formula

- $\alpha = (\alpha_1, \dots, \alpha_n)$ – the Lagrange multipliers, one per training example. These are now our optimisation variables.
- $\sum_i \alpha_i$ – a linear term (we want α 's big).
- $\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$ – a quadratic term. Note: **the data x_i enter only through inner products $\langle x_i, x_j \rangle$** . This is the seed of the kernel trick.
- $\sum_i \alpha_i y_i = 0$ – balances positive and negative classes.
- $0 \leq \alpha_i \leq C$ – multipliers stay in a box.

Why this matters: we've turned a problem about w (which lives in the same space as features – could be huge) into a problem about α (one number per training example). And the data appear only through inner products, which we can replace with kernels for non-linear classification.

Practice Problem

1. State the SVM dual problem. What is the optimisation variable?
2. Why is the dual useful even though it produces the same answer as the primal?
3. Where in the dual do the data points appear? Why is this important?

Solution

- (1) Maximise $\sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$ subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$. The optimisation variable is α , with one entry per training point.
- (2) The dual has only one variable per training point (instead of one per feature, plus b , plus the slacks). When the feature space is huge or infinite (kernels!), this is a massive simplification.
- (3) Through inner products $\langle x_i, x_j \rangle$. This is crucial because we can replace each inner product with a kernel $k(x_i, x_j)$ and obtain non-linear classifiers without ever computing the high-dimensional features. This is the kernel trick (Topic 25).

20 KKT Conditions

Difficulty: Hard for absolute beginners. Mostly used to identify SVM support vectors.

- **How to read:** Skim the four conditions; focus on the SVM application.

Why We Care

For a convex problem with strong duality, the optimal primal and dual solutions satisfy four conditions called the **Karush–Kuhn–Tucker (KKT) conditions**. These conditions are how, in the SVM, we identify which training points end up being support vectors.

20.1 The four KKT conditions

Key Formula

$$(1) \text{ Stationarity: } \nabla f_0(x^*) + \sum_i \lambda_i^* \nabla f_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0$$

$$(2) \text{ Primal feasibility: } f_i(x^*) \leq 0, \quad h_j(x^*) = 0$$

$$(3) \text{ Dual feasibility: } \lambda_i^* \geq 0$$

$$(4) \text{ Complementary slackness: } \lambda_i^* f_i(x^*) = 0, \quad \text{for all } i$$

Reading This Formula

The starred quantities are the optimal values. Plain English versions:

- **(1) Stationarity:** at the optimum, the gradient of the Lagrangian is zero. Just like an unconstrained problem – the Lagrangian’s flat point IS the answer.
- **(2) Primal feasibility:** the optimum has to actually satisfy the original constraints.
- **(3) Dual feasibility:** inequality multipliers are non-negative. This is just the rule for inequality multipliers.
- **(4) Complementary slackness:** this is the magical one. For each inequality constraint, EITHER the multiplier is zero OR the constraint is tight ($f_i(x^*) = 0$, no slack). Constraints that are slack “don’t matter” at the optimum and have multiplier 0.

20.2 What KKT tells us about SVM support vectors

Apply complementary slackness to the soft-margin SVM constraint $1 - \xi_i - y_i(w^T x_i + b) \leq 0$ with multiplier α_i :

$$\alpha_i \cdot [1 - \xi_i - y_i(w^T x_i + b)] = 0.$$

This gives three cases:

α_i	ξ_i	Where is x_i ?
$\alpha_i = 0$	$\xi_i = 0$	$y_i(w^T x_i + b) > 1$, well outside margin – not a SV
$0 < \alpha_i < C$	$\xi_i = 0$	$y_i(w^T x_i + b) = 1$, on the margin – margin SV
$\alpha_i = C$	$\xi_i > 0$	inside the margin or misclassified – bound SV

So **support vectors are exactly the training points with $\alpha_i > 0$** . Everything else can be thrown away after training.

Worked Example

You've solved the SVM dual and got $\alpha = (0, 0.5, 0.3, 0.8, 0)$ with $C = 1$.

Support vectors: indices 2, 3, 4 (the ones with non-zero α).

All three have $0 < \alpha_i < C$, so all three are margin support vectors – exactly on the margin.

$$w = \sum_{i \in \mathcal{S}} \alpha_i y_i x_i.$$

$$b = y_j - w^T x_j \text{ for any margin support vector } j.$$

Exam Focus

A common exam problem: given a small SVM dataset and the corresponding α values, identify the support vectors and compute w and b . You should be able to do this entirely from the formulas above.

Practice Problem

You have trained a soft-margin SVM with $C = 2$ on 5 training points. The dual gave you:

$$\alpha = (0, 1.2, 2.0, 0.8, 0).$$

For each i :

1. Identify which examples are support vectors.
2. Categorise each support vector as “margin SV” (lies on margin) or “bound SV” (inside margin / misclassified).

Solution

Recall: $\alpha_i = 0 \Rightarrow$ not a SV. $0 < \alpha_i < C \Rightarrow$ margin SV. $\alpha_i = C \Rightarrow$ bound SV (inside margin).

- $\alpha_1 = 0$: not a SV.
- $\alpha_2 = 1.2$: in $(0, C)$ since $C = 2$. **Margin SV.**
- $\alpha_3 = 2.0 = C$. **Bound SV** (inside margin or misclassified).
- $\alpha_4 = 0.8$: in $(0, C)$. **Margin SV.**
- $\alpha_5 = 0$: not a SV.

So $\mathcal{S} = \{2, 3, 4\}$.

$$w = \sum_{i \in \mathcal{S}} \alpha_i y_i x_i.$$

b can be computed from any margin SV (here, point 2 or point 4): $b = y_j - w^T x_j$.

Part IV

Classification, Perceptron and Support Vector Machines

21 Classification – The Big Picture

Difficulty: Easy. Mostly vocabulary and the confusion matrix. • **How to read:** Read once. Memorise precision, recall, and F1.

Why We Care

Regression predicts a number (price, temperature). Classification predicts a *label* from a small set of categories: spam vs. ham, benign vs. malignant tumour, dog vs. cat vs. bird. Most of the machine-learning “intelligence” you see in the world today – diagnosing diseases, filtering email, identifying photos – is classification under the hood.

21.1 The setup

You have n training examples (x_i, y_i) , where y_i comes from a finite set of class labels.

- **Binary classification:** only two classes, e.g. $y \in \{0, 1\}$ or $y \in \{-1, +1\}$.
- **Multi-class classification:** K classes, $y \in \{1, 2, \dots, K\}$.

A trained classifier is a function h that maps any new x to a predicted class:

$$\hat{y} = h(x).$$

The classifier carves up feature space into **decision regions** – one region per class.

21.2 The 0-1 loss and accuracy

The most natural way to score a classifier: count the fraction of wrong predictions.

$$\ell_{0-1}(y, \hat{y}) = \mathbb{K}[y \neq \hat{y}] = \begin{cases} 0 & \text{if } y = \hat{y}, \\ 1 & \text{otherwise.} \end{cases}$$

$$\text{Accuracy} = 1 - \frac{1}{n} \sum_{i=1}^n \mathbb{K}[y_i \neq \hat{y}_i].$$

The catch: the 0-1 loss is non-differentiable (it jumps from 0 to 1) and non-convex. We can't optimise it directly with gradients. That's why every actual classifier uses a *surrogate* loss instead: hinge for SVM, cross-entropy for logistic regression, exponential for boosting.

21.3 The confusion matrix

For binary classification with classes “positive” and “negative”:

Truth\Predicted	Positive	Negative
Positive	TP (true positive)	FN (false negative)
Negative	FP (false positive)	TN (true negative)

From the confusion matrix we get more nuanced metrics:

$$\begin{aligned} \text{Precision} &= \frac{\text{TP}}{\text{TP} + \text{FP}} && \text{“Of those flagged positive, how many really are?”} \\ \text{Recall (TPR)} &= \frac{\text{TP}}{\text{TP} + \text{FN}} && \text{“Of all true positives, how many did we catch?”} \\ F_1 &= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} && \text{Harmonic mean of the two; balances them.} \\ \text{FPR} &= \frac{\text{FP}}{\text{FP} + \text{TN}} && \text{“Of true negatives, how many did we wrongly flag?”} \end{aligned}$$

Intuition

When does plain accuracy mislead? When classes are imbalanced. If 99% of emails are not spam and you predict “not spam” for everything, you get 99% accuracy – but you catch zero spam. Recall is 0. In imbalanced problems, look at precision and recall, not just accuracy.

21.4 The ROC curve and AUC

Many classifiers output a *score* (e.g. a probability) rather than a hard label. You then threshold the score: predict positive if score > threshold. Different thresholds give different (TPR, FPR) pairs.

The **ROC curve** plots TPR vs. FPR as the threshold sweeps from 0 to 1. An ideal classifier hugs the top-left corner. A random classifier traces the diagonal.

The **AUC** (Area Under the Curve) is a single number summary: AUC = 1 is perfect; AUC = 0.5 is random guessing. Equivalently, AUC is the probability that a random positive scores higher than a random negative.

Practice Problem

A spam filter is tested on 1000 emails. Confusion matrix:

	Predicted spam	Predicted ham
Actual spam	80	20
Actual ham	30	870

1. Compute accuracy.
2. Compute precision (for the spam class).
3. Compute recall (for the spam class).
4. Compute the F_1 score.

Solution

TP = 80, FN = 20, FP = 30, TN = 870.

(1) Accuracy = $(80 + 870)/1000 = 0.95$.

(2) Precision = $80/(80 + 30) = 80/110 \approx 0.727$.

(3) Recall = $80/(80 + 20) = 80/100 = 0.80$.

(4) $F_1 = 2 \cdot \frac{0.727 \cdot 0.80}{0.727 + 0.80} = 2 \cdot \frac{0.582}{1.527} \approx 0.762$.

22 The Perceptron Algorithm

Difficulty: Easy. Just trace the update rule. • **How to read:** Read once. The XOR limitation is the key lesson.

Why We Care

The perceptron is the great-grandfather of all modern neural networks. Frank Rosenblatt invented it in 1957 to mimic how brain neurons might learn. Today it's mostly a teaching tool, but it embodies the core idea of *adjusting weights based on mistakes* that powers everything from SVMs to deep learning.

22.1 The model and the learning rule

For binary classification with $y_i \in \{-1, +1\}$:

1. Initialise weight vector $w = 0$.
2. Repeatedly cycle through the training examples. For each (x_i, y_i) :
 - Predict $\hat{y}_i = \text{sign}(w^T x_i)$.
 - If $\hat{y}_i \neq y_i$ (misclassified, equivalently $y_i(w^T x_i) \leq 0$), update:

$$w \leftarrow w + y_i x_i.$$

3. Stop when no examples are misclassified.

Reading This Formula

Why does this update make sense?

- Suppose we wrongly predicted negative on a positive example ($y_i = +1$). Then $w^T x_i$ was too small. Adding x_i to w increases $w^T x_i$ in subsequent rounds – pushing the prediction in the right direction.
- Suppose we wrongly predicted positive on a negative example ($y_i = -1$). Then $w^T x_i$ was too big. Adding $-x_i$ to w decreases it. Combined: $w \leftarrow w + y_i x_i$ in both cases.

The size of each update is just the size of the misclassified example (no learning rate to tune).

22.2 Convergence – the Novikoff theorem

If the data are linearly separable – meaning some hyperplane perfectly separates the two classes – then the perceptron is guaranteed to converge in a bounded number of steps.

Specifically, if there exists a unit-norm w^* with $y_i(w^{*T} x_i) \geq \gamma$ for all i (margin at least γ), and all $\|x_i\| \leq R$, then the perceptron makes at most $(R/\gamma)^2$ updates.

22.3 The big limitation: XOR

The perceptron can only learn *linearly separable* concepts – ones where some straight line (or hyperplane) separates the classes.

XOR is the famous counter-example. Two features each in $\{0, 1\}$, label = XOR of the features:

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 0.$$

You cannot draw any straight line that separates the 1s from the 0s in this dataset. The perceptron will cycle forever.

This is why later methods (SVM with kernels, neural networks) had to introduce non-linearity.

Worked Example

$$w^{(0)} = (0, 0).$$

Example 1: $(x_1, y_1) = ((1, 1), +1)$.

$$y_1 w^T x_1 = 1 \cdot (0 \cdot 1 + 0 \cdot 1) = 0 \leq 0. \text{ Misclassified.}$$

$$\text{Update: } w \leftarrow (0, 0) + 1 \cdot (1, 1) = (1, 1).$$

Example 2: $(x_2, y_2) = ((-1, 1), -1)$.

$$y_2 w^T x_2 = (-1) \cdot (1 \cdot (-1) + 1 \cdot 1) = (-1)(0) = 0 \leq 0. \text{ Misclassified.}$$

$$\text{Update: } w \leftarrow (1, 1) - 1 \cdot (-1, 1) = (2, 0).$$

And so on.

Practice Problem

Run the perceptron with $w^{(0)} = (0, 0)$ on these training points:

$$(x_1, y_1) = ((2, 1), +1), \quad (x_2, y_2) = ((1, -2), -1), \quad (x_3, y_3) = ((-1, 1), +1).$$

Process the points in order and report w after each update.

Solution

Initial: $w = (0, 0)$.

Point 1: $y_1 \cdot w^T x_1 = (+1)(0) = 0 \leq 0$. Misclassified.

$$w \leftarrow (0, 0) + (+1)(2, 1) = (2, 1).$$

Point 2: $y_2 \cdot w^T x_2 = (-1)(2 \cdot 1 + 1 \cdot (-2)) = (-1)(0) = 0 \leq 0$. Misclassified.

$$w \leftarrow (2, 1) + (-1)(1, -2) = (1, 3).$$

Point 3: $y_3 \cdot w^T x_3 = (+1)(1 \cdot (-1) + 3 \cdot 1) = (+1)(2) = 2 > 0$. **Correct** – no update.

Final $w = (1, 3)$. Continue cycling through the points until no errors – you may need more passes in real problems.

23 Hard-Margin Classification

Difficulty: Easy. Just one optimisation problem. • **How to read:** Read once. The "minimise $\|w\|^2 = \text{maximise margin}$ " idea is the take-away.

Why We Care

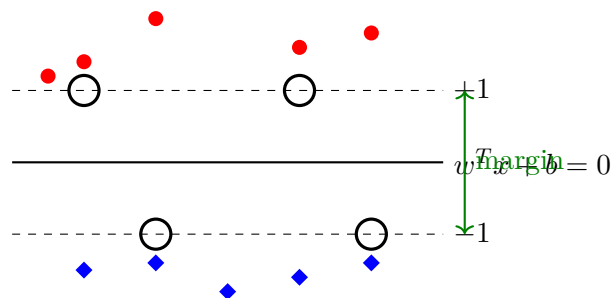
The perceptron finds *some* separating hyperplane (when one exists). But many hyperplanes might separate the data; not all are equally good. A hyperplane that just barely separates the classes is fragile – a small change in data flips many predictions. We want the *maximum-margin* hyperplane: the one that leaves the largest buffer zone between the classes.

23.1 What is the margin?

For a hyperplane $w^T x + b = 0$ and a point x_i , the perpendicular distance from the point to the hyperplane is

$$\frac{|w^T x_i + b|}{\|w\|_2}.$$

The **geometric margin** of the hyperplane is the smallest such distance over all training points. We want it to be as large as possible.



The dashed lines are the “margin boundaries”. Points with circles around them are the closest training points to the boundary – the **support vectors**. All other points could be moved or deleted without changing the optimal hyperplane.

23.2 The hard-margin SVM optimisation

Key Formula

$$\min_{w,b} \frac{1}{2} \|w\|_2^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1, \quad \text{for all } i.$$

Reading This Formula

- $\frac{1}{2} \|w\|_2^2$ – minimising the squared length of w . The geometric margin equals $1/\|w\|$, so making $\|w\|$ small is the same as making the margin big. The $1/2$ is a math convenience; it doesn’t change the optimum.

- $y_i(w^T x_i + b) \geq 1$ – this says every training point lies on or beyond its correct margin boundary. For a positive point, $w^T x_i + b \geq +1$. For a negative one, $w^T x_i + b \leq -1$. Multiplying by y_i unifies both cases.
- The constraint forces correct classification *and* a margin of at least $1/\|w\|$.

What it produces: the unique hyperplane that correctly separates the data and has the largest possible margin. This works only if the data are linearly separable.

23.3 The catch

Hard-margin SVM requires perfect separation. Real data is rarely perfectly separable – there’s noise, outliers, label errors. In that case, the optimisation has no feasible solution. We need the soft version, coming next.

Practice Problem

Hard-margin SVM finds w minimising $\frac{1}{2}\|w\|^2$ subject to $y_i(w^T x_i + b) \geq 1$ for all i .
Two training points: $(x_1, y_1) = ((2, 0), +1)$ and $(x_2, y_2) = ((-2, 0), -1)$.

1. Find w and b for the maximum-margin hyperplane.
2. Compute the geometric margin.
3. Which points are support vectors?

Solution

(1) By symmetry, the boundary should be $x_1 = 0$ (the y -axis). This corresponds to $w = (1, 0)$, $b = 0$ (*after* normalising constraints).

Check the constraint at point 1: $(+1)(1 \cdot 2 + 0 \cdot 0 + 0) = 2 \geq 1$. ✓

But this gives margin $1/\|w\| = 1$, and the constraint slack is 2 not 1 – so we can shrink w . Try $w = (1/2, 0)$, $b = 0$.

At point 1: $(+1)(0.5 \cdot 2) = 1 \geq 1$. ✓

At point 2: $(-1)(0.5 \cdot (-2)) = 1 \geq 1$. ✓

This is feasible and $\|w\|^2 = 0.25$.

(2) Geometric margin $= 1/\|w\| = 1/0.5 = 2$.

(3) Both points are support vectors – they sit exactly on the margin (their constraints are tight).

24 Soft-Margin SVM

Difficulty: Moderate. Builds on hard margin + KKT. • **How to read:** Read carefully. The α -table interpretation is the most common exam question.

Why We Care

Real datasets aren't perfectly separable. We want a method that finds a big-margin hyperplane while *tolerating* some misclassifications. The trick is to allow each point a bit of “slack”, and trade off margin width against total slack.

24.1 The optimisation problem

Key Formula

$$\min_{w,b,\xi} \frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i.$$

Reading This Formula

- ξ_i – **slack variable** for example i . $\xi_i = 0$ means example i obeys the margin. $\xi_i > 0$ means it has slipped into the margin. $\xi_i > 1$ means it has crossed to the wrong side of the boundary – a misclassification.
- $C \sum \xi_i$ – total “slack budget” penalty. Large C = small budget = tolerate few violations. Small C = generous budget = more violations OK if it widens the margin.
- $y_i(w^T x_i + b) \geq 1 - \xi_i$ – the same separation constraint as hard margin, weakened by ξ_i .

Choosing C : this is a hyperparameter. Picked by cross-validation.

24.2 Equivalent “hinge loss” formulation

We can eliminate the ξ_i variables and write the soft-margin SVM as unconstrained minimisation of:

$$\frac{1}{2} \|w\|_2^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b)).$$

Reading This Formula

- $\max(0, 1 - y_i(w^T x_i + b))$ – the **hinge loss** on example i .
 - If $y_i(w^T x_i + b) \geq 1$: example correctly classified with margin – loss is 0.
 - If $0 < y_i(w^T x_i + b) < 1$: correct side but inside margin – loss between 0 and 1.
 - If $y_i(w^T x_i + b) < 0$: misclassified – loss bigger than 1.

This form makes it look like a regularised loss: “hinge loss + ridge regulariser”. We could even train it with gradient descent.

24.3 Identifying support vectors after training

Solving the dual gives us α_i for each training point. Recall the three categories from KKT (Topic 20):

α_i	ξ_i	Position
0	0	outside margin (not a SV)
$0 < \alpha_i < C$	0	exactly on margin (margin SV)
C	> 0	inside margin or misclassified

The final hyperplane is determined entirely by support vectors:

$$w = \sum_{i \in \mathcal{S}} \alpha_i y_i x_i, \quad b = y_j - w^T x_j \text{ for any margin SV } j.$$

Intuition

Why is the SVM so popular?

1. It maximises a clear, geometrically meaningful quantity (the margin).
2. Only support vectors matter – the model is sparse in training data.
3. Combined with kernels (Topic 25), it handles non-linear boundaries elegantly.
4. It has strong theoretical generalisation guarantees.

Practice Problem

After training a soft-margin SVM with $C = 1$ on 5 points, you obtain:

$$y = (+1, +1, -1, -1, +1), \quad \alpha = (0.0, 0.5, 0.7, 1.0, 0.3).$$

The points are $x_1 = (2, 1)$, $x_2 = (1, 0)$, $x_3 = (-1, -1)$, $x_4 = (0, -2)$, $x_5 = (3, 2)$.

1. Identify the support vectors and classify them (margin SV vs. bound SV).
2. Compute $w = \sum_{i \in \mathcal{S}} \alpha_i y_i x_i$.

Solution

(1) With $C = 1$:

- $\alpha_1 = 0$: not a SV.
- $\alpha_2 = 0.5 \in (0, 1)$: margin SV.
- $\alpha_3 = 0.7 \in (0, 1)$: margin SV.
- $\alpha_4 = 1.0 = C$: bound SV (inside margin).
- $\alpha_5 = 0.3 \in (0, 1)$: margin SV.

$$\begin{aligned}
 \text{(2) } w &= (0.5)(+1)(1, 0) + (0.7)(-1)(-1, -1) + (1.0)(-1)(0, -2) + (0.3)(+1)(3, 2) \\
 &= (0.5, 0) + (0.7, 0.7) + (0, 2) + (0.9, 0.6) \\
 &= (0.5 + 0.7 + 0 + 0.9, \quad 0 + 0.7 + 2 + 0.6) \\
 &= (2.1, 3.3).
 \end{aligned}$$

25 Kernel Functions

Difficulty: Hard. The most abstract idea so far. • **How to read:** Read carefully twice. The polynomial kernel is the easiest path to understanding.

Why We Care

The XOR problem (Topic 22) showed that linear classifiers fail on non-linearly-separable data. One fix: invent non-linear features by hand, like x_1^2 , x_1x_2 , $\sqrt{x_1}$. Then a linear boundary in the augmented feature space corresponds to a curved boundary in the original space.

The problem: this feature explosion can be expensive (or infinite). If you square 10 features and include all pairs, you get 65 new features. With cubic terms, far more.

The **kernel trick** sidesteps this: it lets us use the augmented features *without ever computing them*.

25.1 The idea

A **feature map** ϕ takes a raw input x and produces a higher-dimensional feature vector $\phi(x)$. In the new space, we hope the data become linearly separable.

A **kernel function** is one that secretly computes inner products in feature space:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle.$$

The crucial observation: if our algorithm only ever uses inner products of inputs (like the SVM dual!), then we can replace every $\langle x_i, x_j \rangle$ with $k(x_i, x_j)$ and operate as though we'd built the big feature vectors – without ever building them.

25.2 Mercer's theorem – when is k a valid kernel?

A function $k(x, x')$ is a valid kernel *if and only if* for any choice of n points, the $n \times n$ matrix K with $K_{ij} = k(x_i, x_j)$ is symmetric and positive semi-definite.

You don't need to verify this in practice; the standard kernels are all valid.

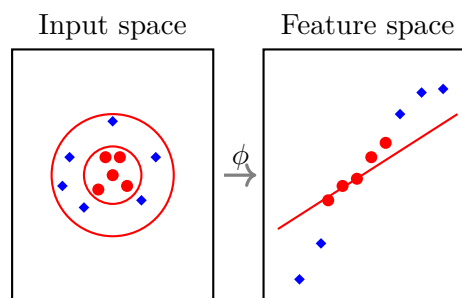
25.3 Common kernels

Kernel	Formula
Linear	$k(x, x') = x^T x'$
Polynomial (degree d)	$k(x, x') = (x^T x' + c)^d, c \geq 0$
RBF / Gaussian	$k(x, x') = \exp(-\gamma \ x - x'\ _2^2), \gamma > 0$
Sigmoid	$k(x, x') = \tanh(\kappa x^T x' + \theta)$

Reading This Formula

- **Linear kernel** = no transformation. Equivalent to vanilla SVM.
- **Polynomial kernel** corresponds to all polynomial features up to degree d . Picking $d = 2$ adds quadratic terms; $d = 3$, cubic terms; etc.
- **RBF kernel** corresponds to an *infinite-dimensional* feature space. γ controls how “narrow” each example’s influence is: small γ = smooth boundaries, large γ = wiggly boundaries.
- RBF is the most popular kernel by far. Default choice when you have no special intuition about the data.

25.4 Picture



The data on the left can’t be separated by a line. After applying the feature map ϕ on the right, they can. The kernel trick computes the right-hand-side inner products without explicitly making the right-hand plot.

Worked Example

Polynomial kernel. $k(x, x') = (x^T x' + 1)^2$.

$x = (1, 2)$, $x' = (2, 3)$.

$x^T x' + 1 = 1 \cdot 2 + 2 \cdot 3 + 1 = 9$.

$k(x, x') = 9^2 = 81$.

The implicit feature map is $\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$ – six features.

You can verify $\langle \phi(x), \phi(x') \rangle$ also equals 81.

Practice Problem

Compute these kernel values.

1. Polynomial kernel of degree 2 with $c = 1$: $k(x, x') = (x^T x' + 1)^2$, with $x = (1, 2)$ and $x' = (3, 1)$.
2. RBF kernel: $k(x, x') = \exp(-\gamma \|x - x'\|^2)$ with $\gamma = 0.5$, $x = (1, 1)$, $x' = (3, 4)$.
3. Build the 3×3 Gram matrix using the linear kernel $k(x, x') = x^T x'$, with $x_1 = (1, 0)$, $x_2 = (0, 1)$, $x_3 = (1, 1)$.

Solution

(1) $x^T x' = (1)(3) + (2)(1) = 5$. $k = (5 + 1)^2 = 36$.

(2) $\|x - x'\|^2 = (1 - 3)^2 + (1 - 4)^2 = 4 + 9 = 13$. $k = \exp(-0.5 \cdot 13) = e^{-6.5} \approx 0.00150$.

Notice: distant points get exponentially small kernel values – which is why RBF SVMs make “local” decisions.

(3) Compute $K_{ij} = x_i^T x_j$:

- $K_{11} = (1)(1) + (0)(0) = 1$. $K_{12} = 0$. $K_{13} = 1$.
- $K_{21} = 0$. $K_{22} = 1$. $K_{23} = 1$.
- $K_{31} = 1$. $K_{32} = 1$. $K_{33} = 2$.

$$K = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}.$$

26 Kernel SVM

Difficulty: Moderate. Just plug kernels into the SVM dual. • **How to read:** Skim. The decision function $f(x) = \sum \alpha_i y_i k(x_i, x) + b$ is the take-away.

Why We Care

Combine soft-margin SVM with the kernel trick: replace every inner product $\langle x_i, x_j \rangle$ with $k(x_i, x_j)$, and you get an SVM that finds non-linear decision boundaries. This was the state-of-the-art classifier from about 1995 until the deep-learning revolution.

26.1 The kernelised dual

Key Formula

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0. \end{aligned}$$

Reading This Formula

This is the same as the standard SVM dual (Topic 19), with the inner products $\langle x_i, x_j \rangle$ swapped for the kernel $k(x_i, x_j)$. Everything else is unchanged. You feed in the $n \times n$ matrix K (the Gram matrix) instead of the data X , and out come the α 's.

26.2 The decision function

After training, predict on a new point x :

$$f(x) = \sum_{i \in S} \alpha_i y_i k(x_i, x) + b, \quad \hat{y}(x) = \text{sign}(f(x)).$$

Reading This Formula

- Notice we never compute w explicitly! When the implicit feature space is infinite-dimensional (RBF kernel), w wouldn't even fit in memory.
- We only need to keep the support vectors (with $\alpha_i > 0$) and evaluate the kernel between x and each one.
- Cost of prediction: roughly proportional to the number of support vectors, NOT the size of the implicit feature space.

This is what makes kernel methods so elegant: complex, high-dimensional ideas turn into operations on small Gram matrices.

26.3 Why kernel SVM works so well

- RBF kernel can represent essentially any smooth function.
- The margin objective gives strong regularisation, preventing overfitting.
- Hyperparameters (C and γ) are tuned by cross-validation.

Exam Focus

You should be able to:

- Compute kernel values $k(x_i, x_j)$ for given x_i, x_j using polynomial and RBF kernels.
- Build a Gram matrix K from a small dataset.
- State why the kernel trick lets us train in (effectively) infinite dimensions.

Practice Problem

You've trained an RBF-kernel SVM and got 3 support vectors with the following information:

$$\alpha_1 = 0.5, y_1 = +1, x_1 = (1, 1); \quad \alpha_2 = 0.7, y_2 = -1, x_2 = (3, 0); \quad \alpha_3 = 0.3, y_3 = +1, x_3 = (0, 2).$$

Bias term: $b = -0.4$. RBF kernel with $\gamma = 0.5$.

Predict the class of the new point $x = (2, 1)$.

Solution

Compute kernel values $k(x_i, x) = \exp(-0.5 \|x_i - x\|^2)$:

- $k(x_1, x) = \exp(-0.5 ((1 - 2)^2 + (1 - 1)^2)) = \exp(-0.5) \approx 0.6065$.
- $k(x_2, x) = \exp(-0.5 ((3 - 2)^2 + (0 - 1)^2)) = \exp(-1.0) \approx 0.3679$.
- $k(x_3, x) = \exp(-0.5 ((0 - 2)^2 + (2 - 1)^2)) = \exp(-2.5) \approx 0.0821$.

Decision function:

$$\begin{aligned} f(x) &= \alpha_1 y_1 k(x_1, x) + \alpha_2 y_2 k(x_2, x) + \alpha_3 y_3 k(x_3, x) + b \\ &= (0.5)(+1)(0.6065) + (0.7)(-1)(0.3679) + (0.3)(+1)(0.0821) + (-0.4) \\ &= 0.3033 - 0.2575 + 0.0246 - 0.4 \\ &= -0.330. \end{aligned}$$

$f(x) < 0 \Rightarrow$ predict class -1 .

Part V

Regularized Regression

27 Ridge Regression

Difficulty: Easy. Just OLS with a tweak. • **How to read:** Read once. The closed form is just OLS with $+\lambda I$.

(Reminder: OLS and matrix inverse was introduced in Topic 6.)

Why We Care

We've seen that OLS has trouble when features are correlated – the $X^T X$ matrix becomes nearly singular and small data changes lead to wildly different weights. The fix is **regularisation**: add a penalty that discourages weights from getting big. Ridge regression is the simplest and most popular form of regularisation.

27.1 The Ridge objective

Key Formula

$$\hat{\theta}_{\text{ridge}} = \arg \min_{\theta} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2.$$

Reading This Formula

- $\|y - X\theta\|_2^2$ – the usual squared-error loss.
- $\|\theta\|_2^2 = \theta_1^2 + \theta_2^2 + \dots + \theta_d^2$ – the squared length of the weight vector. Penalising this *shrinks* all weights towards zero.
- $\lambda \geq 0$ – the **regularisation strength**. Big λ = more shrinkage; small λ = less.
 - $\lambda = 0$: identical to OLS.
 - $\lambda \rightarrow \infty$: all weights driven to zero.

Important: we usually do NOT penalise the intercept θ_0 in practice. Some textbooks make this explicit, others assume you've centred the data.

27.2 The closed-form Ridge solution

$$\hat{\theta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

Reading This Formula

Compare to OLS: $\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y$.

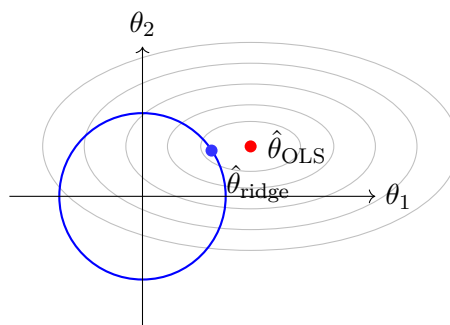
- The only difference is $X^T X + \lambda I$ instead of $X^T X$.
- I is the identity matrix (1's on the diagonal).
- Adding λI ensures the matrix is always invertible – even when $X^T X$ alone wouldn't be (multicollinearity, $p > n$).
- It's like adding a bit of “insurance” to make the inversion always work.

Practical implication: Ridge regression *always* produces an answer, even when OLS fails.

27.3 Properties of Ridge

- Coefficients shrink towards zero but *never become exactly zero*.
- Smaller variance than OLS but slightly biased.
- For λ chosen well by cross-validation, total MSE is lower than OLS's.
- **Always standardise features first.** The ℓ_2 penalty treats all features on the same scale; if they have wildly different units, the penalty is unfair.

27.4 Geometric picture



The grey ovals are RSS contours (the loss part). The blue circle is the Ridge constraint $\|\theta\|_2 \leq \text{const}$. The Ridge solution is where the smallest-RSS contour first kisses the circle – a smooth point on the boundary, never a corner.

27.5 Connection to MAP estimation

Ridge is exactly MAP estimation under a Gaussian prior on θ :

$$\theta \sim \mathcal{N}(0, \tau^2 I) \iff \lambda = \sigma^2 / \tau^2.$$

Big τ (loose prior) = small λ = less shrinkage.

Worked Example

$X = I_2$ (identity, two orthogonal features), $y = (3, 2)^T$, $\lambda = 1$.
 $X^T X = I$, $X^T X + \lambda I = 2I$, $(2I)^{-1} = \frac{1}{2}I$.

$$\hat{\theta}_{\text{ridge}} = \frac{1}{2}I \cdot I \cdot y = \frac{1}{2}y = (1.5, 1.0)^T.$$

Compare to OLS: $\hat{\theta}_{\text{OLS}} = y = (3, 2)^T$ – shrunk by 50%.

Practice Problem

Given $X = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$ and $y = (4, 3)^T$, compute the Ridge estimator with $\lambda = 1$.

Solution

$$X^T X = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}.$$

$$X^T X + \lambda I = \begin{pmatrix} 4+1 & 0 \\ 0 & 1+1 \end{pmatrix} = \begin{pmatrix} 5 & 0 \\ 0 & 2 \end{pmatrix}.$$

$$(X^T X + \lambda I)^{-1} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/2 \end{pmatrix} \text{ (diagonal matrix – inverse is reciprocals on the diagonal).}$$

$$X^T y = \begin{pmatrix} (2)(4) + (0)(3) \\ (0)(4) + (1)(3) \end{pmatrix} = \begin{pmatrix} 8 \\ 3 \end{pmatrix}.$$

$$\hat{\theta}_{\text{ridge}} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/2 \end{pmatrix} \begin{pmatrix} 8 \\ 3 \end{pmatrix} = \begin{pmatrix} 1.6 \\ 1.5 \end{pmatrix}.$$

$$\text{Compare to OLS: } \hat{\theta}_{\text{OLS}} = \begin{pmatrix} 1/4 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 8 \\ 3 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

Both Ridge coefficients are smaller than OLS – the penalty shrunk them.

28 LASSO

Difficulty: Easy. Soft thresholding is the only computation. • **How to read:** Read once. Memorise the soft thresholding rule.

Why We Care

Ridge shrinks all coefficients smoothly but *never* sets any to zero. If you suspect that only a few of your many features actually matter, you'd prefer a method that automatically picks them out – setting unimportant coefficients to exactly zero.

That's what LASSO does, and it's only a tiny tweak away from Ridge: swap $\|\theta\|_2^2$ for $\|\theta\|_1$.

28.1 The LASSO objective

Key Formula

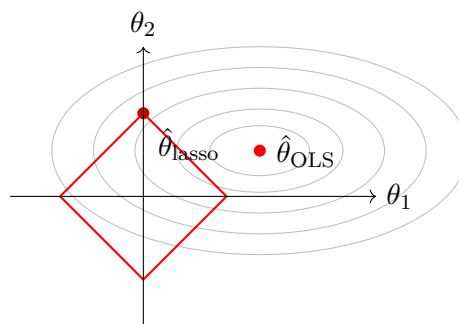
$$\hat{\theta}_{\text{lasso}} = \arg \min_{\theta} \|y - X\theta\|_2^2 + \lambda \|\theta\|_1, \quad \|\theta\|_1 = \sum_{j=1}^d |\theta_j|.$$

Reading This Formula

- $\|\theta\|_1$ – sum of absolute values of the weights. Called the ℓ_1 norm.
- Compare with Ridge's $\|\theta\|_2^2$ – sum of squared weights (ℓ_2 norm squared).
- The change from squaring to absolute value is small in writing but huge in effect: it produces *sparse* solutions.

There is no closed-form solution, because the $|\cdot|$ has a kink at 0 that makes calculus awkward. Algorithms for LASSO include coordinate descent and LARS.

28.2 Why LASSO produces zeros – the geometric reason



The constraint set $\|\theta\|_1 \leq t$ is a **diamond** – a square rotated 45 degrees – with corners on the axes. Where do RSS contours first touch the diamond? Usually at a corner, which means at least one $\theta_j = 0$.

Compare with Ridge's smooth circle, which has no preference for axes.

28.3 Soft thresholding (when X is orthonormal)

In the special case where $X^T X = I$, the LASSO solution has a clean closed form. Let $z = X^T y$. Then for each coefficient:

$$\hat{\theta}_j^{\text{lasso}} = \text{sign}(z_j) \max(|z_j| - \lambda/2, 0).$$

Reading This Formula

This is called **soft thresholding**:

- If $|z_j| \leq \lambda/2$: shrink to exactly 0.
- If $|z_j| > \lambda/2$: keep the sign and reduce magnitude by $\lambda/2$.

You can see the sparsity emerge: small z_j (weak signal) goes to zero; big z_j (strong signal) just gets nudged toward zero.

Worked Example

$z = (3, -1.5, 0.4)$, $\lambda/2 = 1$.

- $\hat{\theta}_1 = \text{sign}(3) \cdot \max(3 - 1, 0) = 2$.
- $\hat{\theta}_2 = \text{sign}(-1.5) \cdot \max(1.5 - 1, 0) = -0.5$.
- $\hat{\theta}_3 = \text{sign}(0.4) \cdot \max(0.4 - 1, 0) = 0$ – shrunk to zero.

28.4 Ridge vs. LASSO – when to use which

Property	Ridge	LASSO
Penalty	$\ \theta\ _2^2$	$\ \theta\ _1$
Solution	closed-form, easy	no closed form
Coefficients	shrunk but never zero	some are exactly zero
Best when	many features, all matter a bit	few features actually matter
Feature selection	no	yes (automatic)

28.5 Elastic Net – the best of both

If you can't decide, just blend them:

$$\hat{\theta}_{\text{EN}} = \arg \min_{\theta} \|y - X\theta\|_2^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2.$$

Two hyperparameters, more flexibility. Often outperforms either Ridge or LASSO alone in practice.

Practice Problem

With orthonormal X , the OLS solution gives $z = X^T y = (4, -2, 0.6, 1.2)$. Apply the LASSO soft-thresholding rule with $\lambda/2 = 1.5$ to find $\hat{\theta}_{\text{lasso}}$.

Solution

Recall: $\hat{\theta}_j = \text{sign}(z_j) \max(|z_j| - \lambda/2, 0)$.

- $z_1 = 4$: $\hat{\theta}_1 = \text{sign}(4) \max(4 - 1.5, 0) = (+1)(2.5) = 2.5$.
- $z_2 = -2$: $\hat{\theta}_2 = \text{sign}(-2) \max(2 - 1.5, 0) = (-1)(0.5) = -0.5$.
- $z_3 = 0.6$: $\hat{\theta}_3 = \text{sign}(0.6) \max(0.6 - 1.5, 0) = (+1)(0) = 0$. **Shrunk to zero!**
- $z_4 = 1.2$: $\hat{\theta}_4 = \text{sign}(1.2) \max(1.2 - 1.5, 0) = (+1)(0) = 0$. **Shrunk to zero!**

Final: $\hat{\theta}_{\text{lasso}} = (2.5, -0.5, 0, 0)$.

LASSO automatically dropped features 3 and 4 – their estimated effects were too small to survive the penalty.

29 Kernelized Ridge Regression

Difficulty: Moderate. Builds on Ridge + kernels. • **How to read:** Skim. The recipe and the closed form are the take-aways.

Why We Care

We saw kernels work magic in SVMs – non-linear decision boundaries with no explicit feature engineering. Can we apply the same trick to plain ridge regression? Yes. The result is a closed-form non-linear regression model that's surprisingly easy to use.

29.1 The representer theorem

For ridge regression, the optimal weight vector θ can always be written as a combination of training data:

$$\theta = X^T \alpha = \sum_{i=1}^n \alpha_i x_i,$$

for some coefficients $\alpha_1, \dots, \alpha_n$. This is a deep mathematical fact called the **representer theorem**.

Why this matters: we can re-parameterise the problem in terms of α (one number per training example) instead of θ (one number per feature). When the feature space is huge (kernels!) but the data set is small, this is a massive simplification.

29.2 The closed-form solution

Key Formula

$$\hat{\alpha} = (K + \lambda I_n)^{-1} y, \quad \text{where } K_{ij} = k(x_i, x_j).$$

$$\hat{f}(x) = \sum_{i=1}^n \hat{\alpha}_i k(x_i, x).$$

Reading This Formula

- K – the $n \times n$ Gram matrix of kernel values between all pairs of training points.
- $K + \lambda I_n$ – always invertible for $\lambda > 0$.
- $\hat{\alpha}$ – a vector of n numbers, one per training point.
- $\hat{f}(x) = \sum_i \hat{\alpha}_i k(x_i, x)$ – to predict at a new point x , take a weighted sum of kernel values between x and each training point.

Computational note: cost shifts from $\mathcal{O}(p^3)$ for plain ridge to $\mathcal{O}(n^3)$ for kernel ridge. Worth it when $p \gg n$.

29.3 Putting it all together

If you wanted non-linear regression and your kernel is RBF:

1. Compute $K_{ij} = \exp(-\gamma \|x_i - x_j\|^2)$ for all pairs.
2. Solve $(K + \lambda I) \alpha = y$.
3. Predict $\hat{f}(x) = \sum_i \alpha_i k(x_i, x)$.

That's it – non-linear regression in three lines, with two hyperparameters (λ and γ) chosen by cross-validation.

Intuition

The deep idea here: **any algorithm that uses inputs only via inner products can be “kernelised”**. We've now seen this for SVM and Ridge. It also works for PCA (kernel PCA), k -means (kernel k -means), Fisher discriminant analysis, and many more. Once you spot the inner product, the kernel trick is one substitution away.

Practice Problem

You have $n = 2$ training points $x_1 = 1$ and $x_2 = 2$ with targets $y_1 = 3$ and $y_2 = 5$. Use a linear kernel $k(x, x') = xx'$ and $\lambda = 1$.

1. Compute the 2×2 Gram matrix K .
2. Solve $(K + \lambda I) \alpha = y$ for α .
3. Predict at $x = 3$ using $\hat{f}(x) = \sum_i \alpha_i k(x_i, x)$.

Solution

$$(1) \ K = \begin{pmatrix} k(x_1, x_1) & k(x_1, x_2) \\ k(x_2, x_1) & k(x_2, x_2) \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}.$$

$$(2) \ K + I = \begin{pmatrix} 2 & 2 \\ 2 & 5 \end{pmatrix}.$$

$$\det = 10 - 4 = 6.$$

$$(K + I)^{-1} = \frac{1}{6} \begin{pmatrix} 5 & -2 \\ -2 & 2 \end{pmatrix}.$$

$$\alpha = (K + I)^{-1} y = \frac{1}{6} \begin{pmatrix} 5 & -2 \\ -2 & 2 \end{pmatrix} \begin{pmatrix} 3 \\ 5 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 15 - 10 \\ -6 + 10 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 5 \\ 4 \end{pmatrix} = \begin{pmatrix} 5/6 \\ 2/3 \end{pmatrix}.$$

$$(3) \text{ At } x = 3: \hat{f}(3) = \alpha_1 \cdot k(x_1, 3) + \alpha_2 \cdot k(x_2, 3) = (5/6)(1 \cdot 3) + (2/3)(2 \cdot 3) = 2.5 + 4 = 6.5.$$

Part VI

Probabilistic Classification

30 Logistic Regression

Difficulty: Foundational. The classification analogue of linear regression. • **How to read:** Read carefully. Memorise the sigmoid and the cross-entropy loss.

Why We Care

Linear regression predicts a real number $\hat{y} \in \mathbb{R}$. But for binary classification we want a *probability* between 0 and 1 – the chance the example is positive. We can’t just use a linear combination of features ($\theta^T x$) because that can output any number, including negative ones or numbers above 1.

The fix is the **sigmoid function** – a smooth “squashing” function that maps any real number into $(0, 1)$. Combine it with a linear model and you get logistic regression – the workhorse of binary classification.

30.1 The sigmoid function

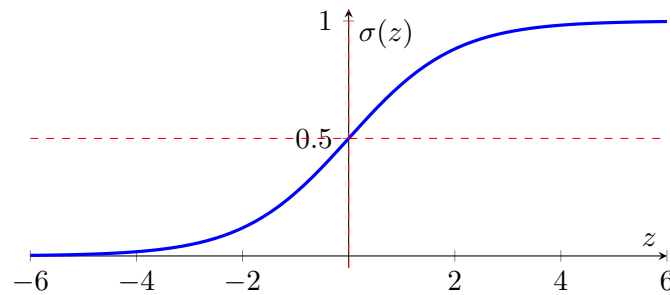
Key Formula

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Reading This Formula

- σ – “sigma”, the sigmoid (S-shaped) function.
- z – any real number, positive or negative.
- e^{-z} – the exponential function. $e \approx 2.718$.
 - When z is very positive, e^{-z} is tiny, so $1/(1 + \text{tiny}) \approx 1$.
 - When z is very negative, e^{-z} is huge, so $1/(1 + \text{huge}) \approx 0$.
 - When $z = 0$, $e^0 = 1$, so $\sigma(0) = 1/2$.

The sigmoid takes any real input and squashes it into $(0, 1)$ – exactly what we need for a probability.



30.2 The model

Logistic regression models the probability of the positive class as the sigmoid of a linear combination:

$$\mathbb{P}(y = 1 \mid x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}.$$

Decision rule: predict positive ($\hat{y} = 1$) if this probability is at least 0.5, which happens iff $\theta^T x \geq 0$.

30.3 The cross-entropy loss

We need a loss function for training. The natural one is the negative log-likelihood under our probabilistic model:

$$\ell_{\text{CE}}(\theta) = - \sum_{i=1}^n \left[y_i \ln h_{\theta}(x_i) + (1 - y_i) \ln(1 - h_{\theta}(x_i)) \right],$$

where $h_{\theta}(x) = \sigma(\theta^T x)$.

Reading This Formula

- $h_{\theta}(x_i)$ – the model’s predicted probability that $y_i = 1$.
- For an example with true label $y_i = 1$:
the loss contribution is $-\ln h_{\theta}(x_i)$.
If we predicted $h \approx 1$ (correct, confident): $-\ln(1) = 0$, no penalty.
If we predicted $h \approx 0$ (wrong, confident): $-\ln(0) = \infty$, huge penalty.
- For an example with true label $y_i = 0$:
the loss is $-\ln(1 - h_{\theta}(x_i))$. Symmetric.

The loss is convex, so gradient descent (or Newton’s method, called “IRLS” for logistic regression) finds the global optimum.

Gradient of cross-entropy (looks remarkably like linear regression’s gradient):

$$\nabla_{\theta} \ell_{\text{CE}}(\theta) = \sum_{i=1}^n (h_{\theta}(x_i) - y_i) x_i.$$

“Error times input, summed.”

Worked Example

$$\theta = (0.5, -1), \quad x = (1, 2).$$

$$\theta^T x = 0.5 \cdot 1 + (-1) \cdot 2 = -1.5.$$

$$\sigma(-1.5) = 1/(1 + e^{1.5}) \approx 1/5.482 \approx 0.182.$$

The predicted probability of being positive is 0.182. Since this is less than 0.5, predict $\hat{y} = 0$ (negative class).

30.4 Logistic regression as a discriminative model

Logistic regression directly models the conditional probability $\mathbb{P}(y | x)$. It does not say anything about how the inputs x are distributed. This makes it a **discriminative** model, in contrast with the **generative** models in the next two topics.

Intuition**Why is logistic regression so popular?**

- Closed-form gradient and (near-)closed-form Hessian – training is fast and reliable.
- Convex loss – no local optima.
- Outputs probabilities (not just labels) – valuable for downstream decisions.
- Well-calibrated when assumptions hold.
- Easily extended to multiple classes (softmax regression).

For binary classification with no special structure, logistic regression is usually a strong baseline.

Practice Problem

You've trained a logistic regression with $\theta = (-1, 2, -3)^T$ where the first coefficient is the intercept and the augmented input is $x = (1, x_1, x_2)$. Predict $\mathbb{P}(y = 1 | x)$ for these inputs:

1. $x = (1, 1, 1)$
2. $x = (1, 0, 1)$
3. $x = (1, 2, 0)$

For each, also state whether you'd predict class 1 or class 0.

Solution

$$\theta^T x = -1 \cdot 1 + 2x_1 - 3x_2.$$

$$(1) \theta^T x = -1 + 2 - 3 = -2. \quad \sigma(-2) = 1/(1 + e^2) = 1/8.389 \approx 0.119. \quad \text{Predict class 0.}$$

$$(2) \theta^T x = -1 + 0 - 3 = -4. \quad \sigma(-4) \approx 1/55 \approx 0.018. \quad \text{Predict class 0.}$$

$$(3) \theta^T x = -1 + 4 - 0 = 3. \quad \sigma(3) = 1/(1 + e^{-3}) \approx 1/1.05 \approx 0.953. \quad \text{Predict class 1.}$$

31 Gaussian Discriminant Analysis (GDA / LDA)

Difficulty: Moderate. Lots of formulas but each is intuitive. • **How to read:** Read once. The MLE estimates are just counts and averages.

Why We Care

Logistic regression takes a discriminative approach: “forget the input distribution; just model the boundary.” Generative models do the opposite: “model the full input distribution for each class, then use Bayes’ rule to derive the boundary.” GDA is the canonical example.

31.1 The generative model

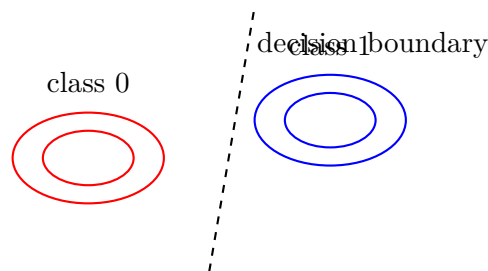
Assume:

- Class label $y \in \{0, 1\}$ with prior probability: $\mathbb{P}(y = 1) = \phi$. (So $\mathbb{P}(y = 0) = 1 - \phi$.)
- Given y , the features x follow a Gaussian:

$$x \mid y = k \sim \mathcal{N}(\mu_k, \Sigma).$$

Each class has its own mean μ_k , but *shares the same covariance matrix* Σ . (When the covariances are shared like this, the method is called **Linear Discriminant Analysis (LDA)**.)

In words: each class is a cloud of points shaped like a Gaussian, and the two clouds have the same shape – only their centres differ.



31.2 Estimating parameters by MLE

$$\hat{\phi} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[y_i = 1] \quad \text{(fraction of class 1 examples)}$$

$$\hat{\mu}_k = \frac{\sum_{i: y_i=k} x_i}{\sum_{i: y_i=k} 1} \quad \text{(mean of features within class } k\text{)}$$

$$\hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_{y_i})(x_i - \hat{\mu}_{y_i})^T \quad \text{(pooled covariance matrix)}$$

Reading This Formula

These look intimidating but are completely intuitive:

- $\hat{\phi}$: count how many examples are class 1, divide by total.
- $\hat{\mu}_k$: average the feature vectors of just the class- k examples.
- $\hat{\Sigma}$: each example contributes $(x_i - \hat{\mu}_{y_i})(x_i - \hat{\mu}_{y_i})^T$ – the squared deviation from its own class's mean. Average all those.

That's it – no iterative optimisation needed. Estimating GDA is basically counting and averaging.

31.3 Predicting with Bayes' rule

For a new x , compute

$$\mathbb{P}(y = k | x) = \frac{\mathbb{P}(x | y = k) \mathbb{P}(y = k)}{\sum_j \mathbb{P}(x | y = j) \mathbb{P}(y = j)}.$$

Predict the class with the higher posterior. When Σ is shared across classes, the boundary is a *straight line* (or hyperplane). When each class has its own covariance, the boundary is quadratic – this is called **QDA** (Quadratic Discriminant Analysis).

31.4 LDA vs. logistic regression

Both produce linear boundaries. Why use one over the other?

	LDA / GDA	Logistic regression
Type	generative	discriminative
Assumes	Gaussian classes, shared Σ	only that $\mathbb{P}(y x)$ is logistic
Best when	assumptions actually hold	robust to weird x distributions
Output	also models $\mathbb{P}(x)$	only $\mathbb{P}(y x)$

Intuition

Rule of thumb: with limited data and roughly Gaussian features, LDA wins (it uses the data more efficiently). With lots of data and unusual feature distributions, logistic regression is safer. With massive non-Gaussian data, neither works as well as a tree-based or neural method.

Practice Problem

Six 1-dimensional training points:

Class 0: $x = -2, -1, 0$.

Class 1: $x = 2, 3, 4$.

Estimate the GDA parameters by MLE:

1. $\hat{\phi}$ (prior of class 1).
2. $\hat{\mu}_0$ and $\hat{\mu}_1$.
3. $\hat{\sigma}^2$ (shared variance).

Solution

$n = 6$, $n_0 = 3$ (class 0), $n_1 = 3$ (class 1).

(1) $\hat{\phi} = n_1/n = 3/6 = 0.5$.

(2) Class means: $\hat{\mu}_0 = (-2 + (-1) + 0)/3 = -1$. $\hat{\mu}_1 = (2 + 3 + 4)/3 = 3$.

(3) Pooled deviations from each point's class mean:

Class 0: $(-2 - (-1))^2 + (-1 - (-1))^2 + (0 - (-1))^2 = 1 + 0 + 1 = 2$.

Class 1: $(2 - 3)^2 + (3 - 3)^2 + (4 - 3)^2 = 1 + 0 + 1 = 2$.

Sum: 4. $\hat{\sigma}^2 = 4/6 \approx 0.67$.

32 Naive Bayes

Difficulty: Easy. Counting + multiplication. • **How to read:** Read carefully. Always remember Laplace smoothing.

Why We Care

The full GDA model needs a covariance matrix Σ which is $p \times p$ – that’s $p(p+1)/2$ parameters to estimate. With many features and not much data, this becomes unreliable. Naive Bayes makes a drastic simplifying assumption: pretend all features are independent given the class. This makes the model trivial to estimate. Despite the simplification, Naive Bayes works astonishingly well on text classification, spam filtering, and other high-dimensional problems.

32.1 The naive assumption

Conditional on the class label, all features are independent of each other:

$$\mathbb{P}(x_1, x_2, \dots, x_p \mid y) = \prod_{j=1}^p \mathbb{P}(x_j \mid y).$$

Reading This Formula

This is the **naive** part. In real life, features are usually correlated even within a class (in spam emails, “free” and “offer” tend to co-occur). Naive Bayes pretends they don’t.

Why does it still work? Even when the assumption is violated, the resulting classifier often makes the right *decision*, even if its probability estimates are over-confident.

32.2 Classification rule

By Bayes’ rule, predict the class maximising:

Key Formula

$$\hat{y}(x) = \arg \max_{k \in \{1, \dots, K\}} \left[\ln \mathbb{P}(y = k) + \sum_{j=1}^p \ln \mathbb{P}(x_j \mid y = k) \right].$$

Reading This Formula

- $\mathbb{P}(y = k)$ – prior probability of class k . Estimated as the fraction of training examples in class k .
- $\mathbb{P}(x_j \mid y = k)$ – probability of feature value x_j given class k . Estimated from class- k training examples.
- We take logs so the product becomes a sum – numerically much more stable when probabilities are tiny.
- For each class, compute the sum. Pick the largest.

32.3 Variants

- **Multinomial NB:** x_j is a count (e.g. how many times a word appears). Used in text classification.
- **Bernoulli NB:** $x_j \in \{0, 1\}$ – presence/absence of a feature. Also for text.
- **Gaussian NB:** x_j is continuous, modelled as $\mathcal{N}(\mu_{jk}, \sigma_{jk}^2)$ within each class.

32.4 Laplace smoothing – the zero-frequency rescue

A nasty edge case: if a feature value never appears in some class during training, the estimated $\mathbb{P}(x_j | y)$ becomes 0. Then the entire product (or sum of logs) is 0 (or $-\infty$). One bad estimate ruins everything.

Fix: add a small “pseudo-count” to every category. For a categorical feature with V possible values:

$$\hat{\mathbb{P}}(x_j = v | y = k) = \frac{|\{i : y_i = k, x_{ij} = v\}| + 1}{|\{i : y_i = k\}| + V}.$$

Reading This Formula

- Numerator: count of class- k examples with $x_j = v$, plus 1.
- Denominator: count of class- k examples, plus V (so probabilities sum to 1).

The +1 is called **Laplace smoothing** or **add-one smoothing**. Even unseen feature values get a small non-zero probability, preventing the zero-product disaster.

Worked Example

Spam filter. Word “lottery” appears 0 times in 200 ham emails. $V = 2$ (“present” or “absent”).

Without smoothing: $\mathbb{P}(\text{lottery} | \text{ham}) = 0/200 = 0$.

Any spam email containing “lottery” gets $\mathbb{P}(\text{ham} | x) = 0$.

The classifier will always say “spam” if “lottery” appears, even when it shouldn’t.

With Laplace: $\mathbb{P}(\text{lottery} | \text{ham}) = (0 + 1)/(200 + 2) = 1/202 \approx 0.005$.

Small, but non-zero – the classifier behaves sensibly.

Exam Focus

On exams you’ll typically be given a tiny dataset and asked to:

- Compute $\mathbb{P}(y)$ from class frequencies.
- Compute each $\mathbb{P}(x_j | y)$, applying Laplace smoothing.
- Combine logs and pick the highest.

Always work in log-space for numerical stability.

Practice Problem

Out of 20 emails, 8 are spam and 12 are ham. Among spam emails, the word “offer” appears in 6 of them. Among ham emails, “offer” appears in 1.

1. Estimate $\mathbb{P}(\text{spam})$ and $\mathbb{P}(\text{ham})$ from the data.

2. Estimate $\mathbb{P}(\text{offer} \mid \text{spam})$ and $\mathbb{P}(\text{offer} \mid \text{ham})$ with Laplace smoothing (treat “offer” as binary: present or absent, so $V = 2$).
3. A new email contains the word “offer”. Compute (unnormalised) $\mathbb{P}(\text{spam} \mid \text{offer})$ and $\mathbb{P}(\text{ham} \mid \text{offer})$, and predict the class.

Solution

(1) $\mathbb{P}(\text{spam}) = 8/20 = 0.4$, $\mathbb{P}(\text{ham}) = 12/20 = 0.6$.

(2) With Laplace ($V = 2$):

- $\mathbb{P}(\text{offer} \mid \text{spam}) = (6 + 1)/(8 + 2) = 7/10 = 0.7$.
- $\mathbb{P}(\text{offer} \mid \text{ham}) = (1 + 1)/(12 + 2) = 2/14 \approx 0.143$.

(3) Unnormalised posteriors (using Bayes’ rule, ignoring the common denominator):

- $\mathbb{P}(\text{spam} \mid \text{offer}) \propto 0.7 \cdot 0.4 = 0.28$.
- $\mathbb{P}(\text{ham} \mid \text{offer}) \propto 0.143 \cdot 0.6 \approx 0.0858$.

$0.28 > 0.0858$, so predict **spam**.

(Optional normalisation: divide each by their sum ≈ 0.366 to get $\mathbb{P}(\text{spam} \mid \text{offer}) \approx 0.766$ and $\mathbb{P}(\text{ham} \mid \text{offer}) \approx 0.234$.)

Part VII

Ensemble Methods

33 Bagging (Bootstrap Aggregating)

Difficulty: Easy. Average a bunch of models. • **How to read:** Read once. Variance reduction is the punchline.

(Reminder: Bootstrap and OOB was introduced in Topic 14.)

Why We Care

A single decision tree, fit very deep, has low bias but enormous variance: re-train on slightly different data and you get a totally different tree. But what if we trained MANY trees on slightly different versions of the data and averaged them? Random fluctuations in individual trees would cancel out, leaving the genuine signal. That's bagging in one paragraph. “Bagging” = “Bootstrap AGGREGatING”.

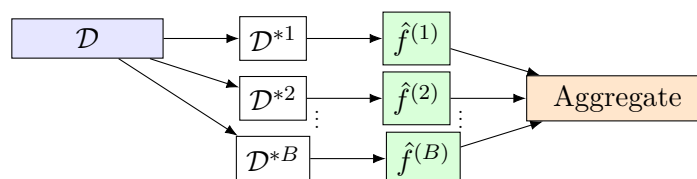
33.1 The bagging algorithm

- For $b = 1, 2, \dots, B$ (typical $B = 100$ to 500):
 - Draw a bootstrap sample $\mathcal{D}^{*b}$ of size n from $\mathcal{D}$ (sample with replacement).
 - Train a base model $\hat{f}^{(b)}$ on $\mathcal{D}^{*b}$.
- Combine the predictions:

Key Formula

$$\text{Regression: } \hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x).$$

$$\text{Classification: } \hat{y}_{\text{bag}}(x) = \text{majority vote of } \hat{f}^{(1)}(x), \dots, \hat{f}^{(B)}(x).$$



33.2 Why does bagging reduce variance?

If you average B independent random variables each with variance σ^2 , the average has variance σ^2/B . Larger $B \Rightarrow$ smaller variance.

For bagged trees, the trees aren't fully independent (they're trained on overlapping bootstrap samples), but they're decorrelated enough that averaging still helps a lot. More precisely, for trees with pairwise correlation ρ :

$$\text{Var}[\hat{f}_{\text{bag}}(x)] = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2.$$

The second term shrinks to zero with B . The first term – $\rho \sigma^2$ – doesn't. This is the floor: bagging cannot reduce variance below $\rho \sigma^2$, no matter how many trees.

That's why **Random Forests** (next topic) work hard to reduce ρ itself.

33.3 Out-of-bag (OOB) error – a free validation set

Recall from Topic 14: each bootstrap sample uses about 63% of the original data; the remaining 37% are out-of-bag for that sample.

For each training point x_i , you can collect the predictions of all trees that did NOT use x_i in their training (OOB predictions). Aggregate those, and you get an unbiased estimate of test error – without ever doing cross-validation.

33.4 Bias of bagging

Bagging **does not reduce bias**. If your base learner is biased (e.g. a stumpy tree that can't capture the truth), bagging it still gives a biased average. This is why bagging is best applied to **high-variance, low-bias** base learners – typically deep, unpruned decision trees.

Practice Problem

Suppose each individual decision tree in a bagged ensemble has variance $\sigma^2 = 4$ on a test prediction. The pairwise correlation between trees is $\rho = 0.3$.

1. Compute the variance of the bagged prediction with $B = 100$ trees.
2. Compute the variance with $B = 1000$ trees.
3. As $B \rightarrow \infty$, what is the limiting variance?
4. What lesson does this teach us about Random Forests?

Solution

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2.$$

$$(1) \quad 0.3 \cdot 4 + (0.7/100) \cdot 4 = 1.2 + 0.028 = 1.228.$$

$$(2) \quad 0.3 \cdot 4 + (0.7/1000) \cdot 4 = 1.2 + 0.0028 = 1.2028.$$

$$(3) \quad \text{As } B \rightarrow \infty, \text{ the second term vanishes and we're left with } \rho \sigma^2 = 1.2.$$

(4) The variance has a **floor** at $\rho \sigma^2$. No amount of B can drive it below 1.2 if ρ stays at 0.3. Random Forests attack ρ directly (by random feature subsampling), which lowers the floor.

34 Random Forests

Difficulty: Easy. Bagging plus a tweak. • **How to read:** Read once. Memorise the defaults: $m = \sqrt{p}$ for classification, $p/3$ for regression.

Why We Care

Random Forests are bagging plus a clever twist. In plain bagging, even though each tree sees a different bootstrap sample, all trees might still pick the same dominant feature for their first split, making them highly correlated. Random Forests forbid this: at every split, only a random subset of features is allowed. Trees become more diverse; ρ in our variance formula drops; the ensemble becomes more powerful.

34.1 The random forest algorithm

1. For $b = 1, 2, \dots, B$:
 - Bootstrap sample $\mathcal{D}^{*b}$ from $\mathcal{D}$.
 - Train a decision tree on $\mathcal{D}^{*b}$, but at every split:
 - Randomly pick m features out of p ($m < p$).
 - Find the best split using only those m features.
2. Aggregate (average for regression, majority vote for classification).

34.2 Default values for m

Key Formula

$$m = \lfloor \sqrt{p} \rfloor \quad (\text{classification}), \quad m = \lfloor p/3 \rfloor \quad (\text{regression}).$$

Reading This Formula

- $\sqrt{p}$ for classification: with $p = 100$ features, each split considers only about 10 features. Most splits will be suboptimal compared to the full-feature optimum, but the resulting trees are decorrelated enough that the ensemble is much better.
- $p/3$ for regression: a slightly larger fraction works better when outputs are continuous.
- These are defaults, not laws. Tune by cross-validation if performance matters.

34.3 Why feature subsampling helps

Recall the variance floor: $\rho \sigma^2$. Random feature subsampling attacks ρ directly. If ρ drops from 0.5 to 0.1, the floor drops 5-fold, even with the same number of trees.

Intuition

An ensemble's wisdom comes from disagreement among its members. Trees that all make the same mistakes (high ρ) provide no “averaging benefit”. Trees that make different mistakes (low ρ) cancel out their errors when averaged. Random feature subsampling forces the trees to disagree, in a smart way.

34.4 Hyperparameters and defaults

- B – number of trees. More is better; performance usually plateaus around 500.
- m – features per split. Defaults are good starting points.
- Tree depth – usually unlimited. Deep trees have low bias; the ensemble takes care of variance.
- Minimum leaf size – usually 1 or 5.

34.5 Why random forests are popular

- Excellent out-of-the-box performance with minimal tuning.
- OOB error gives free validation (no separate test set or CV needed).
- Built-in feature importance scores.
- Handles missing values and mixed feature types reasonably well.
- Robust to outliers.

Worked Example

$p = 100$ classification problem, default $m = \lfloor \sqrt{100} \rfloor = 10$.

At each split in each tree, pick 10 features at random out of 100, find the best split using just those 10. Train 500 such trees. For each test point, take majority vote. OOB error: 0.12.

That's it – a strong classifier with essentially zero hyperparameter tuning.

Practice Problem

1. For a classification problem with $p = 64$ features, what is the default value of m in a Random Forest?
2. For a regression problem with $p = 30$ features, what is the default m ?
3. Why doesn't a Random Forest with $m = p$ outperform plain bagging?

Solution

(1) $m = \lfloor \sqrt{64} \rfloor = 8$.

(2) $m = \lfloor 30/3 \rfloor = 10$.

(3) If $m = p$, every split considers all features and picks the truly best one – exactly like plain (un-randomised) bagging. All trees would tend to make similar splits, especially their first one, producing highly correlated trees. The variance reduction stays limited by the high ρ . Setting $m < p$ forces trees to consider different features at different splits, decorrelating them.

35 Boosting (AdaBoost)

Difficulty: Hard. Most-tested formulas in the guide. • **How to read:** Read carefully twice. Memorise the α_t formula and the weight update.

Why We Care

Bagging trains many independent learners and averages them. Boosting takes a completely different angle: train learners *sequentially*, where each new learner focuses on the examples the previous ones got wrong.

The original boosting algorithm, **AdaBoost** (Adaptive Boosting), turns a collection of barely-better-than-random weak learners into an arbitrarily good ensemble. This was a huge result in the 1990s.

35.1 The AdaBoost algorithm

For binary classification with $y_i \in \{-1, +1\}$:

1. Initialise weights: $w_i^{(1)} = 1/n$ for all i .
2. For $t = 1, 2, \dots, T$:
 - Train a weak classifier h_t that minimises the weighted error:

$$\varepsilon_t = \sum_{i=1}^n w_i^{(t)} \mathbb{I}[h_t(x_i) \neq y_i].$$

- Compute the classifier's strength:

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}.$$

- Update example weights and renormalise:

$$w_i^{(t+1)} \propto w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i)).$$

3. Final classifier:

$$H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right).$$

Reading This Formula

This is dense. Let me unpack each piece.

- $w_i^{(t)}$ – the “importance weight” of example i at round t . All start equal at $1/n$.
- h_t – the weak classifier trained at round t . Often a depth-1 “decision stump”.
- ε_t – weighted error rate. Sum of weights of misclassified examples.
- α_t – how much the new classifier counts in the final vote.
 - If $\varepsilon_t < 0.5$ (better than random), $\alpha_t > 0$.
 - If $\varepsilon_t = 0.5$ (random), $\alpha_t = 0$ – useless.
 - Smaller $\varepsilon_t \Rightarrow$ larger α_t .

- Weight update: examples misclassified by h_t get their weights *multiplied* by $e^{\alpha_t} > 1$, while correctly classified ones get *divided* by e^{α_t} . In words: “mistakes get more attention next round”.
- Final ensemble: weighted vote, where each h_t ’s vote is scaled by α_t .

35.2 Bagging vs. boosting at a glance

Aspect	Bagging	Boosting
Learners	parallel, independent	sequential, dependent
How learners differ	different bootstrap samples	different weights on examples
Reduces	variance	bias
Risk	slight underfit	overfit if T too large
Base learner	often deep / complex	often shallow / weak

35.3 The math made concrete

Worked Example

First round of AdaBoost. Suppose the weak classifier has weighted error $\varepsilon_1 = 0.3$ – it gets 30% of the (weighted) examples wrong.

Strength:

$$\alpha_1 = \frac{1}{2} \ln \frac{1 - 0.3}{0.3} = \frac{1}{2} \ln(2.333) \approx 0.424.$$

Weight multipliers:

- Misclassified examples: weight multiplied by $e^{+0.424} \approx 1.528$.
- Correctly classified: multiplied by $e^{-0.424} \approx 0.654$.

After renormalisation, total weight remains 1. Mistakes are now ≈ 2.3 times as important as correct ones. Round 2 will train a new weak learner that pays special attention to those mistakes.

35.4 When boosting overfits

For too-large T , boosting can eventually overfit. Modern variants (gradient boosting, XGBoost, LightGBM) combat this with **shrinkage** (multiplying α_t by a small “learning rate” like 0.1) and column subsampling.

Exam Focus

Two formulas to memorise cold:

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t}, \quad w_i^{(t+1)} \propto w_i^{(t)} e^{-\alpha_t y_i h_t(x_i)}.$$

Be ready to compute one round given ε_t and a small dataset.

Practice Problem

You're running AdaBoost with 5 training points, all initially weighted $w_i = 0.2$. After round 1, your weak classifier h_1 *misclassified* points 1 and 4, and got the rest correct.

1. Compute ε_1 .
2. Compute α_1 .
3. Compute the unnormalised new weights $\tilde{w}_i^{(2)}$ and then the normalised weights $w_i^{(2)}$.

Solution

(1) $\varepsilon_1 = \text{sum of weights of misclassified points} = 0.2 + 0.2 = 0.4$.

(2) $\alpha_1 = \frac{1}{2} \ln \frac{1-0.4}{0.4} = \frac{1}{2} \ln(1.5) = \frac{1}{2}(0.405) \approx 0.203$.

(3) Weight update: $\tilde{w}_i^{(2)} = w_i^{(1)} \exp(-\alpha_1 y_i h_1(x_i))$. For misclassified ($y_i h_1(x_i) = -1$), this is $w_i \exp(\alpha_1) \approx 0.2 \cdot 1.225 = 0.245$. For correctly classified ($y_i h_1(x_i) = +1$), this is $w_i \exp(-\alpha_1) \approx 0.2 \cdot 0.816 = 0.163$.

- Points 1, 4 (misclassified): $\tilde{w} \approx 0.245$ each.
- Points 2, 3, 5 (correct): $\tilde{w} \approx 0.163$ each.

Sum: $2(0.245) + 3(0.163) = 0.490 + 0.489 = 0.979$.

Normalise (divide each by the sum):

- Points 1, 4: $w \approx 0.245/0.979 \approx 0.250$.
- Points 2, 3, 5: $w \approx 0.163/0.979 \approx 0.167$.

The misclassified points went from weight 0.2 to 0.25 – now get more attention next round. Correctly classified points dropped to 0.167.

Part VIII

Unsupervised Learning

36 Unsupervised Learning – Big Picture

Difficulty: Easy. Vocabulary chapter. • **How to read:** Skim. The three core tasks (clustering, DR, density) are the take-aways.

Why We Care

Everything we’ve covered so far has been supervised: we had labels and tried to predict them. Often we don’t have labels. We just have a pile of data and want to make sense of it – find groups, summarise structure, visualise. That’s unsupervised learning.

36.1 Three core tasks

Task	Output	Examples
Clustering	a partition of the data into groups	k -means, GMM, DBSCAN
Dimensionality reduction	a low-dim representation of each pt	PCA, t-SNE, UMAP
Density estimation	a function $\hat{p}(x)$	KDE, GMM

In this guide we cover one method for each: k -means (clustering) and PCA (dimensionality reduction). Density estimation is a topic for a later course.

36.2 The big difficulty: evaluation without labels

In supervised learning we measure performance against true labels. In unsupervised learning there is no ground truth. How do we know if our clusters are “good” or our PCA is “capturing the right structure”?

- **Silhouette score** for clustering – measures how well-separated clusters are.
- **Reconstruction error** – how well the low-dimensional representation reconstructs the original data (used in PCA, autoencoders).
- **Downstream task performance** – if you have labels for *some* of the data, see whether the clustering / reduction helps a supervised task.
- **Domain expert inspection** – ultimately, look at the clusters and check if they make sense.

Practice Problem

Classify each task as clustering, dimensionality reduction, or density estimation.

1. Reducing a 1,000-pixel image to 20 numbers preserving its essence.
2. Grouping news articles by topic with no labels.
3. Modelling the distribution of website visit times to flag unusual visits.
4. Compressing customer profiles to a 5-dimensional code for visualisation.

Solution

1. Dimensionality reduction (PCA, autoencoder).
2. Clustering (k -means, hierarchical).
3. Density estimation (KDE, GMM). Unusual = low density.
4. Dimensionality reduction (PCA, t-SNE).

37 *K*-Means Clustering

Difficulty: Easy. Just two simple steps repeated. • **How to read:** Read once. Trace the worked example by hand.

Why We Care

You have 1000 customer profiles in some high-dimensional feature space (age, income, purchase frequency, ...). You believe there are roughly $K = 4$ natural “types” of customer. How do you find these types automatically? *K*-means is the simplest and most popular answer.

37.1 The objective: total within-cluster squared distance

Pick K **centroids** $\mu_1, \dots, \mu_K$ and assign each data point x_i to the nearest centroid. Define $c(i)$ as the cluster index assigned to point i . The objective:

Key Formula

$$J(c, \mu) = \sum_{i=1}^n \|x_i - \mu_{c(i)}\|_2^2.$$

Reading This Formula

- For each data point x_i , look at the centroid of its assigned cluster, $\mu_{c(i)}$.
- Compute the squared distance from x_i to that centroid.
- Sum over all points.

J is small when each point is close to its centroid. We want to choose centroids and assignments so J is as small as possible.

37.2 Lloyd’s algorithm

Joint minimisation over c and μ is hard, but we can alternate:

1. Initialise centroids $\mu_1, \dots, \mu_K$ randomly (or use *k-means++*, a smarter init).
2. **Assignment step.** For each point x_i , set $c(i) = \arg \min_k \|x_i - \mu_k\|_2^2$ – assign to the nearest centroid.
3. **Update step.** For each cluster k , set $\mu_k = \frac{1}{|\{i: c(i)=k\}|} \sum_{i: c(i)=k} x_i$ – new centroid is the mean of its assigned points.
4. Repeat 2 and 3 until assignments stop changing.

Convergence: the objective J never increases between steps, so the algorithm always converges. But it converges to a *local* minimum – different starting centroids can give different (sometimes worse) clusterings. Standard practice: run multiple times and keep the result with the smallest J .

37.3 A worked example in 1D

Worked Example

Data: $\{1, 2, 4, 8, 9\}$. $K = 2$. Initial centroids: $\mu_1 = 1$, $\mu_2 = 9$.

Iteration 1 – assign:

Distances to $\mu_1 = 1$: $(0, 1, 3, 7, 8)$.

Distances to $\mu_2 = 9$: $(8, 7, 5, 1, 0)$.

Assignments: $C_1 = \{1, 2, 4\}$, $C_2 = \{8, 9\}$.

Iteration 1 – update:

$\mu_1 = (1 + 2 + 4)/3 = 7/3 \approx 2.33$.

$\mu_2 = (8 + 9)/2 = 8.5$.

Iteration 2 – assign:

Same assignments as iteration 1. Algorithm has converged.

Final J :

$(1 - 2.33)^2 + (2 - 2.33)^2 + (4 - 2.33)^2 + (8 - 8.5)^2 + (9 - 8.5)^2 \approx 5.17$.

37.4 How do you pick K ?

- **Elbow method:** run k -means for $K = 1, 2, 3, \dots$. Plot J vs. K . J always decreases as K grows, but the decrease usually slows down dramatically at the “right” K – look for an elbow / knee.
- **Silhouette score:** for each point compute $s_i = (b_i - a_i) / \max(a_i, b_i)$, where a_i is its mean distance to other points in its cluster and b_i its mean distance to points in the next nearest cluster. Average across points and choose K to maximise.
- **Domain knowledge:** sometimes you actually know the right number of clusters from the application (e.g. “four types of customer”).

37.5 Limitations of k -means

- Assumes **spherical clusters** of similar size. Doesn’t handle elongated, curved, or highly varying-size clusters well.
- Sensitive to outliers (a single far-away point can shift a centroid).
- K must be chosen in advance.
- Initialisation matters; multiple runs are needed.

For non-spherical or noisy clusters, alternatives include **DBSCAN** (density-based), **Gaussian Mixture Models** (GMM, soft probabilistic clustering), and **hierarchical** clustering.

Practice Problem

Run K -means with $K = 2$ on the 1-D data $\{2, 4, 5, 10, 12, 13\}$. Use initial centroids $\mu_1 = 2$ and $\mu_2 = 13$.

Run two iterations and report the final centroids and clusters, plus the WCSS objective J at convergence.

Solution**Iteration 1 – assign:**

Distances to $\mu_1 = 2$: (0, 2, 3, 8, 10, 11).

Distances to $\mu_2 = 13$: (11, 9, 8, 3, 1, 0).

Assignments: cluster 1 = {2, 4, 5}, cluster 2 = {10, 12, 13}.

Iteration 1 – update:

$\mu_1 = (2 + 4 + 5)/3 = 11/3 \approx 3.67$.

$\mu_2 = (10 + 12 + 13)/3 = 35/3 \approx 11.67$.

Iteration 2 – assign:

Same assignments as before – the data clearly splits into two groups.

Converged.**WCSS:**

Cluster 1: $(2 - 3.67)^2 + (4 - 3.67)^2 + (5 - 3.67)^2 \approx 2.78 + 0.11 + 1.78 = 4.67$.

Cluster 2: $(10 - 11.67)^2 + (12 - 11.67)^2 + (13 - 11.67)^2 \approx 2.78 + 0.11 + 1.78 = 4.67$.

$J \approx 9.34$.

38 Principal Component Analysis (PCA)

Difficulty: Hard. Eigenvalues for the first time in anger. • **How to read:** Read carefully twice. Memorise the four-step procedure.

(Reminder: Eigenvalues and the 2×2 formula was introduced in Topic 4.)

Why We Care

You have data with $p = 100$ features. Most of the interesting structure probably lies in only a handful of directions; the rest is noise. PCA finds those few directions automatically. PCA is THE classical dimensionality-reduction method. It's mathematically elegant, fast, and the basis for many advanced techniques (autoencoders, factor analysis, kernel PCA).

38.1 The big idea

PCA finds an orthogonal set of new axes – the **principal components** – such that:

- The first axis (PC_1) is the direction of *maximum variance* in the data.
- The second axis (PC_2) is the direction of next-most variance, perpendicular to PC_1 .
- And so on.

After finding all p axes, you project each data point onto the top k axes – losing the “low-variance” (presumably uninteresting) dimensions.

38.2 The procedure

1. **Centre** the data: subtract the mean from every example. Let $\bar{x} = \frac{1}{n} \sum_i x_i$, set $\tilde{x}_i = x_i - \bar{x}$.
2. **Compute covariance matrix:**

$$\Sigma = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{x}_i^T = \frac{1}{n} \tilde{X}^T \tilde{X}.$$

This is a $p \times p$ matrix capturing how each pair of features co-varies.

3. **Eigendecompose** Σ :

$$\Sigma v_k = \lambda_k v_k.$$

The v_k 's are the principal directions; the λ_k 's are the variances along them. Sort so that $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$.

4. **Project** each point onto the top k components:

$$z_i = V_k^T \tilde{x}_i, \quad V_k = [v_1, v_2, \dots, v_k].$$

z_i is a k -dimensional vector summarising x_i .

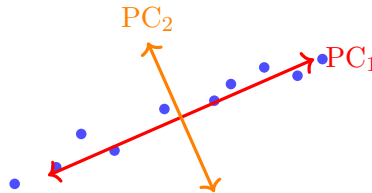
Reading This Formula

A few key things:

- Centring is **required**. Without it, the first principal component points toward the

mean, not the direction of variation.

- Eigenvalues are non-negative because Σ is positive semi-definite (since it's a covariance).
- Eigenvectors are orthogonal – this is what we mean by “new axes”.
- The projection $z_i = V_k^T \tilde{x}_i$ is just k inner products, one per component.



The data form an elongated cloud. PC_1 points along its longest direction; PC_2 is perpendicular to it.

38.3 Variance explained – how to choose k

Key Formula

$$PVE_j = \frac{\lambda_j}{\sum_{l=1}^p \lambda_l}, \quad cPVE_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_{l=1}^p \lambda_l}.$$

Reading This Formula

- PVE_j – “proportion of variance explained” by component j .
- $cPVE_k$ – cumulative version: how much of the total variance you keep when retaining the first k components.

Common rule: pick the smallest k such that $cPVE_k \geq 0.95$ (keep 95% of variance).

38.4 Eigenvalues of a 2×2 matrix (so you can do it on paper)

For $\Sigma = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$:

$$\lambda_{1,2} = \frac{(a+c) \pm \sqrt{(a-c)^2 + 4b^2}}{2}.$$

Worked Example

$$\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}.$$

$$a = 4, b = 2, c = 1.$$

$$(a+c) = 5, \quad (a-c)^2 + 4b^2 = 9 + 16 = 25, \quad \sqrt{25} = 5.$$

$$\lambda_1 = (5+5)/2 = 5, \quad \lambda_2 = (5-5)/2 = 0.$$

$PVE_1 = 5/5 = 1.0$. The first PC explains 100% of the variance – the data lies on a single

line.

Worked Example

Eigenvalues $\lambda = (4, 2, 1, 0.5, 0.5)$. Total = 8.

$\text{cPVE}_1 = 4/8 = 0.500$

$\text{cPVE}_2 = 6/8 = 0.750$

$\text{cPVE}_3 = 7/8 = 0.875$

$\text{cPVE}_4 = 7.5/8 = 0.9375 \geq 0.90$.

To explain 90% of variance, take $k = 4$ components.

38.5 Properties to remember

- Principal components are orthogonal.
- PCA is sensitive to feature scaling. When features have different units, **standardise** them first (zero mean, unit variance).
- PCA is linear – it can only find linear structure. For non-linear structure, use kernel PCA, t-SNE, or autoencoders.
- Loss of information is by design: the directions thrown away have small variance (we hope they were noise).

Exam Focus

You will be asked to:

- Eigendecompose a small (often 2×2) covariance matrix.
- Compute PVE and cumulative PVE from a list of eigenvalues.
- State why mean-centring is mandatory before PCA.
- Distinguish PCA (eigenvectors of Σ) from SVD (singular vectors of $\tilde{X}$) – they're closely related but slightly different formulations.

Practice Problem

A 2×2 centred data matrix $\tilde{X}$ has covariance matrix

$$\Sigma = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}.$$

1. Compute the eigenvalues of Σ .
2. Compute the proportion of variance explained by each principal component.
3. How many principal components would you keep to retain at least 90% of the variance?

Solution

(1) Characteristic equation: $\lambda^2 - (5 + 2)\lambda + (5 \cdot 2 - 2 \cdot 2) = \lambda^2 - 7\lambda + 6 = 0$.

$(\lambda - 6)(\lambda - 1) = 0$.

Eigenvalues: $\lambda_1 = 6$, $\lambda_2 = 1$.

(2) Total variance: $6 + 1 = 7$.

$\text{PVE}_1 = 6/7 \approx 0.857$ (about 85.7%).

$\text{PVE}_2 = 1/7 \approx 0.143$.

(3) The first PC alone gives 85.7%, less than 90%.

With two PCs: $\text{cPVE}_2 = 7/7 = 1.0$.

So we'd need both PCs to reach 90% – but in this 2-dimensional toy example, the second PC contributes a non-trivial 14.3%.

Appendix: Reference Material

A. Glossary

A plain-English definition for every technical term in this guide.

Accuracy

Fraction of correct predictions. $1 -$ average 0-1 loss.

AdaBoost

Adaptive Boosting. Sequentially trains weak classifiers, up-weighting examples misclassified by previous ones.

AIC (Akaike Information Criterion)

Training-error-based model selection criterion with penalty $2d\hat{\sigma}^2$.

Augmented input

A feature vector with a leading 1 prepended, so the intercept can be treated as another weight.

AUC

Area under the ROC curve; probability a random positive scores higher than a random negative.

Bagging

Bootstrap aggregating. Train models on bootstrap samples, then average. Reduces variance.

Base learner

An individual model used inside an ensemble.

Bayes' rule

$\mathbb{P}(A \mid B) = \mathbb{P}(B \mid A)\mathbb{P}(A)/\mathbb{P}(B)$.

Bernoulli distribution

Probability distribution over $\{0, 1\}$ with parameter $p = \mathbb{P}(\text{success})$.

Bias

Systematic error of an estimator: $\mathbb{E}[\hat{\theta}] - \theta^*$. Or, in linear models, the intercept term.

Bias-variance tradeoff

The principle that as model complexity increases, bias decreases but variance increases.

BIC (Bayesian Information Criterion)

Like AIC but with stronger penalty $(\ln n)d\hat{\sigma}^2$. Selects sparser models.

BLUE

Best Linear Unbiased Estimator. Theorem: under Gauss–Markov assumptions, OLS is BLUE.

Bootstrap

Resampling technique: draw B samples of size n with replacement to estimate the standard error of a statistic.

Boosting

Sequential ensemble method that combines weak learners by reweighting training examples.

Centroid

In k -means, the mean of points assigned to a cluster.

Characteristic polynomial

$\det(A - \lambda I) = 0$; its roots are the eigenvalues.

Classification

Supervised learning task with discrete output (label).

Cluster

A group of similar points found by an unsupervised algorithm.

Complementary slackness

KKT condition: $\lambda_i^* f_i(x^*) = 0$ for every i .

Conditional probability

$\mathbb{P}(A \mid B) = \mathbb{P}(A \cap B) / \mathbb{P}(B)$ – probability of A given that B occurred.

Confusion matrix

Table of TP, FP, FN, TN counts for a classifier.

Convex function

Graph looks like a bowl; second derivative ≥ 0 (one variable) or Hessian $\succeq 0$ (multivariable).

Convex set

Any line segment between two points in the set stays in the set.

Correlation

Standardised covariance: $\text{Cov}(X, Y) / \sqrt{\text{Var}(X) \text{Var}(Y)} \in [-1, 1]$.

Covariance

$\text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$. Measures joint variation.

Cross-entropy

Negative log-likelihood loss for classification.

Cross-validation

Estimate test error by repeatedly splitting data into train/validation folds.

Decision boundary

Surface in feature space where the classifier switches from one class to another.

Derivative

Rate of change of a function. $f'(x) = \lim_{h \rightarrow 0} (f(x+h) - f(x))/h$.

Determinant

A scalar associated with a square matrix. Zero $\iff$ matrix is singular.

Discriminative model

Models $\mathbb{P}(y \mid x)$ directly. Logistic regression, SVM.

Dot product

$\mathbf{u}^T \mathbf{v} = \sum u_i v_i$. Single number measuring alignment.

Dual problem

The “partner” optimisation problem obtained from the Lagrangian. For SVM, the variables are α_i .

Eigenvalue

Stretch factor of an eigenvector under matrix action: $A\mathbf{v} = \lambda\mathbf{v}$.

Eigenvector

Vector that gets stretched (not rotated) when a matrix acts on it.

Elastic Net

Regression with both ℓ_1 and ℓ_2 penalties.

Empirical risk

Average loss on the training set; the quantity actually minimised during training.

Ensemble

A combination of multiple models (bagging, boosting, random forest).

Expected value

$\mathbb{E}[X] = \sum x\mathbb{P}(X = x)$ (or integral for continuous). Long-run average.

F1 score

Harmonic mean of precision and recall.

Feature

An input attribute. Element of an input vector.

Feature map

Function ϕ taking raw inputs to richer representations (often higher-dimensional).

Feasible point

A point satisfying all constraints of an optimisation problem.

Gaussian distribution

Bell-curve distribution with PDF $\propto \exp(-(x - \mu)^2 / 2\sigma^2)$. Mean μ , variance σ^2 .

Gauss–Markov theorem

Under the standard linear model assumptions, OLS is the best linear unbiased estimator.

GDA

Gaussian Discriminant Analysis. Generative model assuming Gaussian class-conditional features.

Generative model

Models $\mathbb{P}(x | y)$ and $\mathbb{P}(y)$, then uses Bayes' rule for prediction. GDA, Naive Bayes.

Generalisation

Performance on data not seen during training – the only thing that ultimately matters.

Gradient

Vector of partial derivatives. Direction of fastest increase of a function.

Gradient descent

Iterative optimisation: $\theta \leftarrow \theta - \eta \nabla L(\theta)$.

Gram matrix

$K_{ij} = k(x_i, x_j)$ for a kernel k . Symmetric and PSD.

Hat matrix

$H = X(X^T X)^{-1} X^T$. Projects y onto $\hat{y}$.

Hessian

Matrix of second partial derivatives.

Hinge loss

$\max(0, 1 - y \cdot f(x))$. Loss used in SVM.

Hyperparameter

A parameter chosen by the user (or by CV) rather than learned from data. E.g. λ , C , K , η .

Hypothesis class

The set of candidate models a learning algorithm searches over.

Identity matrix

Square matrix with 1's on diagonal, 0's elsewhere. Acts as 1 for matrices.

iid Independent and identically distributed. Standard assumption on training data.

Inner product

Same as dot product (in finite-dimensional vector spaces).

Intercept

The constant (θ_0 or β_0) in a linear model.

Inverse (matrix)

A^{-1} such that $A^{-1}A = I$. Exists iff A is non-singular.

Kernel

Function $k(x, x')$ that equals an inner product in some (possibly infinite) feature space.

Kernel trick

Using a kernel to compute inner products in high-dim feature space without materialising features.

KKT conditions

Karush–Kuhn–Tucker. Four conditions characterising optimality in constrained optimisation.

Lagrangian

$\mathcal{L}(x, \lambda, \nu) = f_0(x) + \sum \lambda_i f_i(x) + \sum \nu_j h_j(x)$.

Laplace smoothing

Add-one trick that prevents zero probabilities in Naive Bayes.

LASSO

Least Absolute Shrinkage and Selection Operator. Regression with ℓ_1 penalty. Produces sparse solutions.

Leverage

h_{ii} – diagonal of the hat matrix. How much an observation affects its own fitted value.

Likelihood

$\mathcal{L}(\theta) = p(\text{data} \mid \theta)$. Treated as a function of θ .

Linearly separable

A data set is linearly separable if some hyperplane perfectly separates the two classes.

Logistic regression

Discriminative classifier modelling $\mathbb{P}(y = 1 \mid x) = \sigma(\theta^T x)$.

LOOCV

Leave-One-Out CV. k -fold CV with $k = n$.

MAP estimation

Maximum a Posteriori. Maximises posterior $p(\theta \mid \text{data})$.

Margin

Distance from a separating hyperplane to the nearest training point.

Matrix

Rectangular grid of numbers.

Mean

Same as expected value.

MLE

Maximum Likelihood Estimation. Choose θ to maximise the likelihood of observed data.

MSE

Mean Squared Error. $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$.

Multicollinearity

Two or more features carrying nearly the same information; makes $X^T X$ near-singular.

Naive Bayes

Generative classifier assuming features are independent given the class.

Norm

Length of a vector. $\|v\|_2 = \sqrt{\sum v_i^2}$.

Normal distribution

Same as Gaussian.

Normal equations

$X^T X \theta = X^T y$. Setting the OLS gradient to zero.

OLS

Ordinary Least Squares. Minimises sum of squared residuals. Closed form: $(X^T X)^{-1} X^T y$.

OOB

Out-of-bag. Examples not picked in a bootstrap sample (37%).

Optimum

Point where an objective is minimised (or maximised). Local: in a neighbourhood; Global: anywhere.

Overfitting

Model fits training data too closely – low training error, high test error.

p-value

Probability, under H_0 , of observing a test statistic at least as extreme as the one we computed.

Partial derivative

Derivative w.r.t. one variable, holding others fixed.

PCA

Principal Component Analysis. Finds orthogonal directions of maximum variance.

Perceptron

Simple linear classifier. Updates weights on misclassified examples.

Policy

In RL, a strategy mapping states to actions.

Positive (semi-)definite

Symmetric matrix with all eigenvalues > 0 (≥ 0).

Posterior

$p(\theta \mid \text{data})$ – updated belief about θ after seeing data.

Precision

$TP / (TP + FP)$. Of those flagged positive, how many are correct.

Primal

The original constrained optimisation problem (vs. its dual).

Principal component

Eigenvector of the covariance matrix; a direction of variance.

Prior

$p(\theta)$ – belief about θ before seeing data.

PVE

Proportion of Variance Explained. $\lambda_j / \sum_l \lambda_l$ in PCA.

Random forest

Ensemble of bagged decision trees with random feature subsampling at each split.

Recall

$TP / (TP + FN)$. Of all true positives, how many we caught. AKA TPR.

Regression

Supervised learning task with continuous output.

Regularisation

Adding a penalty to the loss to discourage complex models. E.g. Ridge, LASSO.

Residual

$y_i - \hat{y}_i$ – difference between observation and prediction.

Reward

In RL, the scalar feedback signal from the environment.

Ridge regression

Regression with ℓ_2 penalty. Closed form: $(X^T X + \lambda I)^{-1} X^T y$.

ROC curve

Plot of TPR vs FPR as classifier threshold varies.

RSS

Residual Sum of Squares: $\sum (y_i - \hat{y}_i)^2$.

R^2 $1 - \text{RSS}/\text{TSS}$. Fraction of variance explained.

Scalar

A single number (as opposed to a vector or matrix).

SGD

Stochastic Gradient Descent. Updates using one example at a time.

Sigmoid function

$\sigma(z) = 1/(1 + e^{-z})$. Squashes any real number into $(0, 1)$.

Singular matrix

Square matrix without an inverse. Determinant = 0.

Slack variable

ξ_i in soft-margin SVM; allows margin violations.

Soft thresholding

LASSO solution operator: $\text{sign}(z) \max(|z| - \lambda/2, 0)$.

Standard error

Standard deviation of an estimator.

State

In RL, the current situation.

Stationarity

KKT condition: gradient of the Lagrangian = 0 at the optimum.

Stepwise selection

Greedy search adding (forward) or removing (backward) features one at a time.

Supervised learning

Learning from labelled examples (x_i, y_i) .

Support vector

A training point with $\alpha_i > 0$ in the SVM dual; sits on or inside the margin.

SVM

Support Vector Machine. Maximum-margin classifier, possibly with kernels.

 t -statistic

$\hat{\theta}/\widehat{\text{SE}}(\hat{\theta})$. Used to test if a coefficient is significantly non-zero.

Test set

Held-out data used only to evaluate a trained model.

Training set

Data used to fit the model.

Transpose

A^T : matrix with rows and columns swapped.

TSS

Total Sum of Squares: $\sum (y_i - \bar{y})^2$.

Type I error

Rejecting H_0 when it's true. Probability α .

Type II error

Failing to reject H_0 when it's false. Probability β .

Underfitting

Model too simple to capture data structure – high bias.

Unsupervised learning

Learning from unlabelled data.

Variance

Spread of an estimator (or random variable) around its mean.

Vector

List of numbers, often written as a column.

VIF

Variance Inflation Factor. Diagnostic for multicollinearity.

WCSS

Within-Cluster Sum of Squares. k -means objective.

B. Formula Cheat Sheet

Linear models

$$\begin{aligned}\hat{\theta}_{\text{OLS}} &= (X^T X)^{-1} X^T y \\ \hat{\theta}_{\text{ridge}} &= (X^T X + \lambda I)^{-1} X^T y \\ R^2 &= 1 - \text{RSS}/\text{TSS} \\ \bar{R}^2 &= 1 - (1 - R^2) \frac{n-1}{n-d-1}\end{aligned}$$

Optimisation

$$\begin{aligned}\theta^{(t+1)} &\leftarrow \theta^{(t)} - \eta \nabla L(\theta^{(t)}) \\ \sigma(z) &= 1/(1 + e^{-z})\end{aligned}$$

Classification

$$\begin{aligned}\text{SVM dual : } \max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{Naive Bayes : } \hat{y} = \arg \max_k [\ln \mathbb{P}(y = k) + \sum_j \ln \mathbb{P}(x_j \mid y = k)]\end{aligned}$$

Resampling and selection

$$\begin{aligned}\text{CV}_{(k)} &= \frac{1}{k} \sum_{j=1}^k \text{MSE}_j \\ \text{OOB fraction} &\rightarrow e^{-1} \approx 0.368 \\ \text{AIC} &= \frac{1}{n\hat{\sigma}^2} (\text{RSS} + 2d\hat{\sigma}^2) \\ \text{BIC} &= \frac{1}{n} (\text{RSS} + (\ln n)d\hat{\sigma}^2)\end{aligned}$$

Ensembles

$$\begin{aligned}\alpha_t &= \frac{1}{2} \ln \frac{1 - \varepsilon_t}{\varepsilon_t} \\ w_i^{(t+1)} &\propto w_i^{(t)} e^{-\alpha_t y_i h_t(x_i)} \\ \text{Var}(\hat{f}_{\text{bag}}) &= \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2\end{aligned}$$

Unsupervised

$$J_{\text{kmeans}}(c, \mu) = \sum_i \|x_i - \mu_{c(i)}\|^2$$

$$\Sigma v_k = \lambda_k v_k, \quad z_i = V_k^T (x_i - \bar{x})$$

$$\text{cPVE}_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_l \lambda_l}$$

C. Common Confusions Recap

Wrong claim	Correct view
Zero training error means a perfect model.	Likely overfitting; what matters is test error.
Training RSS lets you compare models with different feature counts.	It does not – R^2 is monotone in d . Use CV / AIC / BIC.
Bagging reduces bias.	Bagging reduces variance. Bias is essentially unchanged.
LASSO shrinks coefficients but never to zero.	LASSO does set some to exactly zero. That's its key property.
Naive Bayes requires independence to be literally true.	It works surprisingly well even when the assumption is wrong.
A high R^2 implies good prediction on new data.	It only measures fit on training data.
PCA can be applied without centring the data.	Without centring, the first PC degenerates to the mean direction.
k -means converges to the global minimum.	Only to a local minimum. Run multiple random restarts.
Convex losses guarantee good predictions.	Convexity guarantees you find the global minimum of the <i>loss</i> , not that the loss reflects what you actually care about.

D. Pre-Exam Checklist

Foundations

- ☐ State and recall $\hat{\theta}_{\text{OLS}} = (X^T X)^{-1} X^T y$.
- ☐ Compute R^2 and $\bar{R}^2$ from RSS, TSS, n , d .
- ☐ Explain why Gaussian noise implies MLE = OLS.
- ☐ Match priors on θ to regularisation penalties.

Model selection

- ☐ Decompose expected MSE = Bias² + Var + Irreducible.
- ☐ Walk through forward / backward stepwise on a tiny dataset.
- ☐ Average k -fold CV errors.
- ☐ Compute bootstrap variance from a list of $\hat{\alpha}^{*b}$ values.
- ☐ Compare AIC, BIC, $\bar{R}^2$.

Optimisation

- ☐ Perform 1–2 gradient-descent steps by hand.
- ☐ Test convexity via $f''(x) \geq 0$ or PSD Hessian.
- ☐ Write SVM primal and dual; apply KKT.

Classification

- ☐ Trace perceptron updates; identify when it can't converge.
- ☐ From an α table, identify support vectors and compute w, b .
- ☐ Evaluate polynomial / RBF kernels for given x_i, x_j .
- ☐ State why kernel SVM handles non-linear boundaries.

Regularisation

- ☐ Recall ridge and LASSO objectives.
- ☐ Apply soft thresholding for LASSO.
- ☐ Compute a kernel Gram matrix.
- ☐ Pick ridge or LASSO based on a problem description.

Probabilistic classification

- ☐ Compute $\sigma(z)$ for given z .
- ☐ Estimate GDA parameters by MLE.
- ☐ Apply Naive Bayes with Laplace smoothing.

Ensembles

- ☐ State the bagging variance formula.
- ☐ Recall RF defaults: $m = \sqrt{p}$ classification, $p/3$ regression.
- ☐ Compute one round of AdaBoost: $\varepsilon_t \rightarrow \alpha_t \rightarrow$ weights.

Unsupervised

- ☐ Run k -means on a small toy dataset.
- ☐ Compute eigenvalues of a 2×2 covariance matrix.
- ☐ State why mean-centring is mandatory before PCA.
- ☐ Compute PVE and cPVE.

End of guide. Good luck.